

7.1 引言 7.2 性能分析 7.3 单周期处理器 7.4 多周期处理器 7.5 流水线处理器 7.6 HDL表示\* 7.7 高级微架构\* 7.8 现实视角：RISC-V微架构的演进\* 7.9 总结 习题 面试题

7.1 引言

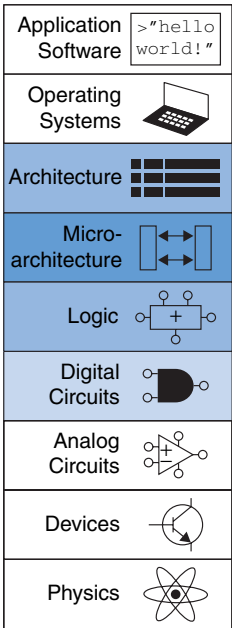
在本章中，你将学习如何拼装出一个微处理器。实际上，你将逐步构建三种不同的版本，每种版本在性能、成本和复杂性之间各有不同的权衡。

对于外行人来说，构建微处理器可能看似玄学。但实际上它相对直接，并且至此，您已掌握了所需的一切知识。具体而言，您已学会根据功能和时序规范设计组合逻辑和时序逻辑。您熟悉算术和存储电路。此外，您还了解了RISC-V架构，该架构从寄存器、指令和内存的角度规定了RISC-V处理器的程序员视角。

本章涵盖microarchitecture，即逻辑与架构之间的联系。微架构是为实现某种架构所需的寄存器、算术逻辑单元（ALU）、有限状态机（FSM）、存储器及其他逻辑构建模块的具体排列方式。一个特定的架构（如RISC-V）可能拥有多种不同的微架构，每种微架构在性能、成本和复杂度方面各有不同的权衡。它们都能运行相同的程序，但内部设计却千差万别。本章将设计三种不同的微架构，以说明这些权衡。

7.1.1 架构状态与指令集

回顾一下，计算机体系结构由其指令集和体系结构状态定义。RISC-V处理器的architectural state包括程序计数器和32个32位寄存器。任何RISC-V微架构都必须包含所有这些状态。基于当前的体系结构状态，处理器使用特定的数据集执行特定指令，以产生新的体系结构状态。某些



*architectural state* 是定义计算机当前运行状态所需的信息。如果保存一份架构状态和内存内容的副本，然后关闭计算机，再重新启动并恢复架构状态和内存，计算机将从中断处继续运行程序，仿佛从未经历过断电与重启。这就像科幻小说中主角的大脑被冷冻，多年后解冻并在新世界中苏醒的场景。

微架构包含额外的 *nonarchitectural state*，以简化逻辑或提升性能；我们将在出现时指出这一点。

为了使微架构易于理解，我们专注于RISC-V指令集的一个子集。具体来说，我们处理以下指令：

R型指令：add、sub、and、or、slt ▶ 内存指令：lw、sw ▶ 分支指令：beq

选择这些特定指令是因为它们足以编写有用的程序。一旦你理解了如何实现这些指令，就可以扩展硬件以处理其他指令。

7.1.2 设计过程

我们将微架构分为两个相互交互的部分：*data-path* 和 *control unit*。数据通路对数据字进行操作，包含存储器、寄存器、算术逻辑单元（ALU）和多路选择器等结构。我们实现的是32位RISC-V（RV32I）架构，因此使用32位数据通路。控制单元从数据通路接收当前指令，并指示数据通路如何执行该指令。具体来说，控制单元产生多路选择器选择信号、寄存器使能信号和存储器写信号，以控制数据通路的操作。

设计复杂系统的一个好方法是从包含状态元素的硬件开始。这些元素包括存储器和架构状态（程序计数器和寄存器）。然后，在状态元素之间添加组合逻辑块，根据当前状态计算新状态。指令从内存的一部分读取；加载和存储指令则从内存的另一部分读取或写入数据。因此，通常将整体内存划分为两个较小的内存，一个包含指令，另一个包含数据，这样更为方便。图7.1展示了一个包含四个状态元素的框图：程序计数器、寄存器文件以及指令和数据存储器。

至少，程序计数器必须有一个复位信号，以便在处理器启动时初始化其值。复位时，RISC-V 处理器通常将 PC 初始化为内存中的一个低地址，例如 0x00001000，我们从此处开始运行程序。

本章中，粗线表示32位数据总线，中粗线表示较窄的总线（如寄存器文件上的5位地址总线），细线表示1位线，蓝色线用于控制信号（如寄存器文件写使能）。寄存器通常具有复位输入，以便在启动时将其置于已知状态，但为减少杂乱，复位信号未予显示。

*program counter*（PC）指向当前指令。其输入 *PCNext* 指示下一条指令的地址。

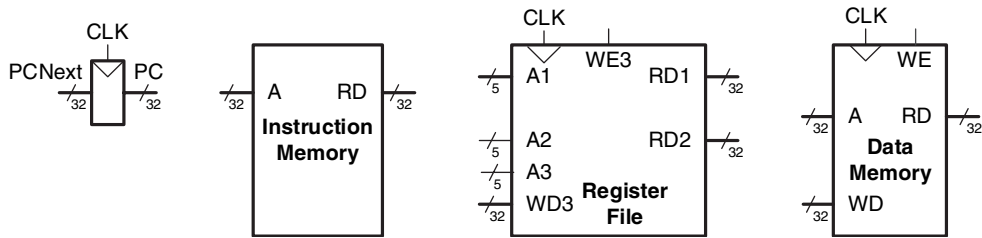


图7.1 RISC-V处理器的状态元素

*instruction memory* 具有单个读取端口。<sup>1</sup> 它接收一个32位的指令地址输入  $A$ ，并从该地址读取32位数据（即指令）到读取数据输出  $RD$ 。

32个元素的 $\times 32$ 位寄存器文件包含寄存器 $x0-x31$ 。请注意， $x0$ 被硬连线为0。该寄存器文件有两个读端口和一个写端口。读端口接收5位地址输入 $A1$ 和 $A2$ ，每个地址指定 $2^5 = 32$ 个32位寄存器中的一个作为源操作数。寄存器文件将32位寄存器值输出到读数据输出 $RD1$ 和 $RD2$ 。写端口（端口3）接收5位地址输入 $A3$ 、32位写数据输入 $WD3$ 、写使能输入 $WE3$ 以及时钟。如果其写使能( $WE3$ )被断言，则寄存器文件在时钟上升沿将数据( $WD3$ )写入指定的寄存器( $A3$ )。

*data memory* 具有单个读/写端口。如果其写使能信号  $WE$  被置位，则在时钟上升沿将数据  $WD$  写入地址  $A$ 。如果其写使能为0，则从地址  $A$  读取数据到读数据总线  $RD$ 。

指令存储器、寄存器文件和数据存储器均为读取型 *combinationally*。换言之，当地址发生变化时，经过一定的传播延迟后，新数据会在  $RD$  处出现，无需时钟参与。时钟仅控制写入操作。这些存储器仅在时钟上升沿进行写入。通过这种方式，系统状态仅在时钟边沿发生变化。地址、数据和写使能信号必须在时钟边沿之前建立，并在时钟边沿之后保持稳定直至保持时间结束。

由于状态元素仅在时钟上升沿改变状态，因此它们是同步时序电路。微处理器

<sup>1</sup> This is an oversimplification used to treat the instruction memory as a ROM. In most real processors, the instruction memory must be writable so that the operating system (OS) can load a new program into memory. The multicycle microarchitecture described in Section 7.4 is more realistic in that it uses a single memory that contains both instructions and data and that can be both read and written.

由时钟状态元件和组合逻辑构成，因此它也是一个同步时序电路。实际上，处理器可以被视为一个巨大的有限状态机，或者一组相互作用的简单状态机的集合。

### 7.1.3 微架构

在本章中，我们为RISC-V架构开发了三种微架构：单周期、多周期和流水线。它们在状态元件的连接方式以及所需的非架构状态数量上有所不同。

*single-cycle microarchitecture* 在一个周期内执行完整指令。它易于解释且控制单元简单。由于它在一个周期内完成操作，因此不需要任何非架构状态。然而，周期时间受限于最慢的指令。此外，处理器需要独立的指令存储器和数据存储器，这通常是不现实的。

经典多周期处理器的例子包括1947年麻省理工学院的Whirlwind、IBM System/360、数字设备公司的VAX、Apple II中使用的6502以及IBM PC中使用的8088。多周期微架构仍被用于廉价的微控制器中，例如8051、68HC11以及在家电、玩具和小装置中常见的PIC16系列。

*multicycle microarchitecture*通过一系列较短的周期执行指令。简单指令比复杂指令所需的周期更少。此外，多周期微架构通过复用加法器和寄存器等昂贵的硬件模块来降低硬件成本。例如，在执行单条指令时，加法器可在不同周期用于多种用途。多周期微处理器通过引入多个非架构寄存器来保存中间结果，从而实现这一目标。多周期处理器一次只执行一条指令，但每条指令需要多个时钟周期。该处理器仅需单个存储器，在一个周期中访问它以获取指令，在另一个周期中读取或写入数据。由于比单周期处理器使用的硬件更少，多周期处理器在历史上是低成本系统的选择。

自1989年80486处理器问世以来，英特尔处理器一直采用流水线技术。几乎所有RISC微处理器也都采用流水线设计，而所有商用RISC-V处理器同样如此。随着晶体管成本持续下降，流水线处理器如今成本不足一美分，包含内存和外设的完整系统也仅需数十美分。因此，即便在最注重成本的应用场景中，流水线处理器也正在取代速度较慢的多周期处理器。

*pipelined microarchitecture*将流水线技术应用于单周期微架构，因此能够同时执行多条指令，显著提升吞吐量。流水线必须增加逻辑以处理同时执行指令间的依赖关系，同时还需要非架构性的流水线寄存器。流水线处理器需在同一周期内访问指令和数据，为此通常采用独立的指令缓存和数据缓存（详见第8章）。增加的逻辑和寄存器是值得的——当今所有商用高性能处理器均采用流水线技术。

我们在后续章节中探讨这三种微架构的细节和权衡。在本章末尾，我们简要提及用于在现代高性能微处理器中实现更高速度的其他技术。

## 7.2 性能分析

正如我们提到的，特定的处理器架构可以拥有许多微架构，它们在成本和性能之间有不同的权衡。成本取决于所需的硬件数量以及实现技术。精确的成本计算需要对实现技术有详细的了解，但通常来说，更多的门电路和更多的内存意味着更高的成本。

本节为性能分析奠定基础。衡量计算机系统性能的方法有很多，而市场营销部门因倾向于选择能让自家计算机看起来最快的测量方式而臭名昭著，全然不顾该测量结果是否与实际性能相关。例如，微处理器制造商常以时钟频率和核心数量来推销产品。然而，他们却回避了以下复杂问题：某些处理器在一个时钟周期内完成的工作量多于其他处理器，且这种差异因程序而异。那么，买家该如何抉择？

衡量性能的唯一无噱头方式是通过测量你感兴趣的程序的执行时间。能最快执行你程序的计算机性能最高。次优选择是测量一组与你计划运行的程序相似的程序的总体执行时间。如果你尚未编写自己的程序，或者没有你程序的其他人正在进行测量，这可能是必要的。这类程序集合被称为**benchmarks**，这些程序的执行时间通常会被公布，以表明处理器的性能表现。

Dhrystone、CoreMark和SPEC是三种流行的基准测试。前两者 *synthetic benchmarks* 由重要的常见程序片段组成。Dhrystone于1984年开发，至今仍常用于嵌入式处理器，尽管其代码与真实程序有些脱节。CoreMark是对Dhrystone的改进，涉及矩阵乘法以测试乘法器和加法器、链表以测试内存系统、状态机以测试分支逻辑，以及循环冗余校验以测试处理器的多个部分。这两个基准测试的大小均小于16 KB，不会对指令缓存造成压力。

来自标准性能评估公司（SPEC）的SPECspeed 2017整数基准测试由真实程序组成，包括x264（视频压缩）、deepsjeng（人工智能国际象棋程序）、omnetpp（仿真）和GCC（C语言编译器）。该基准测试广泛应用于高性能处理器，因为它以具有代表性的方式对整个系统施加压力。

当客户根据基准测试购买计算机时，必须谨慎行事，因为计算机制造商有强烈的动机去操纵基准测试。例如，Dhrystone测试涉及大量字符串复制，但这些字符串具有已知的固定长度和字对齐特性。因此，智能编译器可能会将通常包含循环和字节访问的代码替换为一系列字加载和存储操作，从而使Dhrystone得分提升超过30%，但并未加速实际应用。SPEC89基准测试中包含一个Matrix 300程序，其中99%的执行时间集中在一行代码上。IBM通过一种名为**blocking**的编译器技术，将该程序的速度提升了9倍。多核计算的基准测试更加困难，因为编写程序的方式多种多样：有些程序能随可用核心数量成比例加速，但在单核上效率低下；另一些程序在单核上运行快速，却几乎无法从额外核心中获益。

公式7.1给出了程序的执行时间，以秒为单位。

$$ExecutionTime = (\#instructions) \left( \frac{cycles}{instruction} \right) \left( \frac{seconds}{cycle} \right) \quad (7.1)$$

程序中指令的数量取决于处理器架构。某些架构包含复杂的指令，每条指令能完成更多工作，从而减少程序中的指令总数。然而，这些复杂指令在硬件中执行速度通常较慢。指令数量还极大地取决于程序员的编程技巧。为便于本章讨论，我们假设在RISC-V处理器上执行已知程序，因此每个程序的指令数量是恒定的，与微架构无关。

*cycles per instruction* (CPI) 是执行一条平均指令所需的时钟周期数，它是吞吐量 (*instructions per cycle* 或 IPC) 的倒数。不同的微架构具有不同的CPI。在本章中，我们假设存在一个不影响CPI的理想内存系统。在第8章中，我们将探讨处理器有时需要等待内存的情况，这会导致CPI增加。

每个周期所对应的秒数即为时钟周期， $T_c$ 。时钟周期由处理器中逻辑电路的关键路径决定。不同的微架构具有不同的时钟周期。逻辑设计与电路设计也会显著影响时钟周期。例如，超前进位加法器比行波进位加法器速度更快。制造工艺的进步同样提升了晶体管速度，因此即便微架构与逻辑设计保持不变，当今制造的微处理器也会比十年前的产品速度更快。

微架构师的挑战在于选择一种设计，在满足成本和/或功耗约束的同时最小化执行时间。由于微架构决策会影响CPI和 $T_c$ ，并且受到逻辑和电路设计的影响，因此确定最佳选择需要仔细分析。

许多其他因素也会影响计算机的整体性能。例如，硬盘、内存、图形系统和网络连接可能成为限制因素，使得处理器性能变得无关紧要。即使使用世界上最快的微处理器，也无法在连接质量差的情况下改善上网体验。但这些其他因素超出了本书的讨论范围。

### 7.3 单周期处理器

我们首先设计一种单周期执行指令的微架构。我们通过连接状态元件开始构建数据通路。

图7.1中的元件，包含可执行各种指令的组合逻辑。控制信号决定数据通路在任意时刻执行哪条具体指令。控制单元包含基于当前指令生成相应控制信号的组合逻辑。最后，我们分析单周期处理器的性能。

### 7.3.1 示例程序

为具体起见，我们将让单周期处理器运行图7.2中的短程序，该程序涉及加载、存储、一条R型指令（or）以及一条分支指令（beq）。假设该程序从地址0x1000开始存储在内存中。图中标明了每条指令的地址、指令类型、指令字段以及该指令的十六进制机器语言代码。

假设寄存器 x5 初始值为 6，x9 包含 0x2004。内存地址 0x2000 包含值 10。程序计数器从 0x1000 开始。lw 从地址  $(0x2004 - 4) = 0x2000$  读取 10 并将其存入 x6。sw 将 10 写入地址  $(0x2004 + 8) = 0x200C$ 。or 计算  $x4 = 6 \mid 10 = 0110_2 \mid 1010_2 = 1110_2 = 14$ 。然后，beq 跳转回标签 L7，因此程序无限循环。

### 7.3.2 单周期数据通路

本节逐步构建单周期数据通路，每次在图7.1的状态元件基础上增加一个部分。新增连接以黑色（或蓝色表示新增控制信号）突出显示，而已经研究过的硬件则以灰色呈现。每幅图底部展示了正在执行的示例指令。

程序计数器包含待执行指令的地址。第一步是从指令存储器中读取该指令。图7.3显示，PC直接连接到指令存储器的地址输入端。指令存储器读出（即*fetches*）32位指令，标记为*Instr*。在图7.2的示例程序中，PC为0x1000。（注意，这是一个32位处理器，因此PC实际为0x00001000，但为避免图表杂乱，我们省略了前导零。）

我们在正文中将信号名称用斜体表示，但硬件模块名称不采用斜体。例如，PC是来自PC寄存器的信号，或简称为PC。

| Address | Instruction       | Type | Fields                |       |       |       |                       |          | Machine Language |
|---------|-------------------|------|-----------------------|-------|-------|-------|-----------------------|----------|------------------|
| 0x1000  | L7: lw x6, -4(x9) | I    | imm <sub>11:0</sub>   | rs1   | f3    | rd    | op                    |          |                  |
|         |                   |      | 1111111111100         | 01001 | 010   | 00110 | 0000011               | FFC4A303 |                  |
| 0x1004  | sw x6, 8(x9)      | S    | imm <sub>11:5</sub>   | rs2   | rs1   | f3    | imm <sub>4:0</sub>    | op       |                  |
|         |                   |      | 00000000              | 00110 | 01001 | 010   | 01000                 | 0100011  | 0064A423         |
| 0x1008  | or x4, x5, x6     | R    | funct7                | rs2   | rs1   | f3    | rd                    | op       |                  |
|         |                   |      | 00000000              | 00110 | 00101 | 110   | 00100                 | 0110011  | 0062E233         |
| 0x100C  | beq x4, x4, L7    | B    | imm <sub>12:0:5</sub> | rs2   | rs1   | f3    | imm <sub>4:1:11</sub> | op       |                  |
|         |                   |      | 1111111               | 00100 | 00100 | 000   | 10101                 | 1100011  | FE420AE3         |

图7.2 示例程序执行不同类型的指令



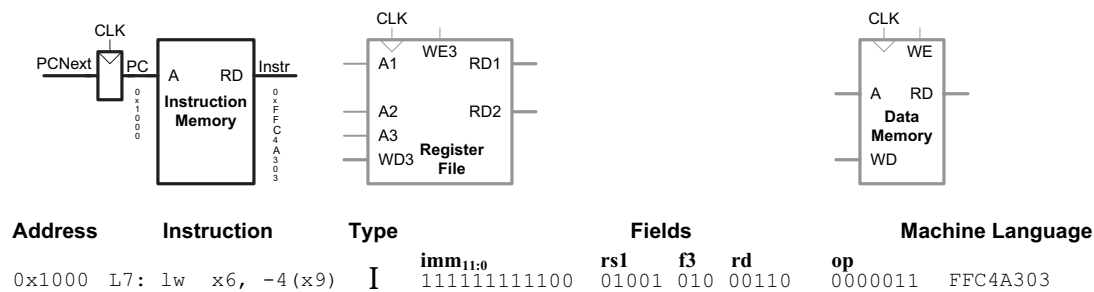


Figure 7.3 Fetch instruction from memory

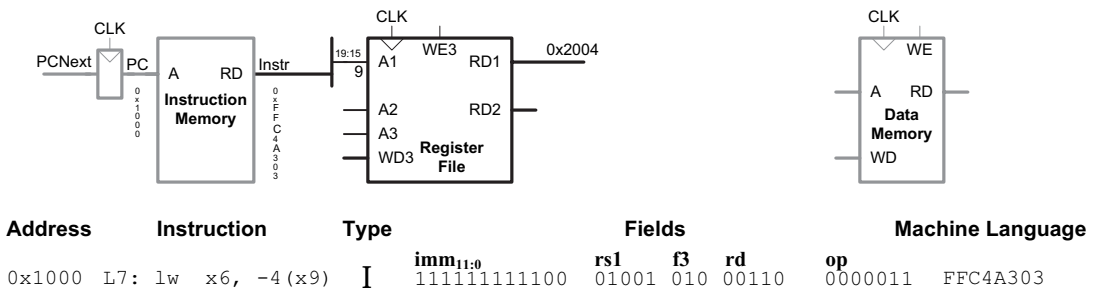


图7.4 从寄存器文件读取源操作数

*Instr*如图7.3底部所示，lw, 0xFFC4A303 这些样本值在图中以浅蓝色标注。

处理器的操作取决于所获取的具体指令。首先，我们将为 lw 指令设计数据路径连接。然后，我们将考虑如何泛化数据路径以处理其他指令。

lw

对于 lw 指令，下一步是读取包含基地址的源寄存器。回顾一下，lw 是 I 型指令，基寄存器在指令的 rs1 字段  $Instr_{19:15}$  中指定。这些指令位连接到寄存器文件的 A1 地址输入，如图 7.4 所示。寄存器文件将寄存器值读取到 RD1 上。在我们的示例中，寄存器文件从 x9 读取 0x2004。

lw 指令也需要一个偏移量。该偏移量存储在指令的 12 位立即数字段  $Instr_{31:20}$  中。它是一个有符号值，因此必须符号扩展为 32 位。符号扩展仅意味着将符号位复制到最高有效位： $ImmExt_{31:12} = Instr_{31}$  和  $ImmExt_{11:0} = Instr_{31:20}$ 。符号扩展由



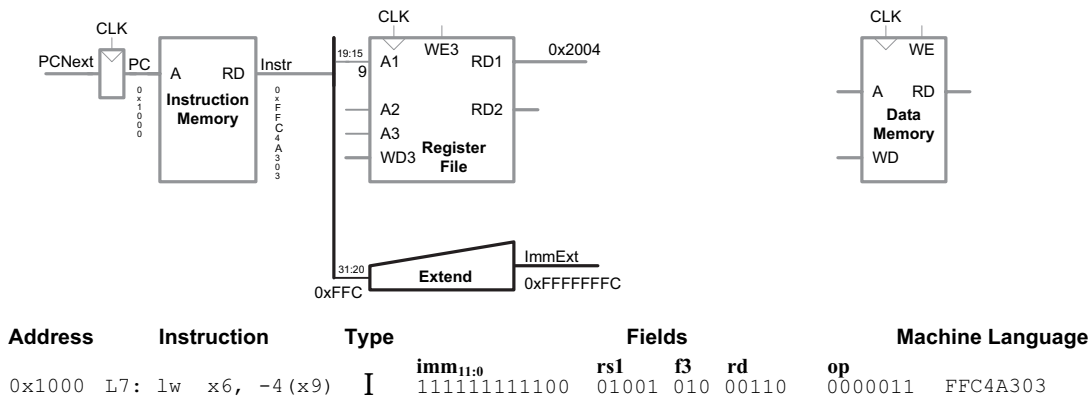


Figure 7.5 Sign-extend the immediate

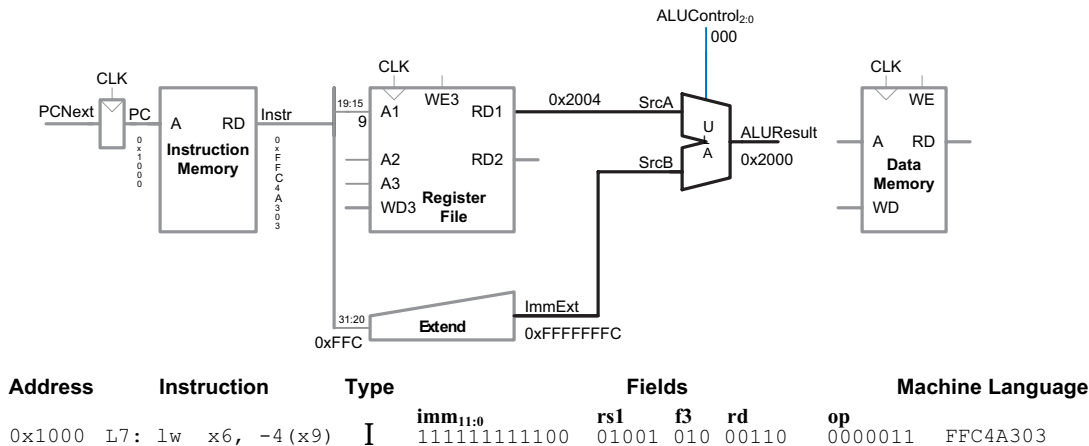


图7.6 计算内存地址

一个扩展单元，如图7.5所示，它接收 $Instr_{31:20}$ 中的12位有符号立即数，并生成32位符号扩展的立即数 $ImmExt$ 。在我们的示例中，二进制补码立即数-4从其12位表示0xFFC扩展为32位表示0xFFFFFFF0。

处理器将基址与偏移量相加，以找到从内存读取的地址。图7.6引入了一个ALU来执行此加法操作。该ALU接收两个操作数， $SrcA$ 和 $SrcB$ 。 $SrcA$ 是来自寄存器文件的基址， $SrcB$ 是来自符号扩展立即数 $ImmExt$ 的偏移量。如第5.2.4节所述，ALU可以执行多种操作。3位 $ALUControl$ 信号

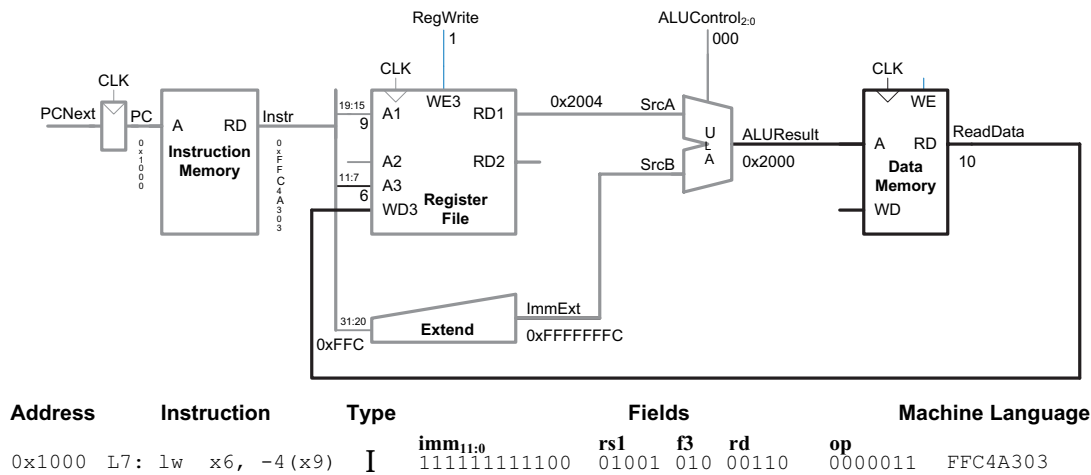


图7.7 读取存储器并将结果写回寄存器文件

指定操作（见第250页表5.3）。ALU接收32位操作数并生成32位 *ALUResult*。对于 `lw` 指令，*ALUControl* 应设置为000以执行加法。*ALUResult* 被发送到数据存储器作为读取地址，如图7.6所示。在我们的示例中，ALU计算  $0x2004 + 0xFFFFF4A303 = 0x2000$ 。同样，这是一个32位值，但为了不使图面杂乱，我们省略了前导零。

来自ALU的该内存地址被提供给数据存储器的地址（A）端口。数据从数据存储器读出到 *ReadData* 总线，然后在周期结束时写回目标寄存器，如图7.7所示。寄存器文件的端口3是写端口。`lw` 指令的目标寄存器（由 `rd` 字段 *Instr<sub>11:7</sub>* 指示）连接到A3（端口3的地址输入）。*ReadData* 总线连接到WD3（端口3的写数据输入）。一个名为 *RegWrite*（寄存器写）的控制信号连接到WE3（端口3的写使能输入），并在 `lw` 指令期间被断言，以便将数据值写入寄存器文件。写入操作发生在周期结束时的时钟上升沿。在我们的示例中，处理器从数据存储器的地址0x2000读取10，并将该值（10）放入寄存器文件的x6中。

在执行指令期间，处理器还必须计算下一条指令的地址 *PCNext*。由于指令为32位（4字节），下一条指令位于  $PC+4$ 。图7.8使用加法器将PC增加4。在我们的示例中， $PCNext = 0x1000 + 4 = 0x1004$ 。新地址将在时钟的下一个上升沿写入程序计数器。至此，`lw` 指令的数据通路完成。

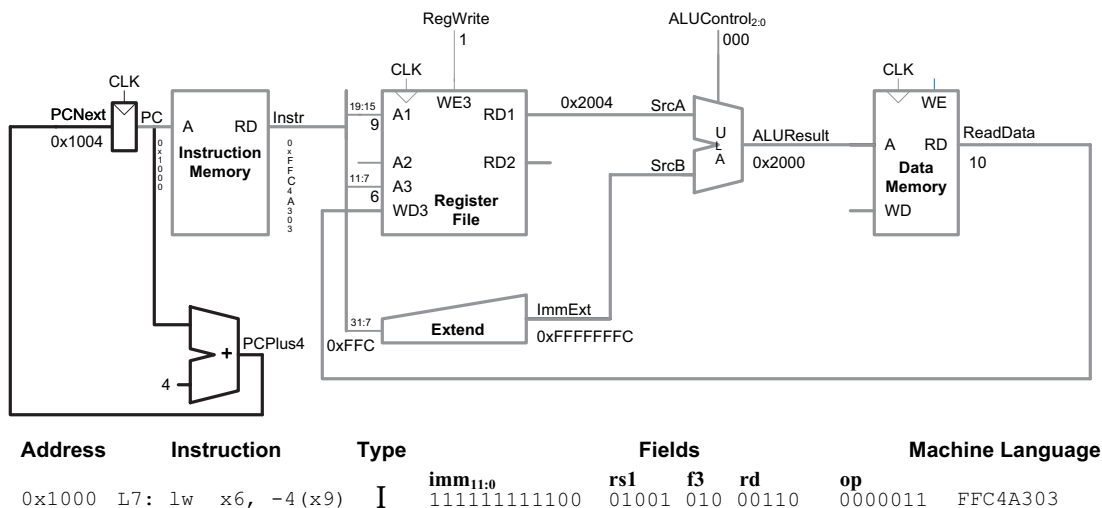


图 7.8 程序计数器递增

sw

接下来，让我们扩展数据通路以处理sw指令，这是一种S型指令。与lw类似，sw从寄存器文件的端口1读取基地址，并对立即数进行符号扩展。ALU将基地址与立即数相加，以找到内存地址。数据通路中已支持所有这些功能，但12位有符号立即数存储在 $Instr_{31:25,11:7}$ 中（而非lw中的 $Instr_{31:20}$ ）。因此，必须修改Extend单元，使其也能接收这些额外的位 $Instr_{11:7}$ 。为简化起见（并为后续指令如jal做准备），Extend单元接收 $Instr_{31:7}$ 的所有位。控制信号ImmSrc决定使用哪些指令位作为立即数。当ImmSrc = 0（对应lw）时，Extend单元选择 $Instr_{31:20}$ 作为12位有符号立即数；当ImmSrc = 1（对应sw）时，则选择 $Instr_{31:25,11:7}$ 。

sw指令还会从寄存器文件中读取第二个寄存器，并将其内容写入数据存储器。图7.9展示了为实现这一新增功能而添加的新连接。该寄存器由rs2字段 $Instr_{24:20}$ 指定，该字段连接到寄存器文件的地址2（A2）输入。寄存器的内容被读取到读数据2（RD2）输出，该输出又连接到数据存储器的写数据（WD）输入。数据存储器的写使能端口WE由MemWrite控制。对于sw指令：MemWrite = 1表示将数据写入存储器；ALUControl = 000表示将基地址与偏移量相加；RegWrite = 0表示不应向寄存器文件写入任何内容。请注意，数据仍然从数据存储器给定的地址读取，但此ReadData被忽略，因为RegWrite = 0。

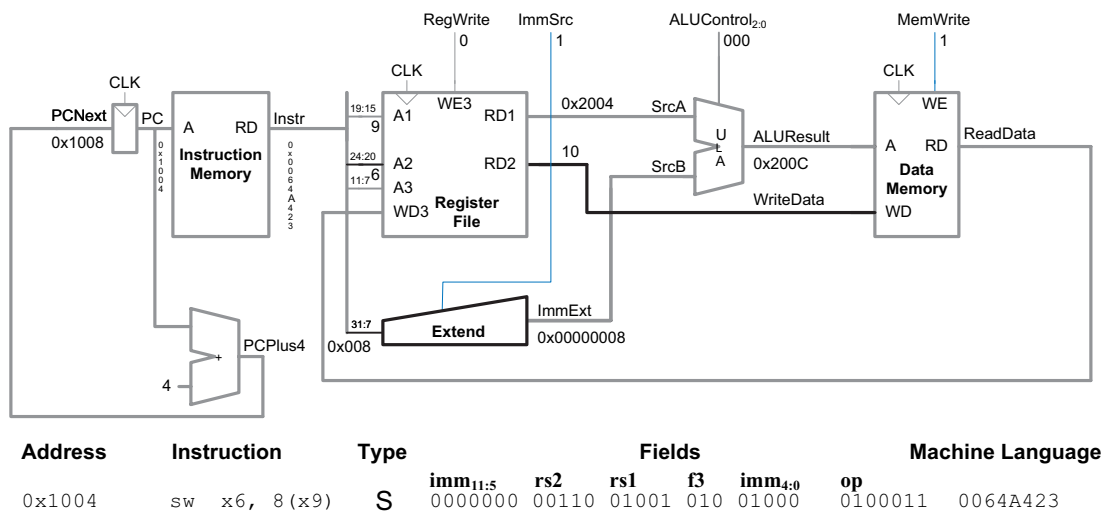


图7.9 将数据写入存储器以执行sw指令

在我们的示例中，PC为0x1004。因此，指令存储器读出`sw`指令0x0064A423。寄存器文件从x9读出0x2004（基地址），从x6读出10，同时扩展单元将立即数偏移量8从12位扩展至32位。ALU计算 $0x2004 + 8 = 0x200C$ 。数据存储器将10写入地址0x200C。与此同时，PC递增至0x1008。

**R-Type 指令**  
接下来考虑扩展数据通路以处理R型指令：`add`、`sub`、`and`、`or`和`slt`。所有这些指令都从寄存器文件中读取两个源寄存器，对它们执行某种ALU操作，并将结果写回目标寄存器。它们仅在具体的ALU操作上有所不同。因此，它们都可以用相同的硬件处理，但使用不同的`ALUControl`信号。回顾第5.2.4节，`ALUControl`为000表示加法，001表示减法，010表示AND，011表示OR，101表示小于则置位。

图7.10展示了处理这些R型指令的增强型数据通路。该数据通路从寄存器文件的端口1和端口2读取`rs1`和`rs2`，并对它们执行ALU操作。我们引入了一个多路选择器和一个新的选择信号`ALUSrc`，用于在`ImmExt`和`RD2`之间选择作为第二个ALU源`SrcB`。对于`lw`和`sw`指令，`ALUSrc`为1以选择`ImmExt`；对于R型指令，`ALUSrc`为0以选择寄存器文件输出`RD2`作为`SrcB`。

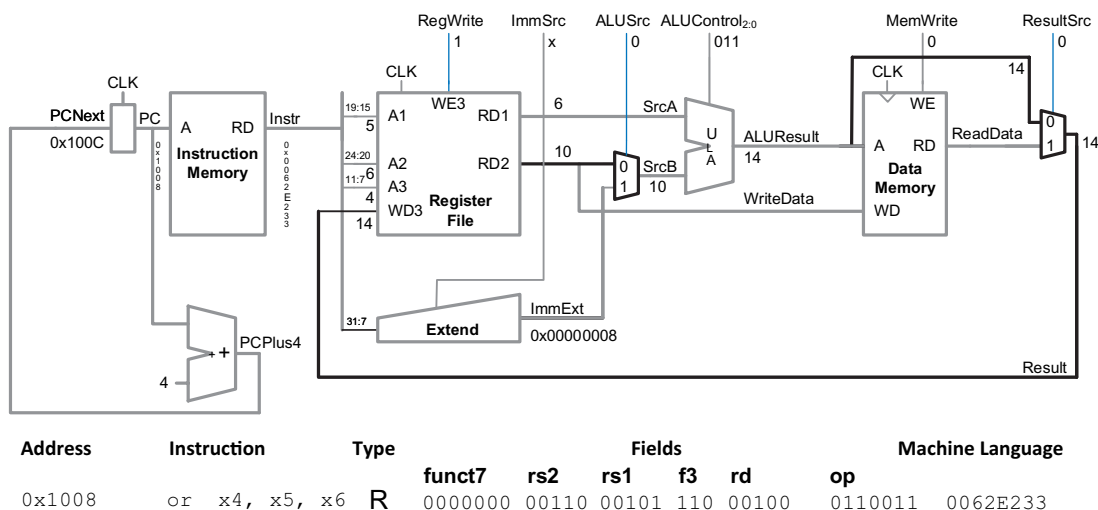


图7.10 R型指令的数据通路增强

让我们将写回寄存器文件的值命名为 $Result$ 。对于lw指令， $Result$ 来自存储器的ReadData输出。然而，对于R型指令， $Result$ 来自ALU的ALUResult输出。我们添加结果多路选择器，根据指令类型选择合适的 $Result$ 。多路选择器的选择信号 $ResultSrc$ 在R型指令时为0，以选择ALUResult作为 $Result$ ； $ResultSrc$ 在lw指令时为1，以选择ReadData。对于sw指令，我们不关心 $ResultSrc$ 的值，因为它不写入寄存器文件。

在我们的示例中，PC为0x1008。因此，指令存储器读出or指令0x0062E233。寄存器文件从x5读取源操作数6，从x6读取10。ALUControl为011，所以ALU计算 $6 \mid 10 = 0110_2 \mid 1010_2 = 1110_2 = 14$ 。结果写回x4。同时，PC递增至0x100C。

beq

最后，我们扩展数据通路以处理条件相等分支（beq）指令。beq比较两个寄存器，若相等，则通过将分支偏移量加到程序计数器（PC）上来执行分支。

分支偏移量是一个13位有符号立即数，存储在B型指令的12位立即数字段中。因此，扩展逻辑需要另一种模式来选择正确的立即数。 $ImmSrc$ 增加到2位，使用表7.1中的编码。 $ImmExt$ 现在要是

请注意，我们的硬件会计算不同指令所需的所有可能答案（例如ALUResult和ReadData），然后使用多路选择器根据指令选择合适的结果。这是一项重要的设计策略。在本章的剩余部分，我们将添加多路选择器来选择所需的答案。

软件与硬件的主要区别之一在于软件是顺序执行的，因此我们只需计算所需的结果。而硬件是并行运行的，所以我们通常会计算出所有可能的结果，然后从中选出所需的那一个。例如，在执行R型指令的ALU运算时，即使我们并不关心存储器中的数据是什么，存储器仍然会接收地址并从中读取数据。

表7.1 ImmSrc编码

| ImmSrc | ImmExt                                                       | Type | Description             |
|--------|--------------------------------------------------------------|------|-------------------------|
| 00     | {{20{Instr[31]}}, Instr[31:20]}                              | I    | 12-bit signed immediate |
| 01     | {{20{Instr[31]}}, Instr[31:25], Instr[11:7]}                 | S    | 12-bit signed immediate |
| 10     | {{20{Instr[31]}}, Instr[7], Instr[30:25], Instr[11:8], 1'b0} | B    | 13-bit signed immediate |

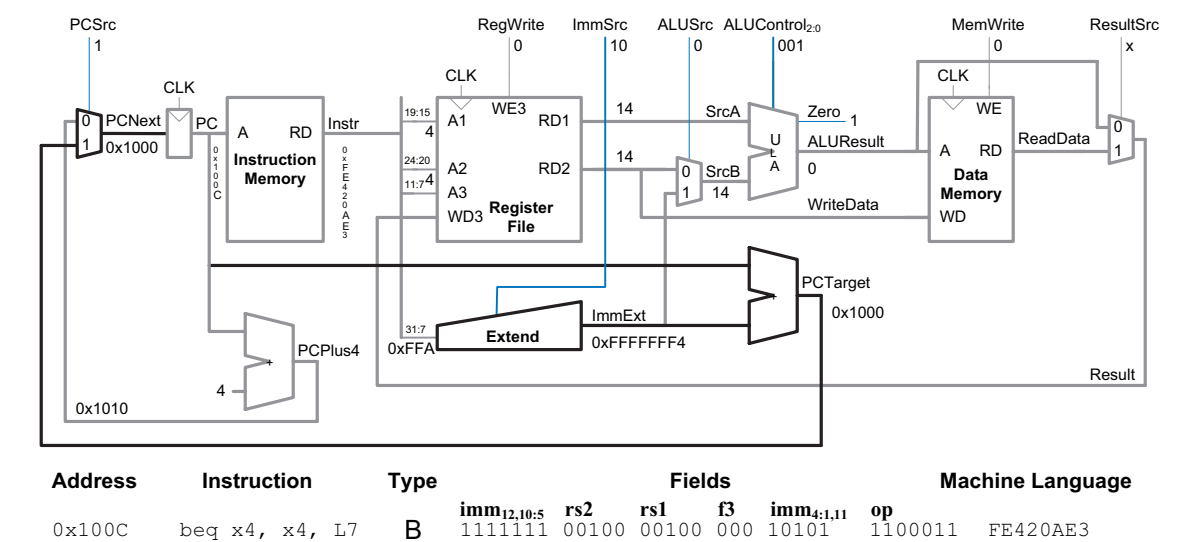


图7.11 beq的数据通路增强

从逻辑上讲，我们可以基于ImmSrc和指令的各种位字段，从一个32位3:1多路复用器构建扩展单元，该多路复用器从三个可能的输入中选择一个。实际上，符号扩展立即数的高位始终来自指令的第31位Instr<sub>31</sub>，因此我们可以优化设计，仅使用一个多路复用器来选择低位。

符号扩展的立即数（当ImmSrc=为00或01时）或分支偏移量（当ImmSrc=为10时）。

图7.11展示了数据路径的修改。我们需要另一个加法器来计算分支目标地址， $PCTarget = PC + ImmExt$ 。两个源寄存器通过使用ALU计算( $SrcA - SrcB$ )来进行比较。如果ALUResult为0，如ALU的Zero标志所示，则寄存器相等。我们添加一个多路选择器，从PCPlus4或PCTarget中选择PCNext。如果指令是分支且Zero标志被断言，则选择PCTarget。对于beq指令，ALUControl = 001，因此ALU执行减法操作。ALUSrc = 0用于从寄存器文件中选择SrcB。RegWrite和MemWrite为0，因为分支不会写入寄存器文件或内存。我们不需要关心ResultSrc的值，因为寄存器文件不会被写入。

在我们的示例中，PC为0x100C，因此指令存储器读出beq指令0xFE420AE3。两个源寄存器均为x4，因此

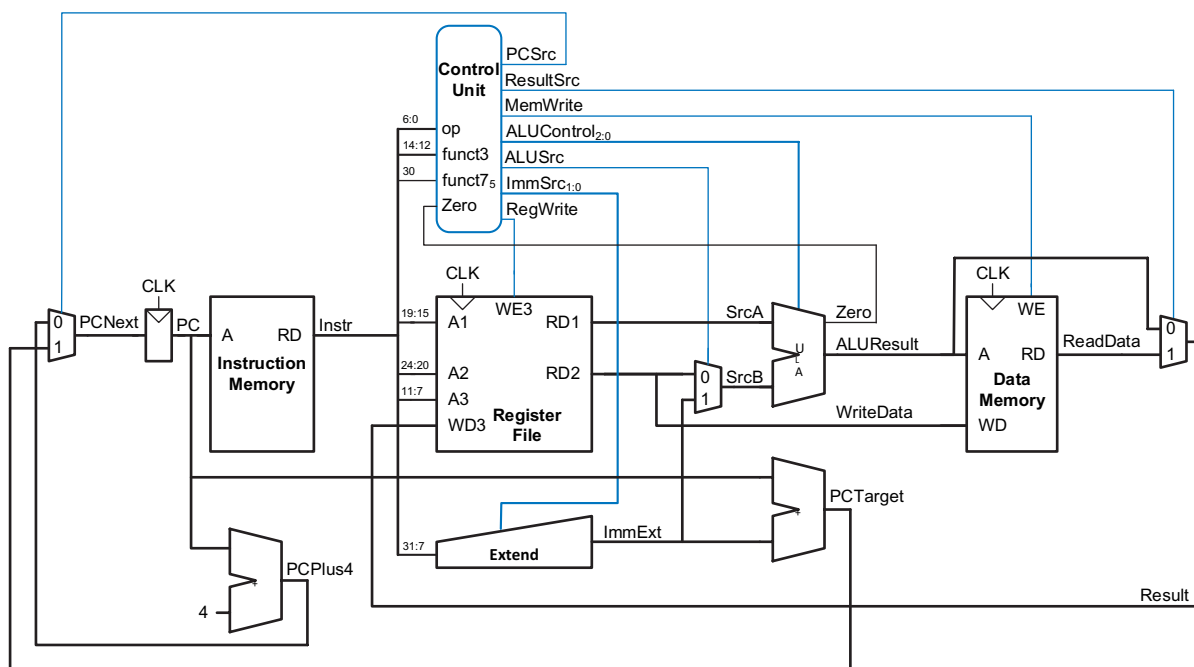


图7.12 完整的单周期处理器

寄存器文件在两个端口上都读取14。ALU计算 $14 - 14 = 0$ ，并且 *Zero* 标志被置位。同时，扩展单元产生 $0xFFFFFFFF4$ （即 $-12$ ），该值被加到 *PC* 上，得到  $PCTarget = 0x1000$ 。请注意，我们在扩展单元的输入上显示了13位立即数的未交错高12位（ $0xFFA$ ）。*PCNext*多路选择器选择  $PCTarget$  作为下一个 *PC*，并在下一个时钟边沿分支回代码的起始位置。

至此，单周期处理器数据通路的设计完成。我们不仅展示了设计本身，还阐述了设计过程：识别状态元件，并系统地添加组合逻辑以连接这些状态元件。在下一节中，我们将探讨如何计算控制信号，以指导数据通路的操作。

我们根据多路选择器（mux）产生的信号来命名它们。例如，*PCNext* 多路选择器产生 *PCNext* 信号，*Result* 多路选择器产生 *Result* 信号。

### 7.3.3 单周期控制

单周期处理器的控制单元根据 *op*、*funct3* 和 *funct7* 计算控制信号。对于 RV32I 指令集，仅使用 *funct7* 的第5位，因此我们只需考虑 *op* ( $Instr_{6:0}$ )、*funct3* ( $Instr_{14:12}$ ) 和 *funct7\_5* ( $Instr_{30}$ )。图7.12展示了整个单周期处理器，其中控制单元连接到数据通路。



图7.13 单周期处理器控制单元

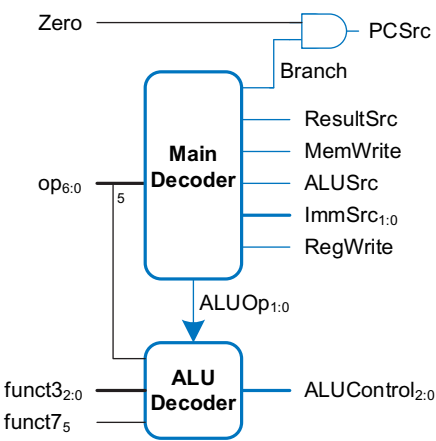


表7.2 主译码器真值表

| Instruction | Op      | RegWrite | ImmSrc | ALUSrc | MemWrite | ResultSrc | Branch | ALUOp |
|-------------|---------|----------|--------|--------|----------|-----------|--------|-------|
| lw          | 0000011 | 1        | 00     | 1      | 0        | 1         | 0      | 00    |
| sw          | 0100011 | 0        | 01     | 1      | 1        | x         | 0      | 00    |
| R-type      | 0110011 | 1        | xx     | 0      | 0        | 0         | 0      | 10    |
| beq         | 1100011 | 0        | 10     | 0      | 0        | x         | 1      | 01    |

图7.13对控制单元进行了层次化分解，该单元也被称为*controller*或*decoder*，因为它解码指令应执行的操作。我们将其分为两个主要部分：主译码器（Main Decoder），负责生成大部分控制信号；以及ALU译码器（ALU Decoder），用于确定ALU执行何种操作。

表7.2展示了主译码器在数据通路设计过程中确定的控制信号。主译码器根据操作码判断指令类型，然后为数据通路生成相应的控制信号。主译码器产生数据通路所需的大部分控制信号，同时还生成内部信号*Branch*和*ALUOp*，这些信号用于控制器内部。主译码器的逻辑可根据真值表，采用您偏好的组合逻辑设计技术来开发。

ALU 解码器根据 *ALUOp* 和 *funct3* 生成 *ALUControl*。对于 *sub* 和 *add* 指令，ALU 解码器还使用 *funct7<sub>5</sub>* 和 *op<sub>5</sub>* 来确定 *ALUControl*，如表 7.3 所示。

表7.3 ALU解码器真值表

| ALUOp | funct3 | {op <sub>5</sub> , funct7 <sub>5</sub> } | ALUControl          | Instruction |
|-------|--------|------------------------------------------|---------------------|-------------|
| 00    | x      | x                                        | 000 (add)           | lw, sw      |
| 01    | x      | x                                        | 001 (subtract)      | beq         |
| 10    | 000    | 00, 01, 10                               | 000 (add)           | add         |
|       | 000    | 11                                       | 001 (subtract)      | sub         |
|       | 010    | x                                        | 101 (set less than) | slt         |
|       | 110    | x                                        | 011 (or)            | or          |
|       | 111    | x                                        | 010 (and)           | and         |

*ALUOp*00 表示加法（例如，用于查找加载或存储的地址）。*ALUOp* 01 表示减法（例如，用于比较两个数以进行分支）。*ALUOp* 10 表示 R 型 ALU 指令，此时 ALU 解码器必须查看 *funct3* 字段（有时还需查看 *op<sub>5</sub>* 和 *funct7<sub>5</sub>* 位）以确定要执行的 ALU 操作（例如，加法、减法、与、或、slt）。

#### 示例7.1 单周期处理器操作

确定在执行与指令时使用的控制信号值以及数据路径的部分。

解答 图7.14展示了执行and指令期间的控制信号和数据流。PC指向存储该指令的内存位置；指令存储器输出该指令。通过寄存器文件和ALU的主要数据流以粗蓝线表示。寄存器文件读取由*Instr*指定的两个源操作数。*SrcB*应来自寄存器文件的第二个端口（而非*ImmExt*），因此*ALUSrc*必须为0。ALU执行按位与运算，因此*ALUControl*必须为010。结果来自ALU，因此*ResultSrc*为0，并将结果写入寄存器文件，因此*RegWrite*为1。该指令不写入内存，因此*MemWrite*为0。

PC使用*PCPlus4*进行更新的过程以粗灰线表示。*PCSrc*为0以选择递增后的PC。请注意，数据确实会流经未高亮显示的路径，但这些数据的值会被忽略。例如，立即数被扩展并从内存中读取一个值，但这些值不会影响系统的下一个状态。

根据书内封面的表B.1，add、sub和addi的*funct3* =均为000。add的*funct7* =为0000000，而sub的*funct7* =为0100000，因此*funct7<sub>5</sub>*足以区分这两者。但我们很快就会考虑支持addi，它没有*funct7*字段，其*op*为0010011。稍加思考便可发现，若*funct3* =为000，则当*op<sub>5</sub>*和*funct7<sub>5</sub>*均为1时为sub，否则为add或addi。



表7.4 增强支持addi的主译码器真值表

| Instruction | Opcode         | RegWrite | ImmSrc    | ALUSrc   | MemWrite | ResultSrc | Branch   | ALUOp     |
|-------------|----------------|----------|-----------|----------|----------|-----------|----------|-----------|
| lw          | 0000011        | 1        | 00        | 1        | 0        | 1         | 0        | 00        |
| sw          | 0100011        | 0        | 01        | 1        | 1        | x         | 0        | 00        |
| R-type      | 0110011        | 1        | xx        | 0        | 0        | 0         | 0        | 10        |
| beq         | 1100011        | 0        | 10        | 0        | 0        | x         | 1        | 01        |
| addi        | <b>0010011</b> | <b>1</b> | <b>00</b> | <b>1</b> | <b>0</b> | <b>0</b>  | <b>0</b> | <b>10</b> |

*ImmSrc*为00（见表7.1）。*SrcB*来自立即数，因此 $ALUSrc = 1$ 。该指令不写入内存，也不是分支指令，因此 $MemWrite = Branch = 0$ 。结果来自ALU而非内存，因此 $ResultSrc = 0$ 。最后，ALU应执行加法操作，因此 $ALUOp = 10$ ；ALU解码器使 $ALUControl = 000$ ，因为 $funct3 = 000$ 且 $op_5 = 0$ 。

敏锐的读者可能会注意到，这一变化也提供了其他I型ALU指令：andi、ori和sli。这些其他指令共享相同的操作码值0010011，需要相同的控制信号，仅在 $funct3$ 字段上有所不同，而ALU解码器已使用该字段来确定 $ALUControl$ ，进而确定ALU操作。

#### 示例7.3 jal 指令

展示如何修改RISC-V单周期处理器以支持跳转并链接（jal）指令。jal将PC+4写入rd，并将PC更改为跳转目标地址PC + imm。

解决方案 处理器通过将PC与指令中编码的21位有符号立即数相加来计算跳转目标地址，即 $PC_{Next}$ 的值。该立即数的最低位始终为0，接下来的20个最高有效位来自 $Instr_{31:12}$ 。这个21位立即数随后进行符号扩展。数据通路中已有硬件用于将PC与符号扩展后的立即数相加、选择该结果作为下一个PC、计算PC+4以及向寄存器文件写入值。因此，在数据通路中，我们只需修改扩展单元以对21位立即数进行符号扩展，并扩展结果多路选择器以选择PC+4（即 $PC_{Plus4}$ ），如图7.15所示。表7.5展示了支持jal长立即数的 $ImmSrc$ 新编码。

控制单元需要将 $PCSrc = 1$ 以执行跳转。为此，我们添加一个或门和另一个控制信号 $Jump$ ，如图7.16所示。当 $Jump$ 有效时， $PCSrc = 1$ ，并且 $PCTarget$ （跳转目标地址）被选为下一个PC。

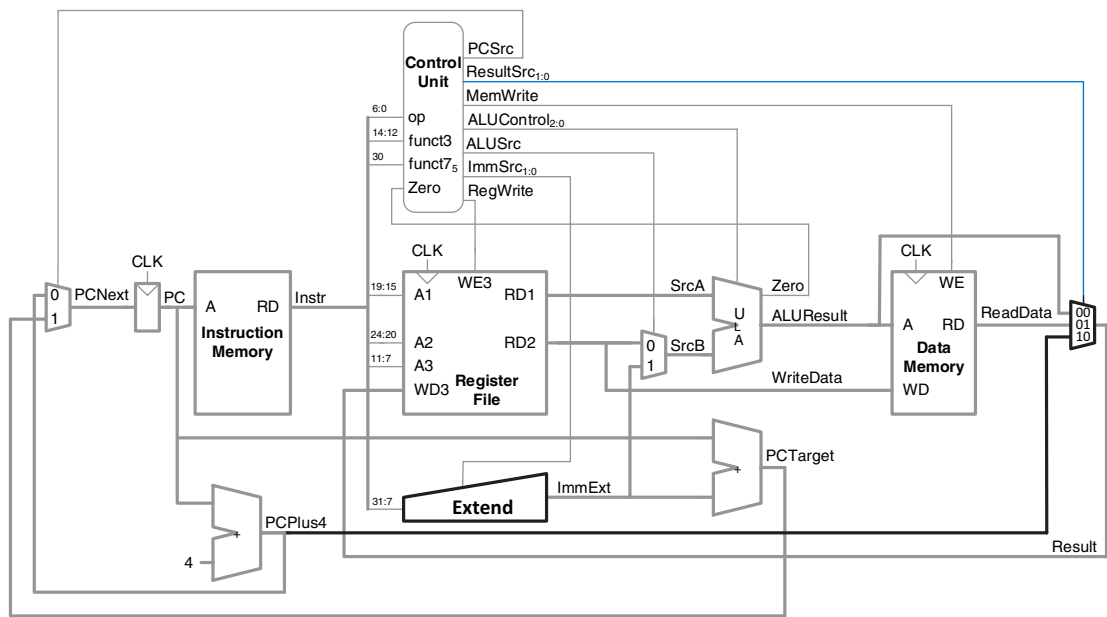


图7.15 jal的增强型数据通路

表7.5 ImmSrc编码。

| ImmSrc | ImmExt                                                         | Type | Description             |
|--------|----------------------------------------------------------------|------|-------------------------|
| 00     | {{20{Instr[31]}}, Instr[31:20]}                                | I    | 12-bit signed immediate |
| 01     | {{20{Instr[31]}}, Instr[31:25], Instr[11:7]}                   | S    | 12-bit signed immediate |
| 10     | {{20{Instr[31]}}, Instr[7], Instr[30:25], Instr[11:8], 1'b0}   | B    | 13-bit signed immediate |
| 11     | {{12{Instr[31]}}, Instr[19:12], Instr[20], Instr[30:21], 1'b0} | J    | 21-bit signed immediate |

表7.6展示了更新后的主译码表，其中为jal新增了一行。RegWrite = 1和ResultSrc = 10用于将PC+4写入rd。ImmSrc = 11用于选择21位跳转偏移量。ALUSrc和ALUOp无关紧要，因为ALU未被使用。MemWrite = 0，因为该指令不是存储指令；Branch = 0，因为该指令不是分支指令。新的Jump信号为1，用于选择跳转目标地址作为下一个PC。

7.3.5 性能分析

回顾公式7.1，程序的执行时间是指令数、每指令周期数以及

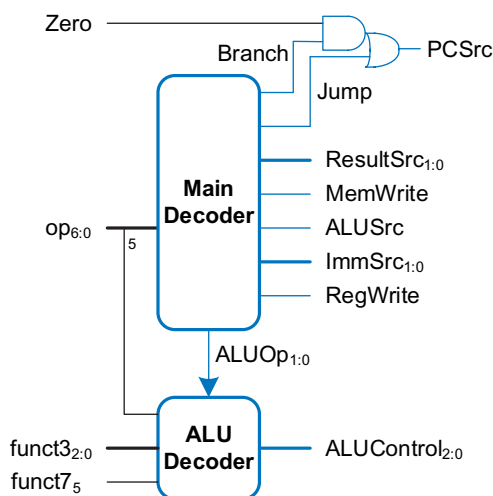


图7.16 jal的增强控制单元

表7.6 增强支持jal的主译码器真值表

| Instruction | Opcode  | RegWrite | ImmSrc | ALUSrc | MemWrite | ResultSrc | Branch | ALUOp | Jump |
|-------------|---------|----------|--------|--------|----------|-----------|--------|-------|------|
| lw          | 0000011 | 1        | 00     | 1      | 0        | 01        | 0      | 00    | 0    |
| sw          | 0100011 | 0        | 01     | 1      | 1        | xx        | 0      | 00    | 0    |
| R-type      | 0110011 | 1        | xx     | 0      | 0        | 00        | 0      | 10    | 0    |
| beq         | 1100011 | 0        | 10     | 0      | 0        | xx        | 1      | 01    | 0    |
| I-type ALU  | 0010011 | 1        | 00     | 1      | 0        | 00        | 0      | 10    | 0    |
| jal         | 1101111 | 1        | 11     | x      | 0        | 10        | 0      | xx    | 1    |

周期时间。单周期处理器中的每条指令占用一个时钟周期，因此每指令时钟周期数（CPI）为1。周期时间由关键路径决定。在我们的处理器中，lw指令耗时最长，涉及图7.17所示的关键路径。如粗蓝线所示，关键路径始于PC在时钟上升沿加载新地址。随后指令存储器读取新指令，寄存器文件读取rs1作为SrcA。在寄存器文件读取的同时，立即数字段根据ImmSrc进行符号扩展，并在SrcB多路选择器处被选中（灰色高亮路径）。ALU将SrcA与SrcB相加以计算内存地址。数据存储器从该地址读取数据，结果多路选择器选择ReadData作为Result。最后，Result必须在寄存器文件之前建立。

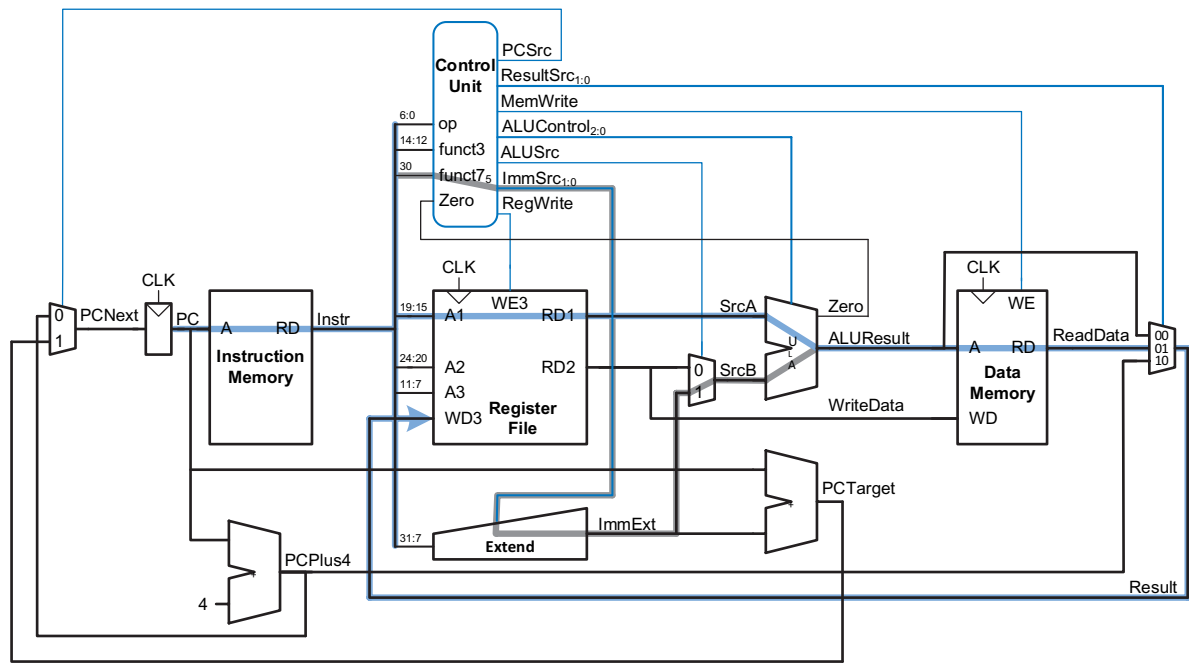


图7.17 lw的临界路径

下一个时钟上升沿，以便能够正确写入。因此，单周期处理器的周期时间为：

$$T_{c\_single} = t_{pcq\_PC} + t_{mem} + \max[t_{RFread}, t_{dec} + t_{ext} + t_{mux}] + t_{ALU} + t_{mem} + t_{mux} + t_{RFsetup} \quad (7.2)$$

请记住，lw 不使用寄存器文件的第二个读取端口（A2/RD2）。

在大多数实现技术中，ALU、存储器和寄存器堆的速度明显慢于其他组合逻辑模块。因此，关键路径经过寄存器堆——而非经过译码器（控制器）、扩展单元和多路选择器——即图7.17中蓝色高亮显示的路径。周期时间简化为：

$$T_{c\_single} = t_{pcq\_PC} + 2t_{mem} + t_{RFread} + t_{ALU} + t_{mux} + t_{RFsetup} \quad (7.3)$$

这些时间的数值将取决于具体的实现技术。

其他指令的关键路径较短。例如，R型指令无需访问数据存储器。然而，我们遵循同步时序设计原则，因此时钟周期是固定的，且必须足够长以容纳最慢的指令。

表7.7 电路元件的延迟

| Element                | Parameter     | Delay (ps) |
|------------------------|---------------|------------|
| Register clk-to-Q      | $t_{pcq}$     | 40         |
| Register setup         | $t_{setup}$   | 50         |
| Multiplexer            | $t_{mux}$     | 30         |
| AND-OR gate            | $t_{AND-OR}$  | 20         |
| ALU                    | $t_{ALU}$     | 120        |
| Decoder (control unit) | $t_{dec}$     | 25         |
| Extend unit            | $t_{ext}$     | 35         |
| Memory read            | $t_{mem}$     | 200        |
| Register file read     | $t_{RFread}$  | 100        |
| Register file setup    | $t_{RFsetup}$ | 60         |

#### 示例7.4 单周期处理器性能

Ben Bitdiddle 正在考虑采用 7 纳米 CMOS 制造工艺构建单周期处理器。他已确定逻辑元件的延迟如表 7.7 所示。请帮助他计算一个包含 1000 亿条指令的程序所需的执行时间。

根据公式7.3，单周期处理器的周期时间为  $T_{c\_single} = 40 + 2(200) + 100 + 120 + 30 + 60 = 750$  皮秒。根据公式7.1，总执行时间为  $T_{single} = (100 \times 10^9 \text{ 条指令}) (1 \text{ 周期/指令}) (750 \times 10^{-12} \text{ 秒/周期}) = 75 \text{ 秒}$ 。

## 7.4 多周期处理器

单周期处理器存在三个显著缺陷。首先，它需要独立的指令存储器和数据存储器，而大多数处理器仅有一个同时存储指令和数据的单一外部存储器。其次，其时钟周期必须足够长以支持最慢的指令（lw），尽管大多数指令本可以更快执行。最后，它需要三个加法器（一个在ALU中，两个用于PC逻辑）；加法器是相对昂贵的电路，尤其是在需要高速运行时。

多周期处理器通过将指令分解为多个较短的步骤来解决这些弱点。内存、ALU和寄存器



文件具有最长的延迟，因此为了使每个短步骤的延迟大致相等，处理器可以在每个步骤中使用这些单元中的*only one*个。处理器使用单个存储器，因为指令在一个步骤中被读取，而数据在后续步骤中被读取或写入。并且处理器只需要一个加法器，该加法器在不同步骤中被重复用于不同目的。各种指令使用的步骤数量不同，因此较简单的指令可以比更复杂的指令更快完成。

我们按照与单周期处理器相同的步骤设计多周期处理器。首先，通过将架构状态元件和存储器与组合逻辑连接来构建数据通路。但这一次，我们还添加了非架构状态元件，用于在步骤之间保存中间结果。然后，我们设计控制器。在执行单条指令期间，控制器在每一步产生不同的信号，因此现在控制器使用有限状态机而非组合逻辑。最后，我们分析多周期处理器的性能，并将其与单周期处理器进行比较。

7.4.1 多周期数据通路

再次，我们从处理器的存储器和架构状态开始设计，如图7.18所示。在单周期设计中，我们使用了独立的指令存储器和数据存储器，因为我们需要在一个周期内同时读取指令存储器并读取或写入数据存储器。现在，我们选择使用一个统一的存储器来同时存储指令和数据。这更符合实际情况，并且是可行的，因为我们可以一个周期内读取指令，然后在另一个周期内读取或写入数据。PC和寄存器文件保持不变。

与单周期处理器类似，我们通过逐步添加组件来处理每条指令的每一步，从而构建数据通路。PC包含待执行指令的地址。第一步是从内存中读取该指令。图7.19显示PC被

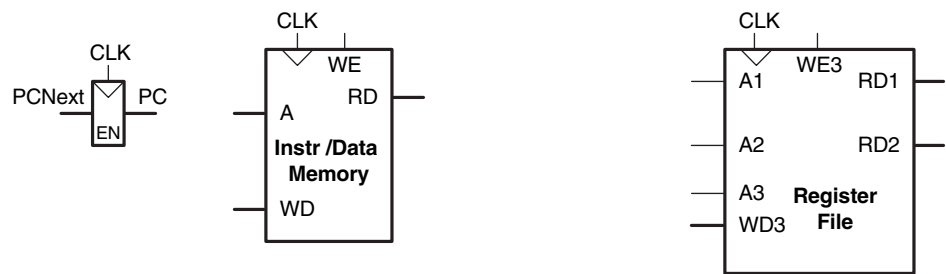


图7.18 统一指令/数据存储器的状态元件

直接连接到存储器的地址输入。指令被读取并存储在一个新的非架构指令寄存器（IR）中，以便在未来的周期中可用。IR接收一个使能信号，称为 $IRWrite$ ，当IR应加载新指令时，该信号被置位。

与单周期处理器类似，我们根据多路选择器和非架构寄存器产生的信号来命名它们。例如，指令寄存器产生指令信号（ $Instr$ ），结果多路选择器产生 $Result$ 信号。

$lw$

与单周期处理器类似，我们首先为 $lw$ 指令设计数据通路连接。在获取 $lw$ 指令后，第二步是读取包含基地址的源寄存器。该寄存器由 $rs1$ 字段 $Instr_{19:15}$ 指定。如图7.20所示，指令的这些位连接到寄存器文件的地址输入 $A1$ 。寄存器文件将寄存器内容读取到 $RD1$ ，该值存储在另一个非架构寄存器 $A$ 中。

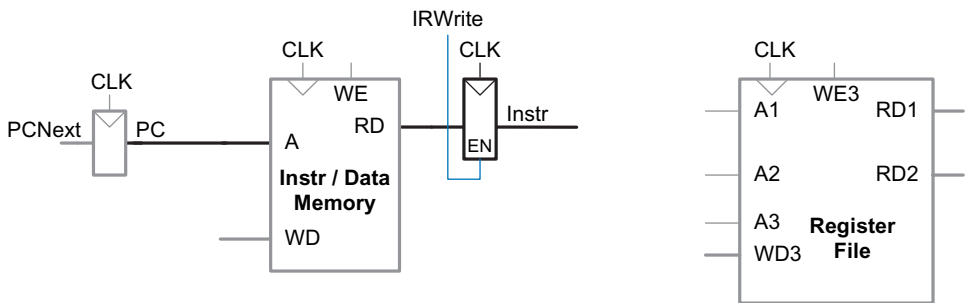


图7.19 从内存中取指令

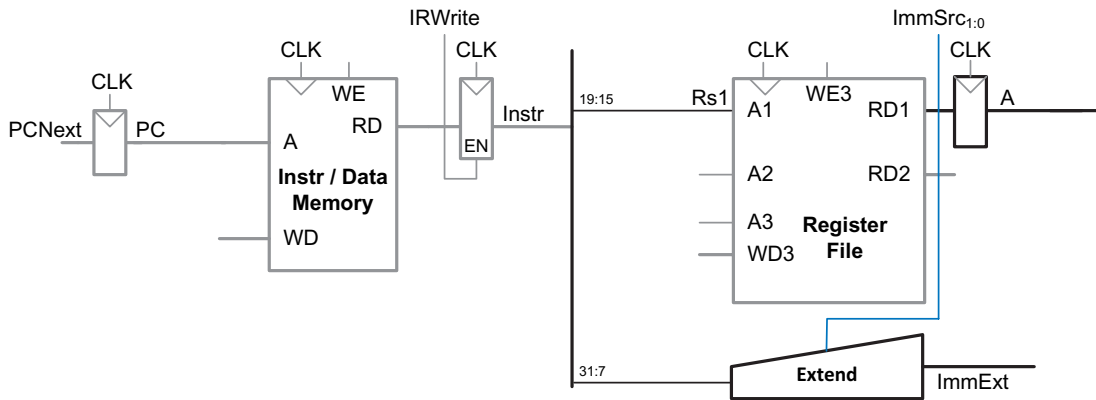


图7.20 从寄存器文件读取一个源操作数，并从立即数扩展第二个源操作数  $ImmSrc_{1:0}$  字段

lw指令还需要一个12位偏移量，该偏移量位于指令的立即数字段 $Instr_{31:20}$ 中，必须将其符号扩展为32位，如图7.20所示。与单周期处理器一样，扩展单元接收一个2位 $ImmSrc$ 控制信号，以指定对12位、13位或21位立即数进行扩展，适用于不同类型的指令。32位扩展后的立即数称为 $ImmExt$ 。为保持一致性，我们可能将 $ImmExt$ 存储在另一个非架构寄存器中。然而， $ImmExt$ 是 $Instr$ 的组合函数，在当前指令处理期间不会改变，因此无需专门分配寄存器来保存该常量值。

加载地址是基地址与偏移量之和。在第三步中，我们使用ALU计算该和值，如图7.21所示。 $ALUControl$ 应设置为000以执行加法操作。 $ALUResult$ 存储在一个名为 $ALUOut$ 的非架构寄存器中。

第四步是从内存中计算出的地址加载数据。我们在内存前添加一个多路选择器，根据 $AdrSrc$ 选择信号，从PC或 $ALUOut$ 中选择内存地址 $Adr$ ，如图7.22所示。从内存读取的数据存储在另一个非架构寄存器中，称为 $Data$ 。请注意，地址（ $Adr$ ）多路选择器允许我们在lw指令期间复用内存单元。在第一步中，地址取自PC以获取指令。在第四步中，地址取自 $ALUOut$ 以加载数据。因此， $AdrSrc$ 在单条指令的不同步骤中必须具有不同的值。在7.4.2节中，我们将开发生成这些控制信号序列的FSM控制器。

最后，数据被写回寄存器文件，如图7.23所示。目标寄存器由指令的rd字段 $Instr_{11:7}$ 指定。结果来自数据寄存器。与其将数据寄存器直接连接到寄存器文件的WD3写入端口，不如在 $Result$ 总线上添加一个多路选择器，以便在将 $Result$ 反馈回寄存器文件之前，选择 $ALUOut$ 或 $Data$ 。

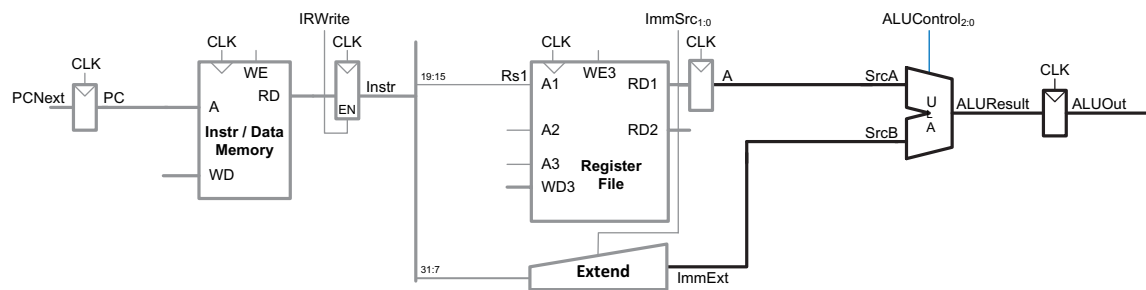


图7.21 将基地址与偏移量相加

writedata 端口 (WD3)。这将很有帮助, 因为其他指令需要将 ALU 的结果写入寄存器文件。RegWrite 信号为 1 表示寄存器文件应被更新。

在此过程中，处理器必须通过将PC加4来更新程序计数器。在单周期处理器中，需要单独的加法器。而在多周期处理器中，我们可以在指令获取阶段使用现有的ALU，因为它此时空闲。为此，必须插入源多路选择器，选择PC和常数4作为ALU输入，如图7.24所示。一个由 $ALUSrcA$ 控制的多路选择器选择PC或A作为 $SrcA$ 。另一个多路选择器选择4或 $ImmExt$ 作为 $SrcB$ 。我们还展示了额外的多路选择器输入，这些将在实现更多指令时使用。为了更新PC，ALU将 $SrcA$ （PC）与 $SrcB$ （4）相加，结果写入程序计数器。结果多路选择器从 $ALUResult$ 而非 $ALUOut$ 中选择该和；这需要第三个多路选择器输入。 $PCWrite$ 控制信号使PC仅在特定周期被写入。至此完成了lw指令的数据通路。

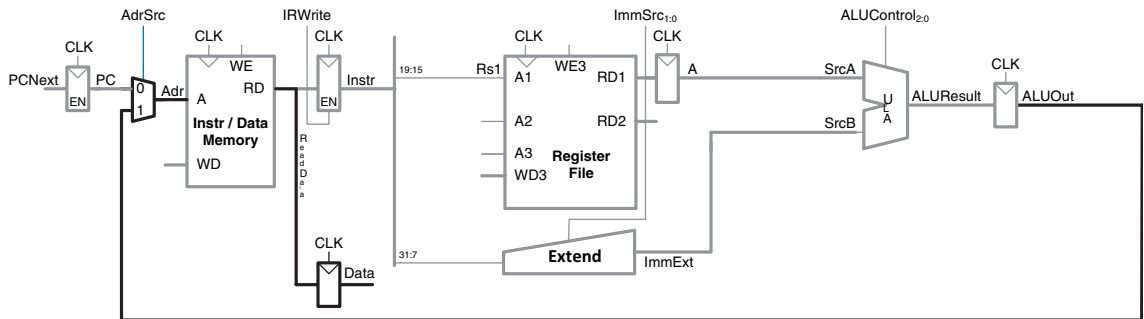


图7.22 从内存加载数据

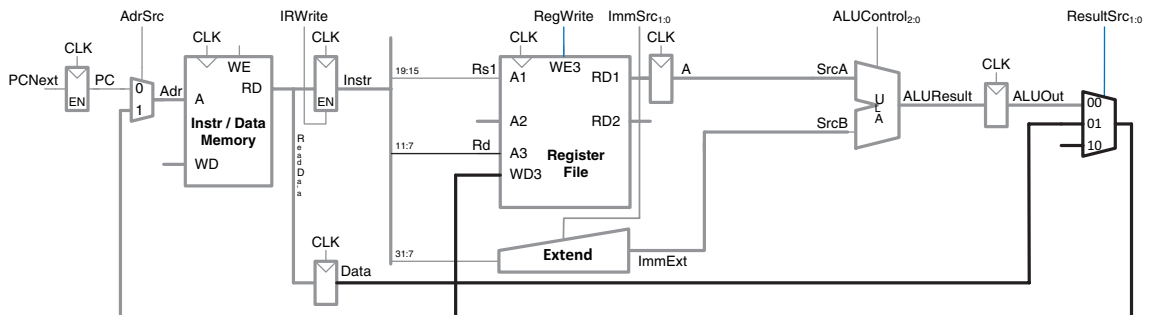


图7.23 将数据写回寄存器文件

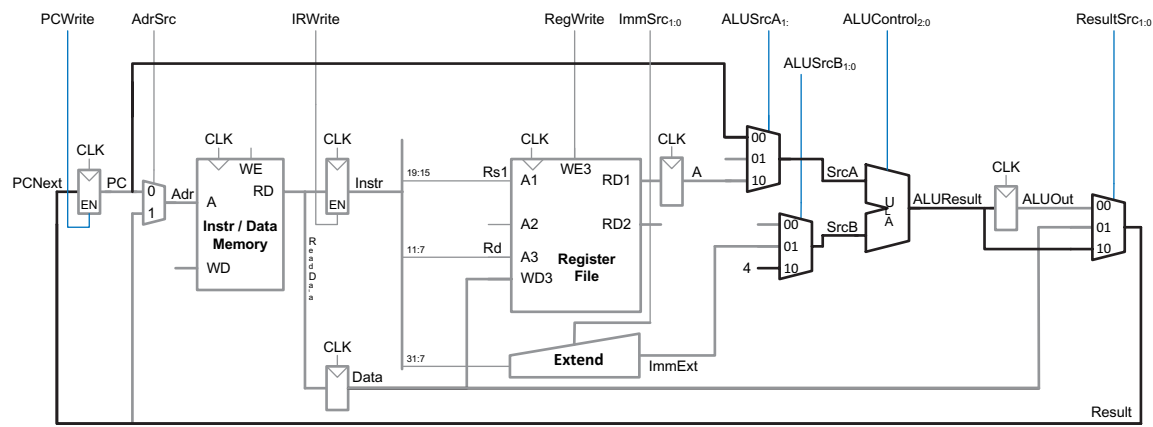


图7.24 PC增加4

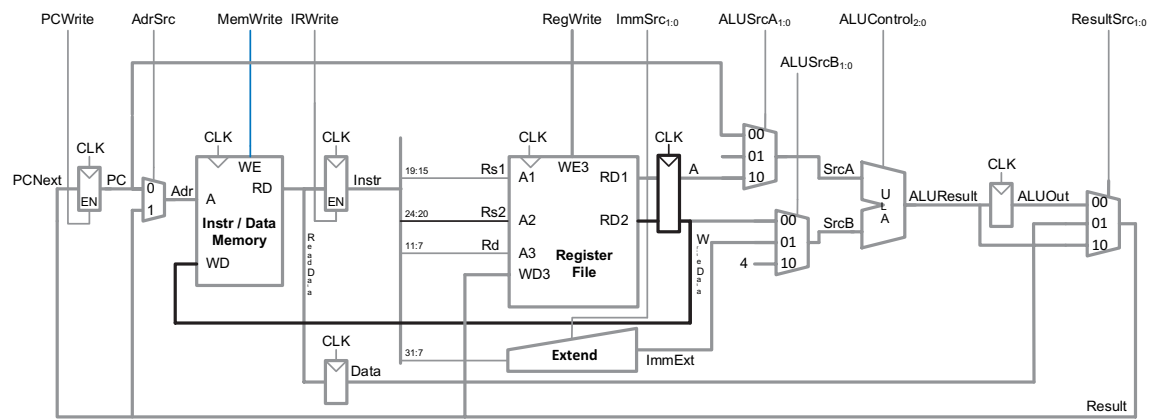


图7.25 sw指令的增强型数据通路

SW

接下来，让我们扩展数据通路以处理 `sw` 指令。与 `lw` 类似，`sw` 在第一步从寄存器文件的端口 1 读取基址，并在第二步扩展立即数。然后，ALU 在第三步将基址与立即数相加，以找到内存地址。`sw` 唯一的新特性是，我们必须从寄存器文件中读取第二个寄存器，并将其内容写入内存，如图 7.25 所示。该寄存器由指令的 `rs2` 字段 `Instr24:20` 指定，该字段连接到寄存器文件的第二个端口 A2。读取后

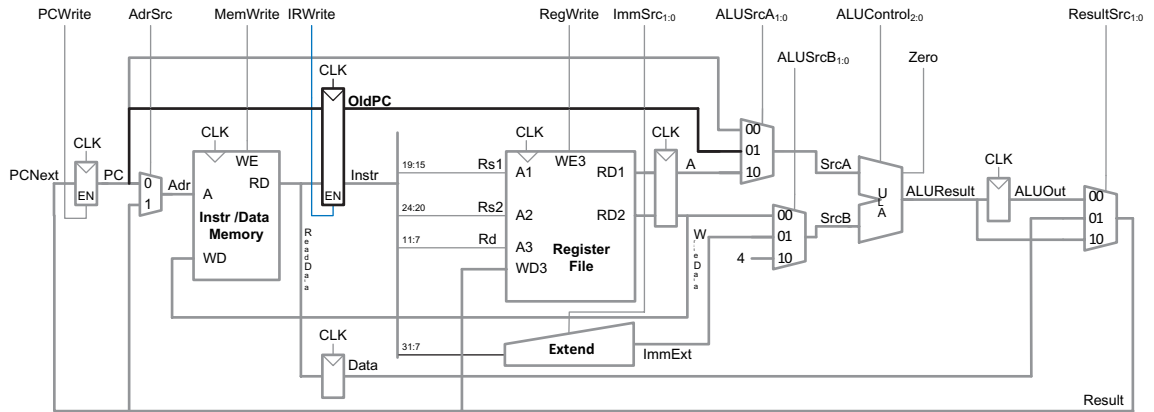


图7.26 用于beq目标地址计算的增强型数据通路

在第二步中，寄存器的内容随后被存储在一个非架构寄存器中，即位于A寄存器正下方的WriteData寄存器。然后，它被发送到数据存储器的写数据端口（WD），以便在第四步进行写入。存储器接收*MemWrite*控制信号，该信号在需要写入存储器时被置位。

#### R-Type 指令

R型指令对两个源寄存器进行操作，并将结果写回寄存器文件。数据通路已经包含了这些步骤所需的所有连接。

#### beq

beq 检查两个寄存器的内容是否相等，并通过将当前 PC 与一个 13 位有符号分支偏移相加来计算分支目标地址。数据通路中已存在通过减法比较寄存器的硬件。

在指令执行的第二步中，ALU未被使用，因此我们利用它来计算分支目标地址  $PCTarget = PC + ImmExt$ 。在此步骤中，指令已从内存中取出，且PC已更新为PC+4。因此，在第一步中，当前指令的PC *OldPC* 必须存储在一个非架构寄存器中。在第二步中，当寄存器也被取出时，ALU通过选择 *OldPC* 作为 *SrcA*、*ImmExt* 作为 *SrcB*，并将 *ALUControl* = 设置为000以执行加法，从而计算出  $PC + ImmExt$ 。处理器将此和存储在 *ALUOut* 寄存器中。图7.26展示了更新后的beq数据通路。

在第三步中，ALU 减去源寄存器，如果它们相等则断言 *Zero* 输出。如果相等，控制单元则断言

*PCWrite*结果多路选择器选择*ALUOut*（包含目标地址）送入PC，无需新增硬件。

至此，多周期数据通路的设计完成。其设计过程与单周期处理器类似，即通过系统地将硬件连接在状态元件之间来处理每条指令。主要区别在于指令分多个步骤执行，并插入非架构寄存器来保存每一步的结果。这样，存储器可同时用于指令和数据，ALU也可多次复用，从而降低硬件成本。下一节中，我们将开发一个有限状态机控制器，为每条指令的每一步向数据通路提供相应的控制信号序列。

7.4.2 多周期控制

与单周期处理器一样，控制单元根据指令的*op*、*funct3*和*funct7s*字段（*Instr*<sub>6:0</sub>、*Instr*<sub>14:12</sub>和*Instr*<sub>30</sub>）计算控制信号。图7.27展示了完整的多周期处理器，其中控制单元连接到数据通路。数据通路以黑色显示，控制单元以蓝色显示。

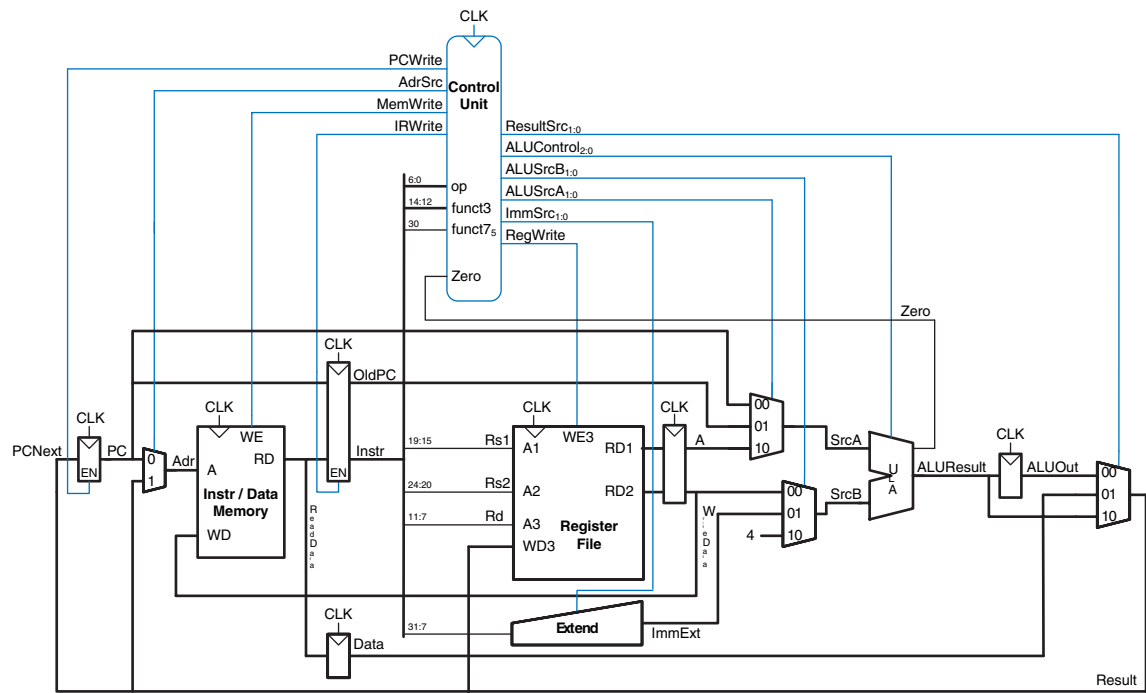


图7.27 完整多周期处理器

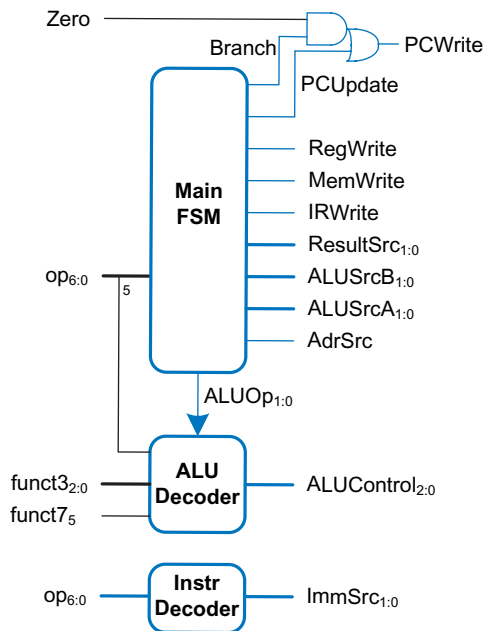


图7.28 多周期控制单元

控制单元由主FSM、ALU译码器和指令译码器（Instr Decoder）组成，如图7.28所示。ALU译码器与单周期处理器中的相同（见表7.3），但多周期处理器中用主FSM替代了单周期处理器中的组合逻辑主译码器，以在适当的周期产生一系列控制信号。一个小型指令译码器根据操作码组合逻辑地生成 $ImmSrc$ 选择信号，该信号基于表7.6中的 $ImmSrc$ 列。我们将主FSM设计为摩尔机，因此输出仅取决于当前状态。本节其余部分将为主FSM开发状态转换图。

主FSM为数据路径生成多路选择器选择信号、寄存器使能信号和存储器写使能信号。为使后续状态转换图清晰可读，仅列出相关控制信号。多路选择器选择信号仅在其值重要时列出；否则为无关项。使能信号（ $RegWrite$ 、 $MemWrite$ 、 $IRWrite$ 、 $PCUpdate$ 和 $Branch$ ）仅在其被断言时列出；否则为0。

获取  
任何指令的第一步都是从PC中保存的地址处从存储器中取出指令。FSM在复位时进入此取指状态。控制信号如图7.29所示。为了读取指令

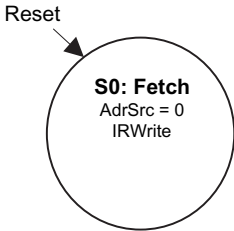


图7.29 获取



根据记忆,  $AdrSrc = 0$ , 因此地址取自PC。 *IRWrite* 被断言以将指令写入指令寄存器IR。同时, 当前PC被写入*OldPC*寄存器。此步骤及*lw*指令后续两个步骤中通过数据通路的数据流如图7.32所示, 其中取指阶段的数据流以灰色高亮显示。

解码

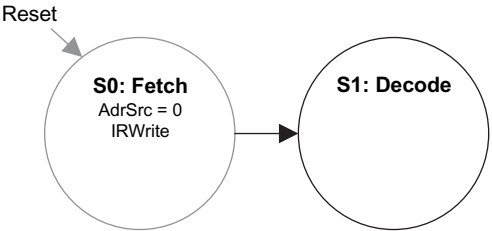
第二步是读取寄存器文件并解码指令。控制单元*decodes*指令, 即根据*op*、*funct3*和*funct7*<sub>5</sub>确定应执行何种操作。在此状态下, 处理器还会读取源寄存器*rs1*和*rs2*, 并将读取的值存入*A*和*WriteData*非架构寄存器中。这些任务无需任何控制信号。图7.30展示了主FSM中的解码状态, 图7.32以中蓝色线条显示了该状态下数据通路的流程。完成此步骤后, 由于指令已被取出并解码, 处理器能够根据指令区分其操作。我们将首先展示*lw*指令的剩余步骤, 然后继续介绍其他RISC-V指令的步骤。

内存地址

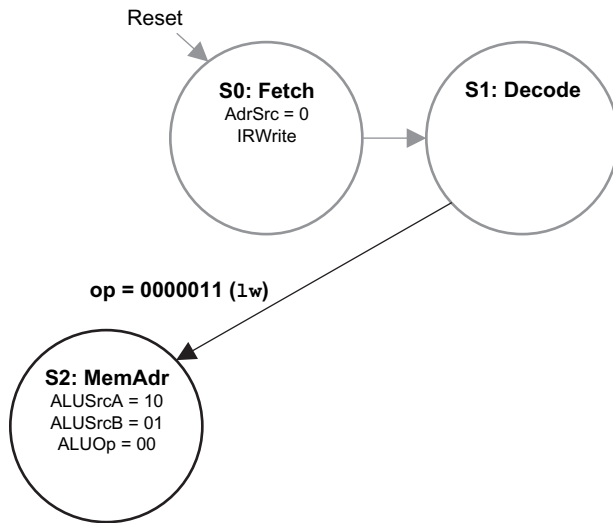
*lw*的第三步是计算内存地址。ALU将基址和偏移量相加, 因此 $ALUSrcA = 10$ 选择*A* (从*rs1*读取的值) 作为*SrcA*,  $ALUSrcB = 01$ 选择*ImmExt*作为*SrcB*。 *ImmSrc*由指令解码器确定为00, 用于对*I*型立即数进行符号扩展,  $ALUOp$ 为00, 用于将*SrcA*和*SrcB*相加。在此状态结束时, ALU结果 (即地址计算结果) 存储在*ALUOut*寄存器中。图7.31展示了添加到主FSM中的*MemAdr*状态, 图7.32展示了该状态期间的数据通路流程, 以深蓝色线条突出显示。

内存读取

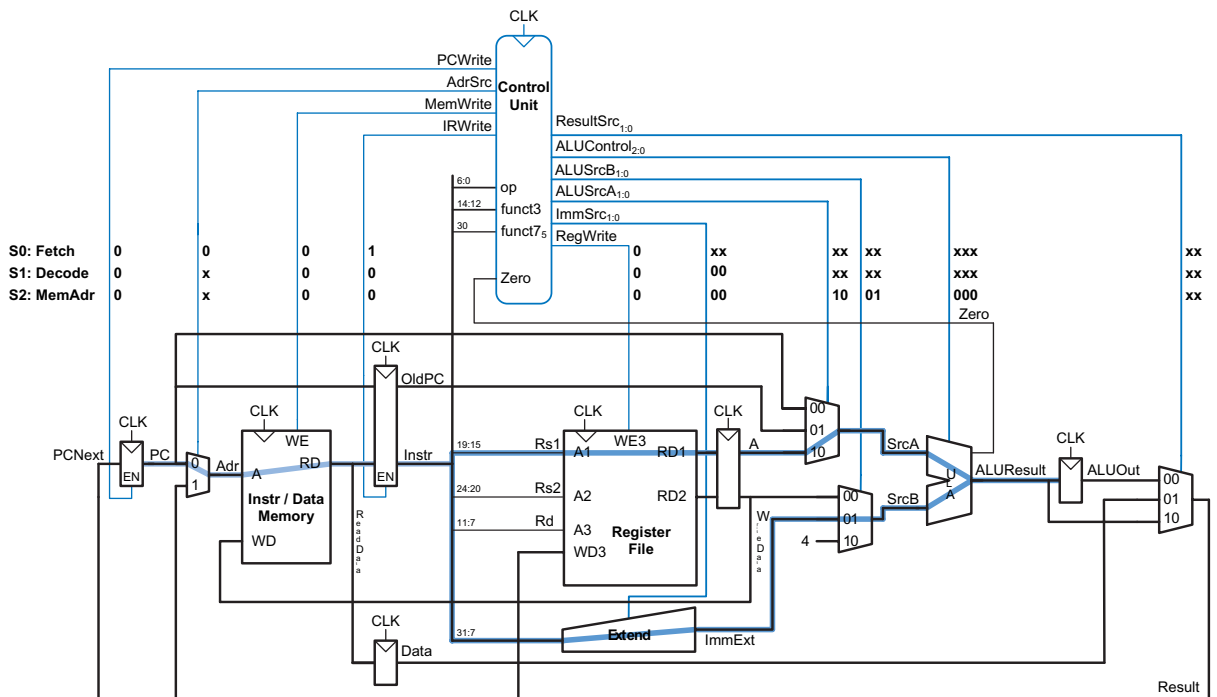
内存读取 (MemRead) 步骤通过将*ALUOut*发送到内存的地址端口*Adr*, 将计算出的地址发送到内存。



**Figure 7.30** Decode



**Figure 7.31** Memory address computation



**图7.32** 取指、译码和MemAdr状态期间的数据流

*ResultSrc*= 00 和 *AdrSrc* = 1 通过 *Result* 和 *Adr* 多路选择器路由到存储器地址输入。*ReadData* 获取从存储器中所需地址读取的值。在此状态结束时，*ReadData* 被写入数据寄存器。

MemWB

在存储器写回（MemWB）阶段，从存储器读取的数据 *Data* 被写入目标寄存器。*ResultSrc* 为 01 以选择 *Data* 作为 *Result*，并且 *RegWrite* 被置位以将数据写入寄存器文件。寄存器文件端口 3 的地址和写数据输入（*A3* 和 *WD3*）已分别连接到 *rd*（*Instr*<sub>11:7</sub>）和 *Result*。图 7.33 和图 7.34 展示了 MemRead 和 MemWB 状态以及这两个阶段的数据通路流程。MemWB 是 *lw* 指令的最后一步。图 7.33 还显示了从 MemWB 返回的转换。

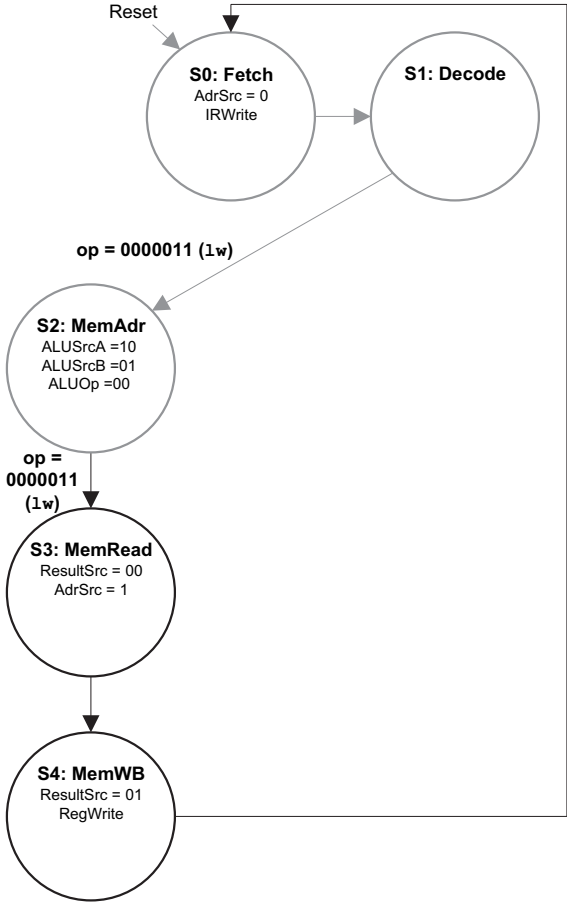


图7.33 存储器读取（MemRead）和存储器写回（MemWB）状态

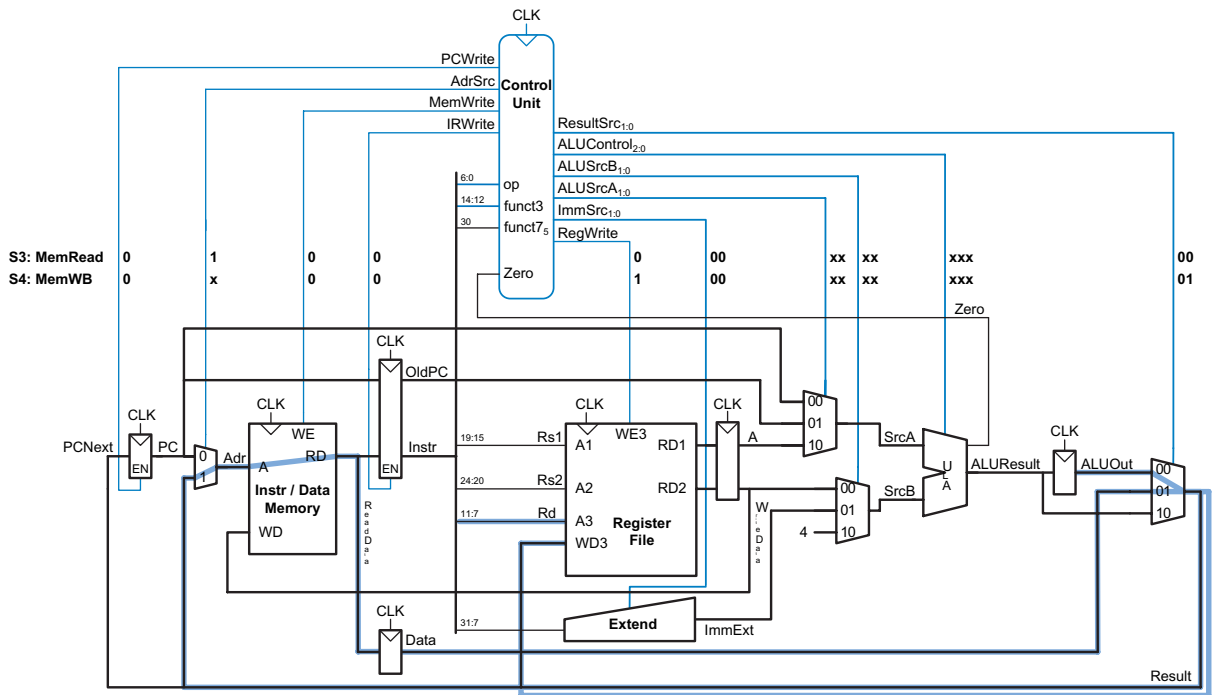


图7.34 MemRead和MemWB期间的数据流

进入取指状态，以便可以取下一条指令。然而，程序计数器尚未递增。我们接下来解决这个问题。

在执行lw指令之前，处理器必须递增PC以便获取下一条指令。我们可以为此增加一个状态，但为了节省一个周期，注意到在取指阶段ALU未被使用，因此处理器可以在取指的同时利用该状态计算PC+4。 $ALUSrcA = 00$  用于从PC（即信号OldPC）获取SrcA。 $ALUSrcB = 10$  用于获取常量4作为SrcB。 $ALUOp = 00$ ，使得ALU将PC与4相加。为了用PC+4更新PC， $ResultSrc = 10$  选择ALUResult作为Result，并且 $PCUpdate = 1$  强制PCWrite为高电平（见图7.28）。图7.35展示了修改后的取指状态。图中其余部分与图7.33相同。图7.36用蓝色高亮显示了计算PC+4的数据流。同时进行的指令取指用灰色高亮显示。

我们在本节开始时提到，每个步骤中只能使用一个耗时单元（内存、ALU或寄存器文件）。然而，这里我们使用both寄存器文件和ALU。只要这些单元同时使用——即并行使用——一个步骤中就可以使用多个单元。

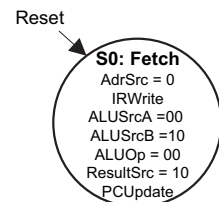


图7.35 在取指状态下递增PC

SW

现在，我们扩展主FSM以处理更多RISC-V指令。所有指令都经过前两个状态：取指（Fetch）和解码（Decode）。

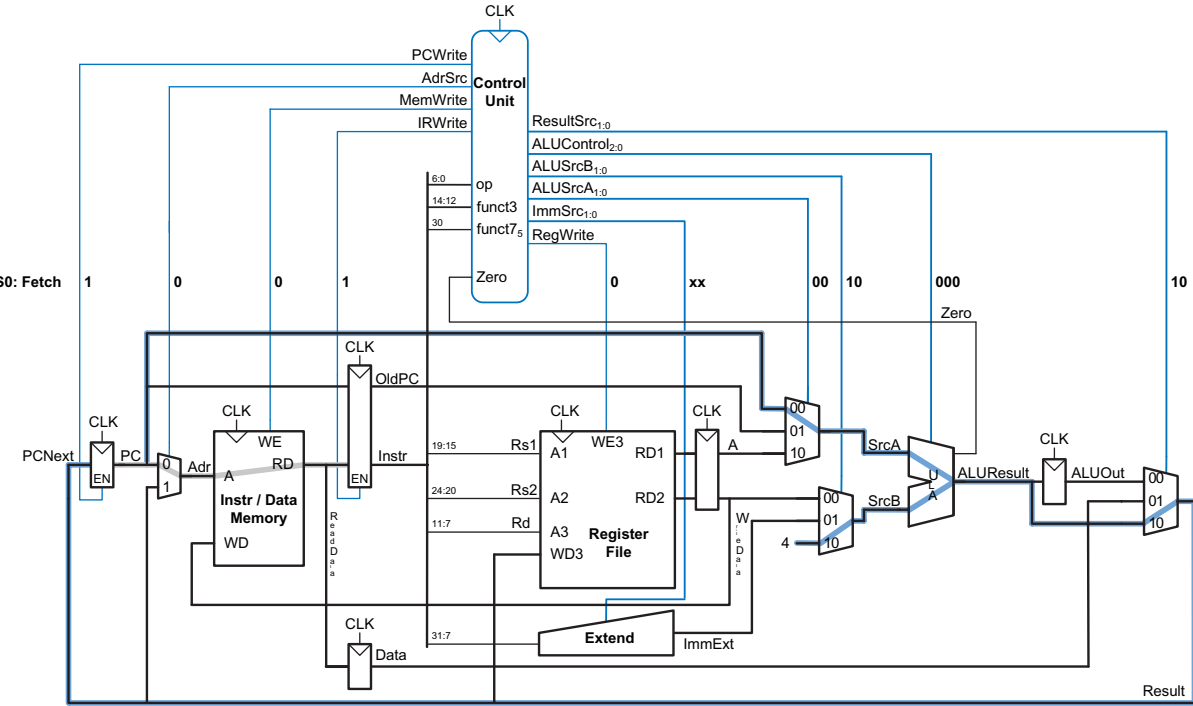


图7.36 取指状态下递增PC时的数据流

sw指令使用与lw相同的MemAdr状态来计算内存地址，但随后进入内存写入（MemWrite）状态，在该状态下，WriteData（来自rs2的值）被写入内存。WriteData被硬连线到内存的写数据端口（WD）。内存的地址端口Adr通过将ResultSrc设为= 00、AddrSrc设为= 1，设置为ALUOut中计算出的地址。MemWrite被置位以写入内存。至此sw指令完成，主FSM返回Fetch状态以开始下一条指令。图7.37和图7.38展示了扩展后的主FSM以及MemWrite状态的数据通路流程。请注意，FSM的前两个状态（Fetch和Decode）未在图7.37中显示，它们与图7.33中的相同。

在MemAdr状态下，ImmSrc信号对于lw和sw有所不同。但请回忆，ImmSrc是由指令解码器组合逻辑生成的（见图7.28）。

**R-Type 指令**  
在解码状态之后，R型ALU指令进入执行（ExecuteR）状态，该状态执行所需的ALU计算。即，ALUSrcA = 10 和 ALUSrcB = 00 选择rs1的内容作为SrcA，rs2的内容作为SrcB。ALUOp = 10 使得ALU解码器使用指令的控制字段来确定要执行的操作。

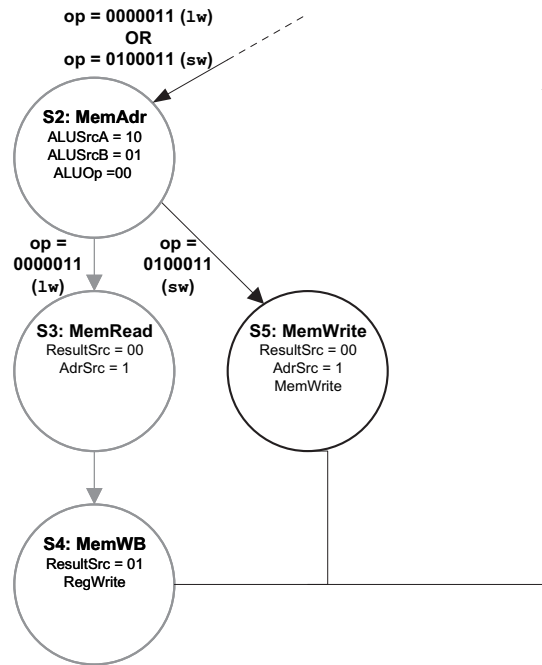


Figure 7.37 Memory write

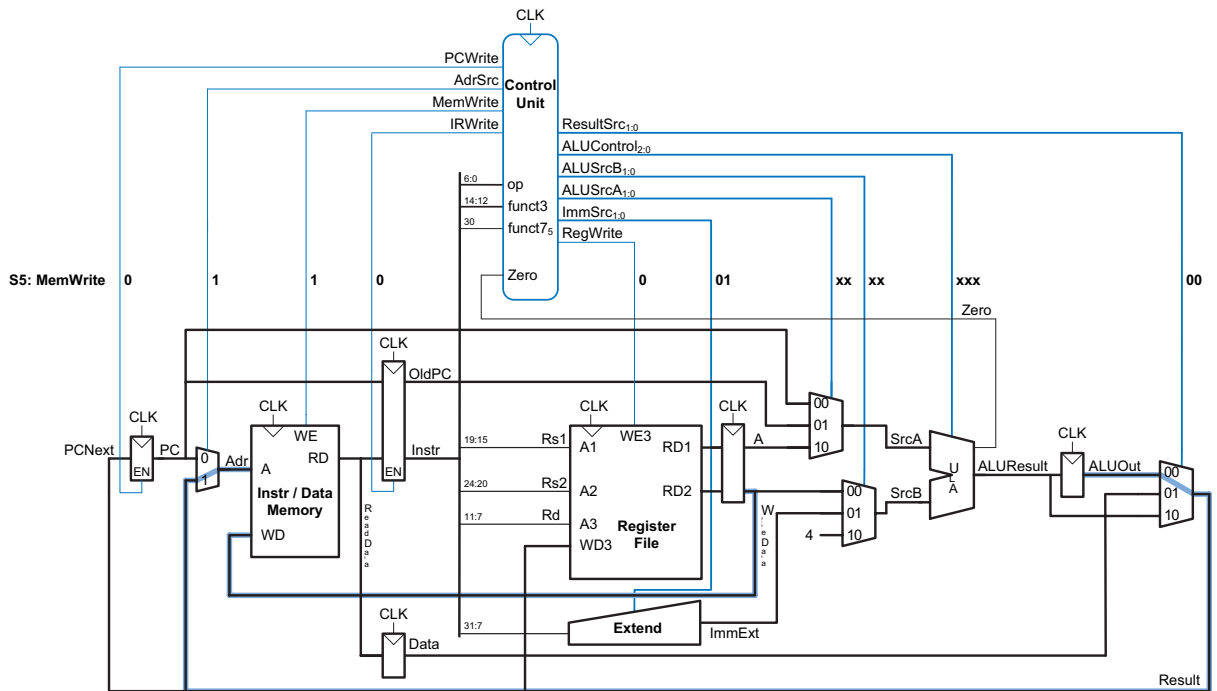


图7.38 内存写入 (MemWrite) 状态下的数据流

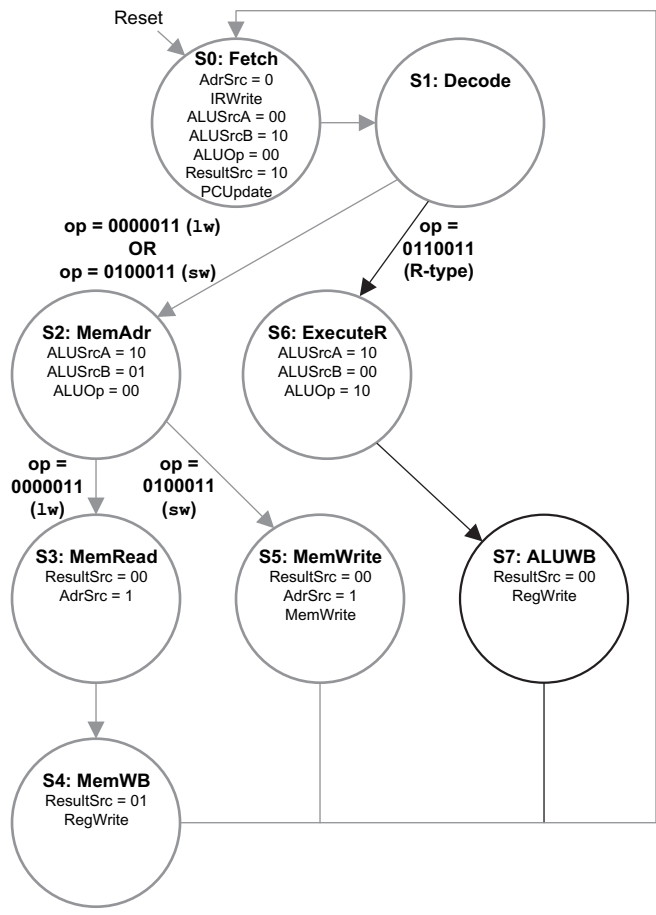


图7.39 执行R型（ExecuteR）和ALU写回（ALUWB）状态

*ALUResult*在周期结束时，计算结果被写入ALUOut寄存器。R型指令随后进入ALU写回（ALUWB）状态，在该状态下，计算结果*ALUOut*被写回寄存器文件。在ALUWB状态中，*ResultSrc*=设置为00以选择*ALUOut*作为*Result*，且*RegWrite*=设置为1，以便将结果写入rd。图7.39展示了添加到主FSM中的ExecuteR和ALUWB状态。图7.40展示了这两个状态下的数据流，其中ExecuteR数据流以粗浅蓝色线表示，ALUWB数据流以粗深蓝色线表示。

beq

最后一条指令beq比较两个寄存器并计算分支目标地址。到目前为止，在译码状态下ALU处于空闲状态，因此我们可以在该状态下使用ALU来计算分支目标。

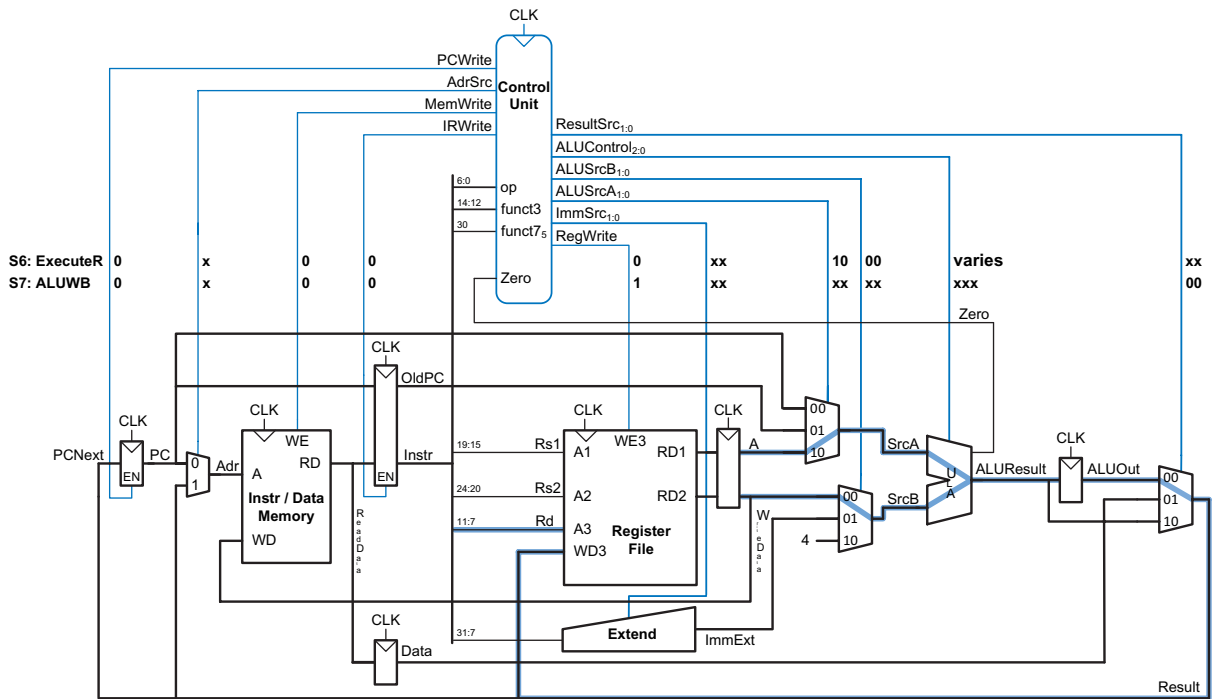


图7.40 执行R和ALUWB状态期间的数据流

地址， $OldPC + ImmExt$ 。ALUSrcA和ALUSrcB均为01，因此OldPC为SrcA，分支偏移量（ImmExt）为SrcB。ALUOp = 00以使ALU执行加法。目标地址在译码状态结束时存储于ALUOut寄存器中。图7.41展示了增强后的译码状态以及后续的BEQ状态，该状态将在下文讨论。在图7.42中，译码状态期间的数据流以浅蓝色和灰色线条表示。分支目标地址计算以浅蓝色高亮显示，寄存器读取和立即数扩展以粗灰色线条高亮显示。

在解码状态之后，beq进入BEQ状态，在该状态下比较源寄存器。ALUSrcA = 10 和 ALUSrcB = 00 用于选择从寄存器文件中读取的值作为SrcA 和 SrcB。ALUOp = 01 使得ALU执行减法。如果源寄存器相等，则ALU的Zero输出置位（因为 $rs1 - rs2 = 0$ ）。Branch = 在此状态下为1，因此如果Zero也被置位，则PCWrite被置位（如图7.28的PCWrite逻辑所示），并且分支目标地址（在ALUOut中）成为下一个PC。ALUOut通过ResultSrc为00被路由到PC寄存器。图7.41显示了BEQ状态，图7.42

尽管在解码状态开始时指令尚未被解码——甚至可能不是beq指令——但分支目标地址仍会像分支指令一样被计算。如果最终发现该指令不是分支指令，或者分支未被采用，则计算结果将直接被丢弃。



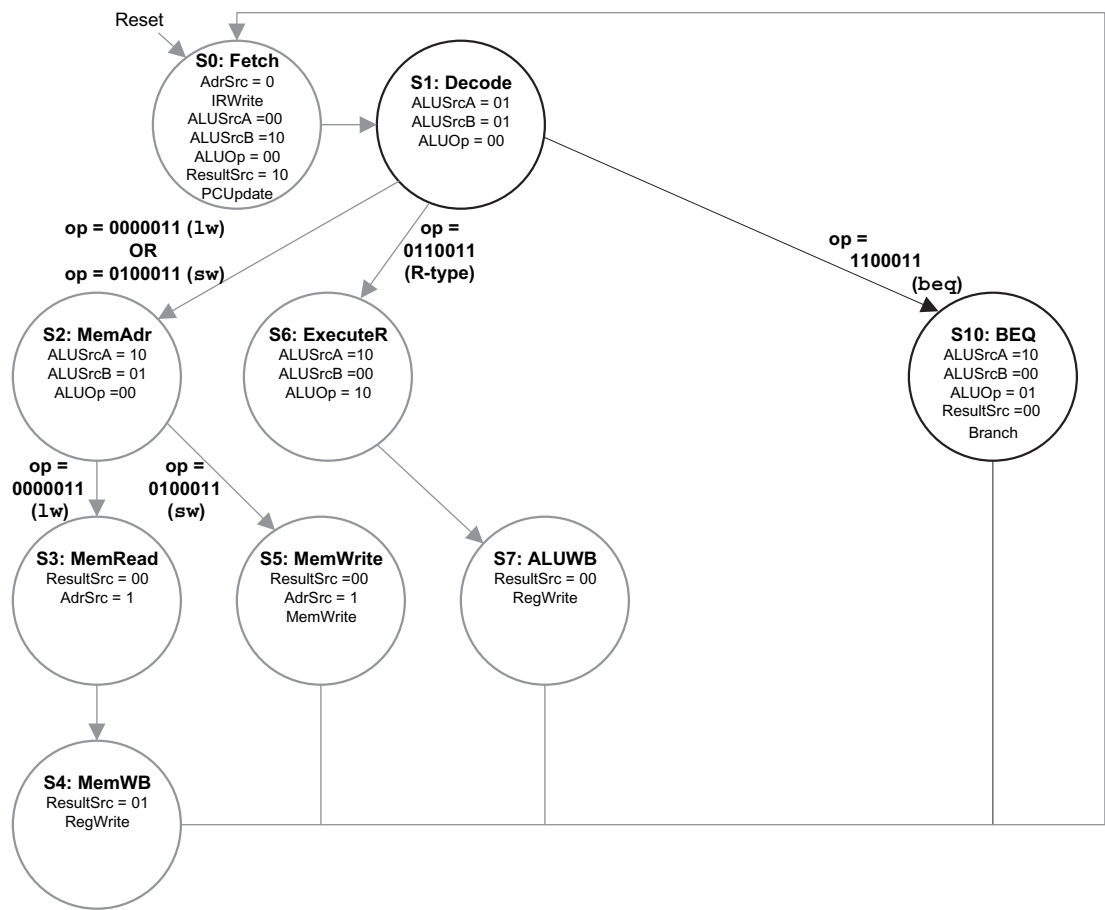


图7.41 增强型解码状态，包含分支目标地址计算和BEQ状态

展示了BEQ状态下的数据流。比较rs1和rs2的路径以深蓝色显示，（条件性地）将PC设置为目标地址的路径通过结果寄存器以灰色显示。至此，这些指令的控制器的设计完成。

7.4.3 更多指令

与单周期处理器类似，接下来我们考虑如何修改多周期处理器的数据通路和控制器以处理新指令：I型ALU指令（addi、andi、ori、slti）和jal。

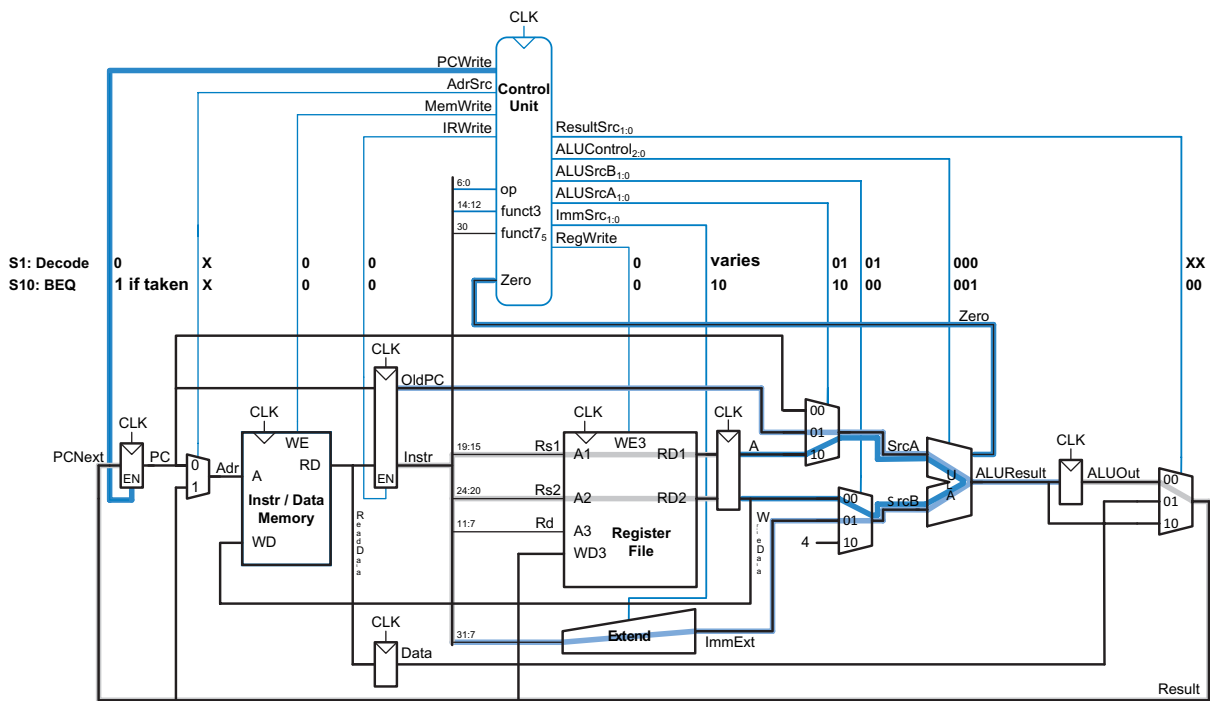


图7.42 解码和BEQ状态期间的数据流

示例7.5 扩展多周期处理器以包含I型ALU指令

扩展多周期处理器以包含I型ALU指令addi、andi、ori和slti。

解决方案 这些I型ALU指令与其R型等效指令（add、and、or和slt）几乎相同，区别仅在于第二个源操作数来自ImmExt而非寄存器文件。我们引入ExecuteI状态来执行所有I型ALU指令所需的计算。该状态与ExecuteR类似，不同之处在于ALUSrcB = 01用于选择ImmExt作为SrcB。在ExecuteI状态之后，I型ALU指令进入ALU写回（ALUWB）状态，将结果写入寄存器文件。图7.43展示了增强后的主有限状态机，其中还包含了示例7.6中的JAL状态。

示例7.6 扩展多周期处理器以包含jal

扩展多周期处理器以包含跳转并链接指令（jal）。

解决方案 与例7.5中的I型ALU指令类似，实现jal指令无需额外硬件。仅需修改主有限状态机

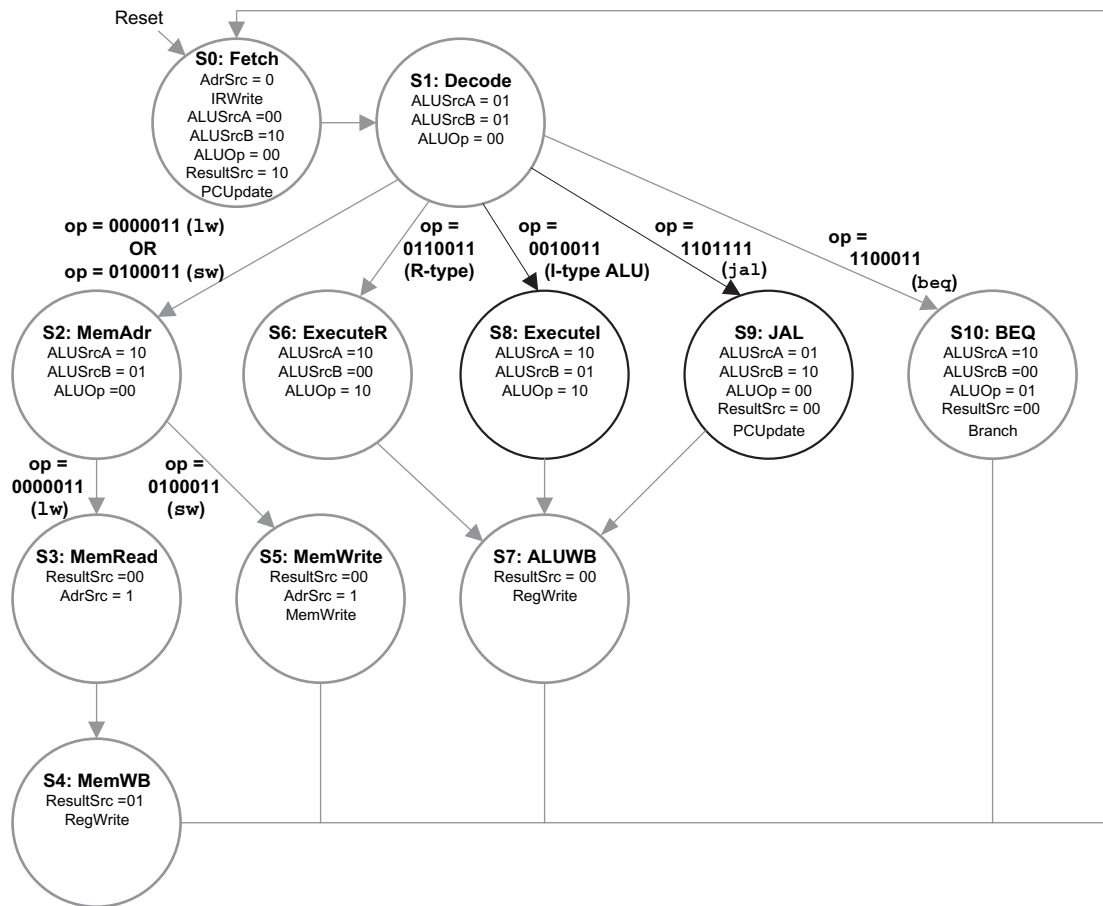


图7.43 增强型主FSM：Executel和JAL状态

您可能会注意到，当处理器到达JAL状态时，PC寄存器已经更新为PC+4。因此，我们本可以直接使用PC寄存器的输出写入rd。但这将需要扩展结果多路选择器以接收PC作为输入。上述解决方案使用了现有数据通路，因此所需硬件更少。

已更新。前两步与其他指令相同。在解码状态下，跳转目标地址的计算流程与分支目标地址计算相同，但使用指令解码器设置的ImmSrc = 11。因此，在解码状态下，跳转偏移量经过符号扩展后与当前PC（包含在信号OldPC中）相加，形成跳转目标地址，并在该状态结束时写入ALUOut寄存器。随后，jal进入JAL状态，处理器将目标地址写入PC，并计算返回地址（PC+4），以便在下一状态将其写入rd。ALU使用ALUSrcA = 01（SrcA = OldPC）、ALUSrcB = 10（SrcB = 4）和ALUOp = 00进行加法运算，计算PC+4（即OldPC+4）。为了将目标地址写入PC，ResultSrc = 00用于选择目标地址（保存在ALUOut中）作为Result，并且PCUpdate = 1以使PCWrite生效。图7.43

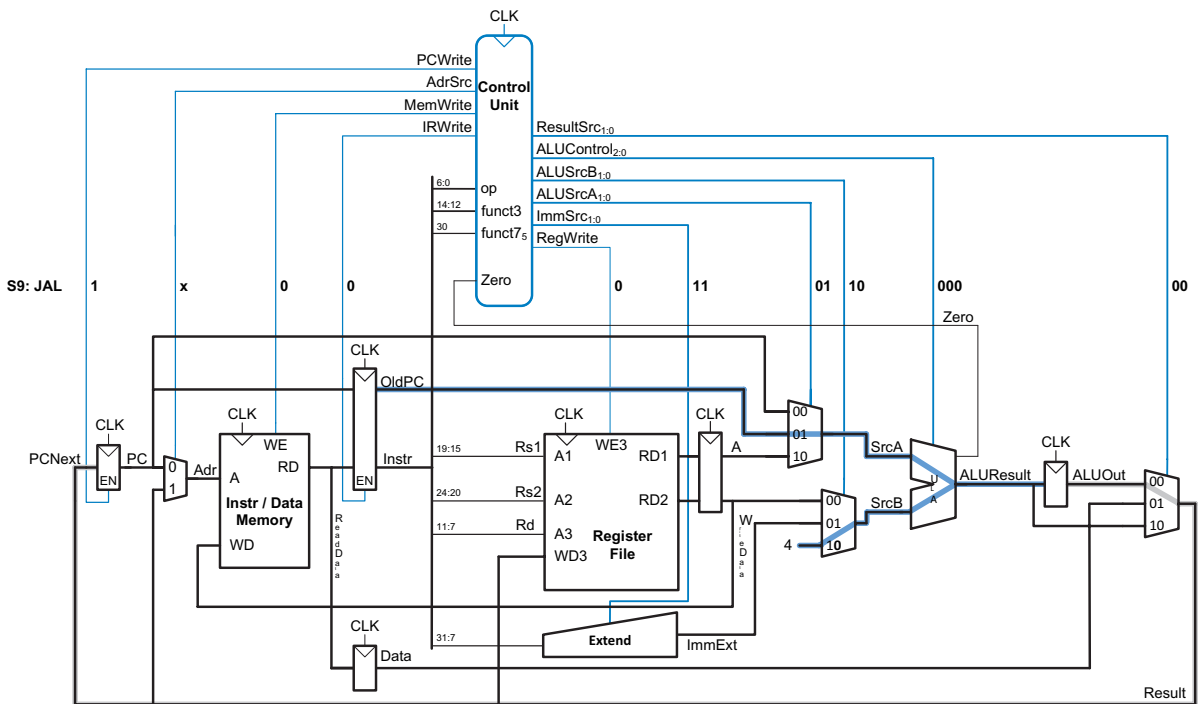


图7.44 JAL状态期间的数据流

显示了新的JAL状态，图7.44展示了JAL状态期间的数据流。更新PC至目标地址的流程以灰色表示，PC+4计算以蓝色表示。在JAL状态之后，jal进入ALUWB状态，在此状态下返回地址（ALUOut = PC+4）被写入rd。至此jal指令结束，主FSM随后返回Fetch状态。

综合这些步骤，图7.45展示了多周期处理器完整的主FSM状态转换图。图中下方总结了每个状态的功能。使用第3章的技术将状态图转换为硬件是一项直接但繁琐的任务。更佳的做法是，可以用HDL编写该FSM，并利用第4章的技术进行综合。

#### 7.4.4 性能分析

指令的执行时间取决于其使用的周期数以及周期时间。单周期处理器在一个周期内执行所有指令，而多周期处理器则使用

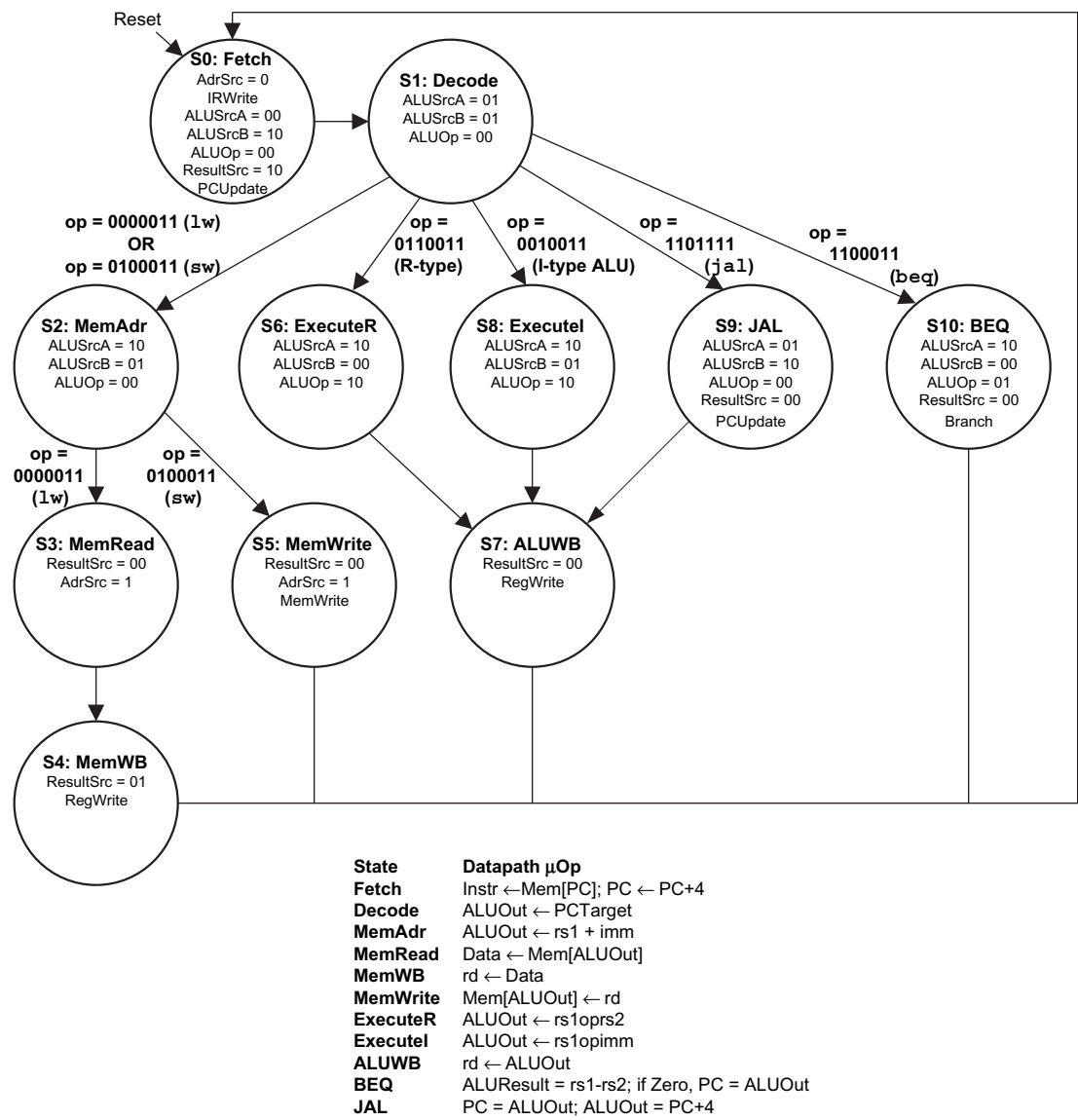


图7.45 完整多周期控制有限状态机

不同指令的周期数各不相同。然而，多周期处理器在单个周期内完成的工作较少，因此其周期时间更短。

多周期处理器对于分支指令需要三个周期，对于R型、I型ALU、跳转和存储指令需要四个周期，对于加载指令需要五个周期。每条指令的时钟周期数（CPI）取决于每条指令被使用的相对概率。

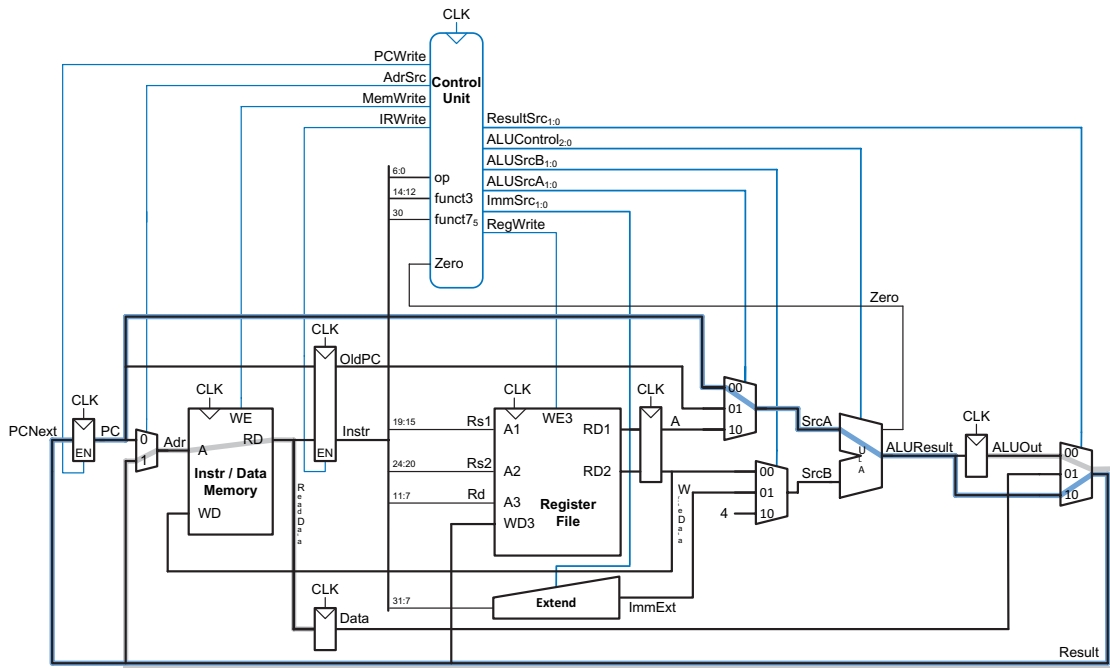


图7.46 多周期处理器潜在关键路径

### 示例7.7 多周期处理器CPI

SPECINT2000基准测试包含大约25%的加载指令、10%的存储指令、11%的分支指令、2%的跳转指令以及52%的R型或I型ALU指令。<sup>2</sup> 计算该基准测试的平均CPI。

解 平均CPI是每条指令的CPI乘以其使用频率的加权。对于该基准测试，平均CPI =  $(0.11)(3) + (0.10 + 0.02 + 0.52)(4) + (0.25)(5) = 4.14$ 。这优于最坏情况下的CPI 5，如果所有指令都花费相同周期数，则需要该值。

回想一下，我们设计多周期处理器时，使得每个周期只涉及一次ALU操作、一次内存访问或一次寄存器文件访问。假设寄存器文件比内存快，且写入内存比读取内存快。检查数据通路会发现两条可能限制周期时间的关键路径，如图7.46所示：

1. 计算PC+4的路径：从PC寄存器出发，经过SrcA多路选择器、ALU和结果多路选择器，再返回PC寄存器（以粗蓝线突出显示）；或

<sup>2</sup>Instruction frequencies from Patterson and Hennessy, *Computer Organization and Design*, 4th Edition, Morgan Kaufmann, 2011.

2. 从内存读取数据的路径：从ALUOut寄存器经过Result和Adr多路选择器，将数据读入Data寄存器（以粗灰线高亮显示）

这两条路径在状态更新后（即经过 $t_{pcq}$ 延迟后）都需要经过解码器的延迟，以产生控制（多路选择器选择信号和 $ALUControl$ ）信号。因此，时钟周期如公式7.4所示。

$$T_{c\_multi} = t_{pcq} + t_{dec} + t_{mux} + t_{ALU} + t_{mem} + t_{setup} \quad (7.4)$$

这些时间的数值将取决于具体的实现技术。

示例7.8 多周期处理器性能比较。

Ben Bitdiddle 想知道多周期处理器是否比单周期处理器更快。对于这两种设计，他计划使用 7 纳米 CMOS 制造工艺，其延迟如表 7.7（第 415 页）所示。请帮助他比较每个处理器执行 SPECINT2000 基准测试中 1000 亿条指令的执行时间（参见示例 7.4）。

解 根据公式7.4，多周期处理器的周期时间为  $T_{c\_multi} = t_{pcq} + t_{dec} + 2t_{mux} + t_{mem} + t_{setup} = 40 + 25 + 2(30) + 200 + 50 = 375$  ps。使用例7.7中的CPI 4.14，总执行时间为  $T_{multi} = (100 \times 10^9 \text{ 条指令})(4.14 \text{ 周期/指令})(375 \times 10^{-12} \text{ 秒/周期}) = 155$  秒。

根据示例7.4，单周期处理器的总执行时间为75秒，因此多周期处理器更慢。

构建多周期处理器的初衷之一，是为了避免让所有指令都像最慢指令那样耗时。然而，该示例表明，在给定的CPI和电路元件延迟假设下，多周期处理器比单周期处理器更慢。根本问题在于，即使最慢的指令lw被拆分为五个步骤，多周期处理器的周期时间也并未实现近五倍的改善。部分原因是各步骤长度并不完全相同，另一部分原因是寄存器时钟到Q端延迟与建立时间这90皮秒的时序开销，现在必须在每个步骤中支付，而非仅在整个指令中支付一次。总体而言，工程师们认识到，除非计算速度差异显著，否则很难利用某些计算比其他计算更快这一特性。

与单周期处理器相比，多周期处理器可能成本更低，因为它共享一个用于指令和数据的存储器，并且省去了两个加法器。然而，它确实需要五个非架构寄存器和额外的多路选择器。

## 7.5 流水线处理器

流水线（第3.6节介绍）是提升数字系统吞吐量的强大方法。我们通过将单周期处理器细分为五个流水线阶段来设计流水线处理器。这样，五条指令可以同时执行，每个阶段执行一条。由于每个阶段仅包含整个逻辑的五分之一，时钟频率大约提升五倍。因此，理想情况下每条指令的延迟保持不变，但吞吐量提升五倍。微处理器每秒执行数百万或数十亿条指令，因此吞吐量比延迟更为重要。流水线会引入一些开销，因此吞吐量无法达到理想中的最高水平，但流水线以极低的成本带来了巨大优势，以至于所有现代高性能微处理器都采用流水线设计。

回想一下，*throughput* 是每秒完成的任务数（在此情况下为指令数）。*Latency* 是指令从开始到结束所需的时间。（参见第3.6节）

对内存和寄存器文件进行读写以及使用ALU通常是处理器中最大的延迟来源。我们选择五个流水线阶段，使得每个阶段恰好包含这些慢速步骤之一。具体来说，我们将这五个阶段称为*Fetch*、*Decode*、*Execute*、*Memory*和*Writeback*。它们类似于多周期处理器执行*lw*指令时所使用的五个步骤。在*Fetch*阶段，处理器从指令内存中读取指令。在*Decode*阶段，处理器从寄存器文件中读取源操作数，并对指令进行解码以生成控制信号。在*Execute*阶段，处理器使用ALU执行计算。在*Memory*阶段，处理器读取或写入数据内存（如果适用）。最后，在*Writeback*阶段，处理器将结果写入寄存器文件（如果适用）。

图7.47展示了比较单周期与流水线处理器的时序图。时间轴为水平方向，指令轴为垂直方向。该图假设了表7.7（见第415页）中的组件延迟，但为简化起见忽略了多路选择器和寄存器。在图7.47(a)的单周期处理器中，第一条指令在时间0从存储器读取。接着，操作数从寄存器文件中读取。然后，ALU执行必要的计算。最后，可能访问数据存储器，并在680 ps时将结果写回寄存器文件。第二条指令在第一条指令完成后开始。因此，在该图中，单周期处理器的指令延迟为 $200 + 100 + 120 + 200 + 60 = 680$  ps（见表7.7第415页），吞吐量为每680 ps一条指令（14.7亿条指令每秒）。

在图7.47(b)所示的流水线处理器中，流水线级长度由最慢的级（取指或访存阶段的存储器访问）设定为200 ps。每个流水线级由蓝色实线或虚线垂直标出。在时间0时，第一条指令从存储器中取出。在200 ps时，第一条指令进入译码阶段，同时第二条指令被取出。在400 ps时，第一条指令执行，第二条指令进入译码阶段，同时第三条指令被取出。

Remember that for this abstract comparison of single-cycle and pipelined processor performance, we are ignoring the overhead of decoder, multiplexer, and register delays.



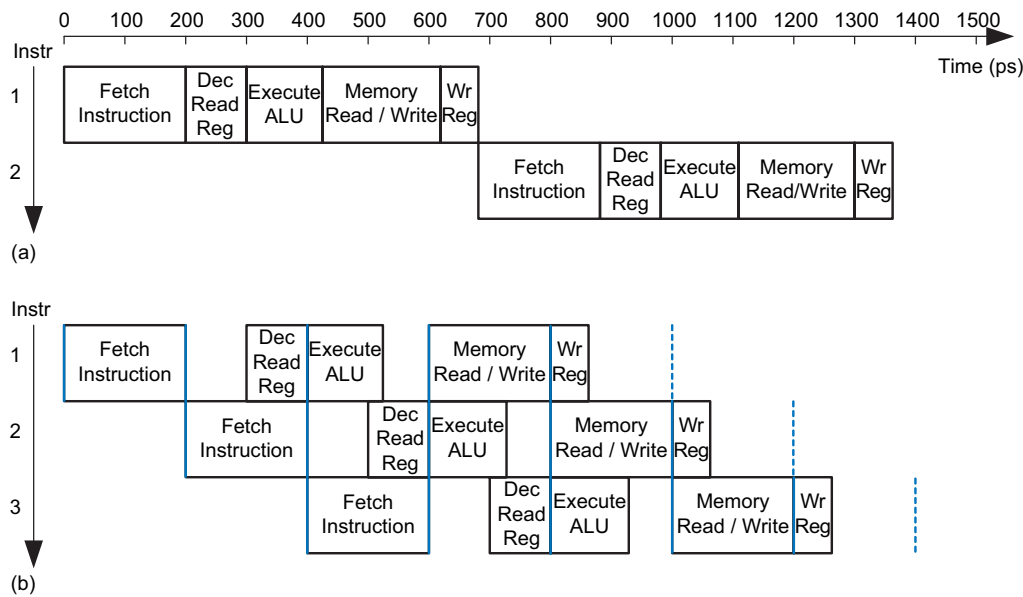


图7.47 时序图：(a) 单周期处理器 (b) 流水线处理器

依此类推，直到所有指令完成。指令延迟为 $5 \times 200 = 1000$ 皮秒。由于各阶段逻辑量不完全均衡，流水线处理器的延迟比单周期处理器更长。吞吐量为每200皮秒一条指令（每秒50亿条指令），即每个时钟周期完成一条指令。这一吞吐量是单周期处理器的3.4倍——虽未达到5倍，但仍实现了显著的加速。

图7.48展示了流水线运行时的抽象视图，其中每个阶段均以图形方式呈现。每个流水线阶段通过其主要组件——指令存储器（IM）、寄存器文件（RF）读取、ALU执行、数据存储器（DM）和寄存器文件写回——来示意指令在流水线中的流动。横向阅读一行可显示特定指令在每个阶段所处的时钟周期。例如，`sub`指令在第3周期被取指，在第5周期执行。纵向阅读一列可显示特定周期中各流水线阶段的操作。例如，在第6周期，寄存器文件正在将和写入`s3`，数据存储器空闲，ALU正在计算`s11 & t0`，`t4`正在从寄存器文件读取，而`or`指令正在从指令存储器取指。阶段以阴影标记表示其使用状态。例如，数据存储器在第4周期被`lw`使用，在第8周期被`sw`使用。指令存储器和ALU在每个周期均被使用。除`sw`外，每条指令都会写入寄存器文件。在流水线处理器中，寄存器文件

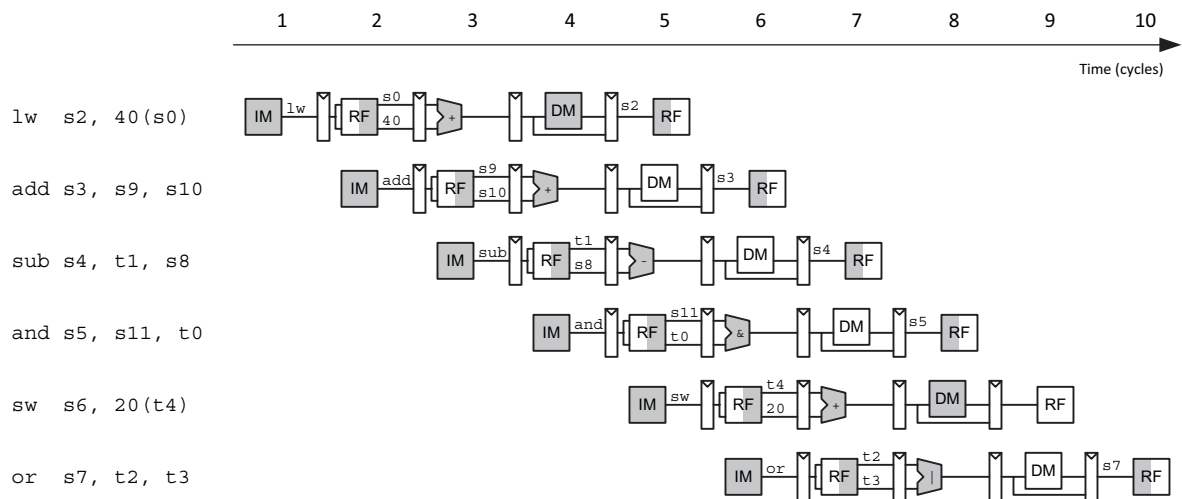


图7.48 流水线运行的抽象视图

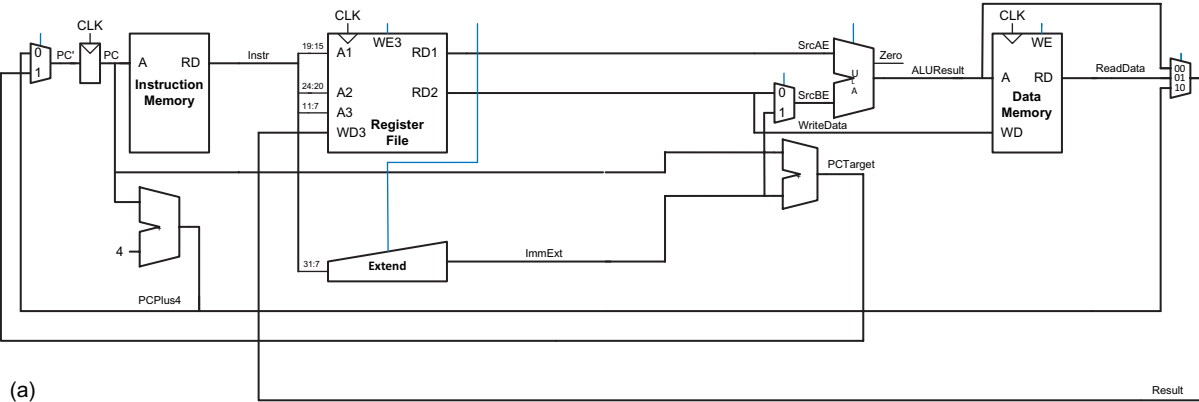
每个周期使用两次：如阴影所示，它在周期的第一部分被写入，在第二部分被读取。这样，数据可以在一个周期内由一条指令写入并由另一条指令读取。

流水线系统中的一个核心挑战是处理当一条指令的结果在先前指令完成之前就被后续指令所需时发生的冒险。例如，如果图7.48中的加法指令使用s2而非s10作为源操作数，就会发生冒险，因为在第3个周期加法指令读取s2寄存器时，lw指令尚未写入该寄存器。在设计流水线数据通路和控制逻辑之后，本节将探讨*forwarding*、*stalls*和*flushes*作为解决冒险的方法。最后，本节将重新审视考虑时序开销和冒险影响的性能分析。

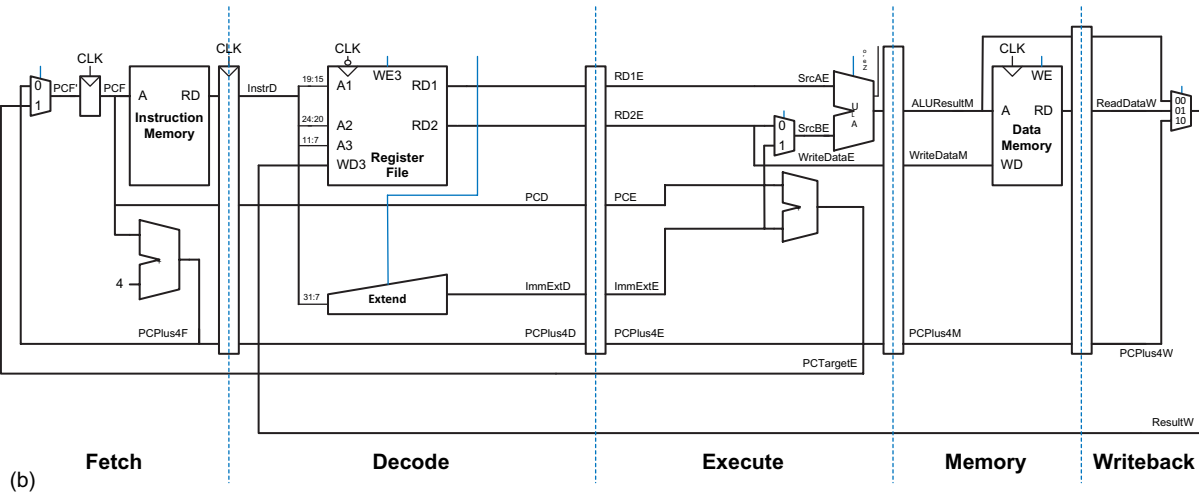
7.5.1 流水线数据通路

流水线数据路径通过将单周期数据路径切割为五个阶段，并用流水线寄存器分隔各阶段而形成。图7.49(a)展示了为容纳流水线寄存器而拉长的单周期数据路径。图7.49(b)展示了通过插入四个流水线寄存器将数据路径分隔为五个阶段后形成的流水线数据路径。各阶段及其边界以蓝色标示。信号带有后缀（F、D、E、M或W），以指示其所属的阶段。

寄存器文件较为特殊，因为它在译码阶段被读取，而在写回阶段被写入。因此，尽管寄存器文件绘制在译码阶段，但其写入地址和写入数据来自写回阶段。这种反馈将导致流水线冒险，相关内容将在



(a)



(b)

Fetch

Decode

Execute

Memory

Writeback

图7.49 数据通路：(a)单周期与(b)流水线

7.5.3节。流水线处理器中的寄存器文件在CLK的下降沿写入，以便在周期的前半部分写入结果，并在周期的后半部分读取该结果，供后续指令使用。

流水线中一个微妙但关键的问题是，与特定指令相关的所有信号必须同步通过流水线。图7.49(b)中存在与此问题相关的错误。你能找到它吗？

错误位于寄存器文件写入逻辑中，该逻辑应在写回阶段运行。数据值来自写回阶段信号ResultW，但目标寄存器来自解码阶段信号RdD（InstrD<sub>11:7</sub>）。在图7.48的流水线图中，在第5个周期，lw指令的结果将被错误地写入s5而非s2。

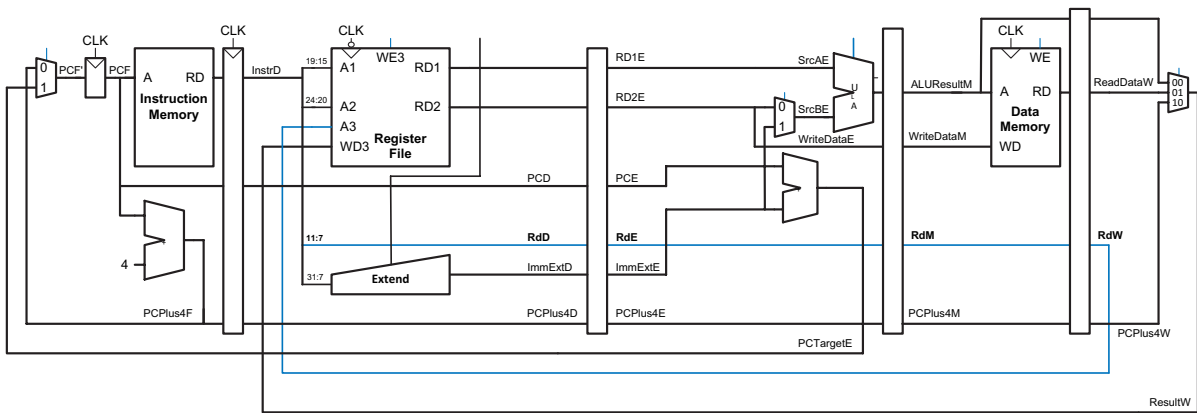


图7.50 修正后的流水线数据通路

图7.50展示了一条修正后的数据通路，修改部分以蓝色标出。*Rd*信号现在通过执行级、存储级和写回级进行流水线传输，从而与指令的其余部分保持同步。*RdW*和*ResultW*在写回级一起反馈回寄存器文件。

敏锐的读者可能会注意到，生成*PCF'*（下一个PC）的逻辑也存在问題，因为它可能通过取指阶段或执行阶段的信号（*PCPlus4F*或*PCTargetE*）进行更新。这一控制冒险将在第7.5.3节中解决。

### 7.5.2 流水线控制

流水线处理器使用与单周期处理器相同的控制信号，因此具有相同的控制单元。控制单元在译码阶段检查指令的*op*、*funct3*和*funct7*<sub>5</sub>字段以生成控制信号，如第7.3.3节对单周期处理器所述。这些控制信号必须与数据一起流水线传递，以保持与指令同步。

包含控制单元的完整流水线处理器如图7.51所示。*RegWrite*必须在写回阶段进行流水线处理，然后才能反馈到寄存器文件，就像*Rd*在图7.50中进行了流水线处理一样。除了R型ALU指令、*lw*、*sw*和*beq*之外，该流水线处理器还支持*jal*和I型ALU指令。

### 7.5.3 危害

在流水线系统中，多条指令被并发处理。当一条指令*dependent*依赖于另一条尚未完成的结果时，

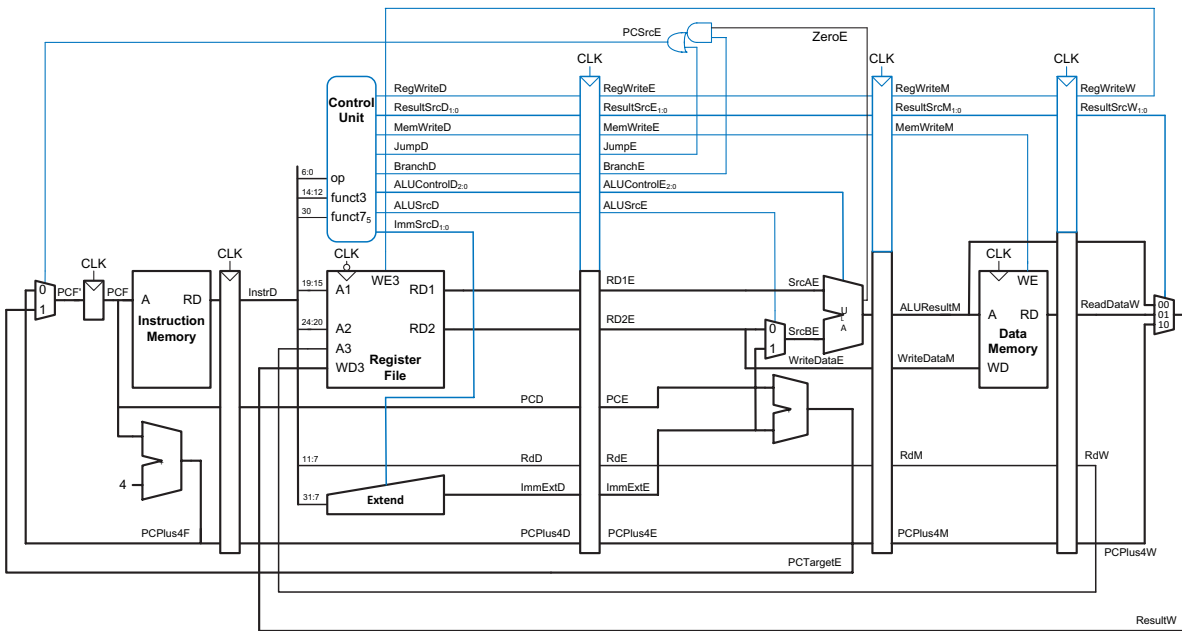


图7.51 带控制单元的流水线处理器

尚未完成时，发生**hazard**。寄存器文件在周期前半段写入，后半段读取，因此寄存器可在同一周期内写入并读回，而不会引入冒险。

图7.52展示了一条指令写入寄存器(s8)而后续指令读取该寄存器时发生的冒险。蓝色箭头突出显示了s8写入寄存器文件的时间（第5周期）与后续指令需要该值的时间对比。这被称为**read after write (RAW) hazard**。add指令在第5周期前半段将结果写入s8。然而，sub指令在第3周期读取s8，获取了错误的值。or指令在第4周期读取s8，同样获取了错误的值。and指令在第5周期后半段读取s8，获取了正确的值（该值在第5周期前半段被写入）。后续指令也能正确读取s8的值。该图表明，当一条指令写入寄存器，且后续两条指令中的任意一条读取该寄存器时，该流水线中可能发生冒险。若不进行特殊处理，流水线将计算出错误的结果。

一种软件解决方案是要求程序员或编译器在add和sub指令之间插入nop指令，以便依赖指令在结果(s8)可用之前不会读取它。

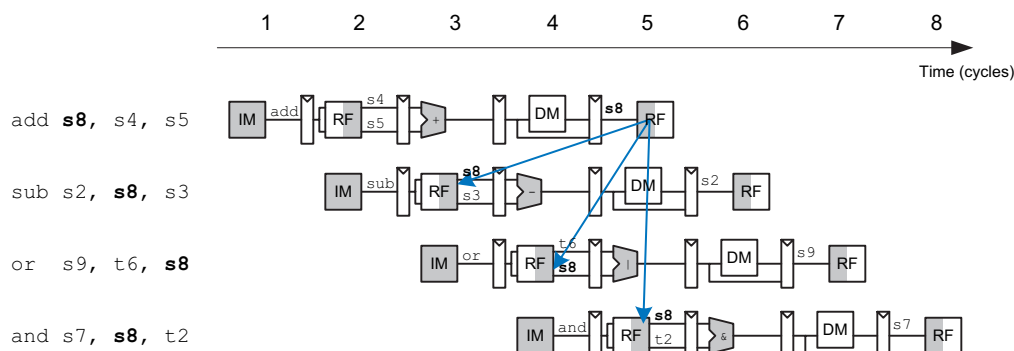


Figure 7.52 Abstract pipeline diagram illustrating hazards

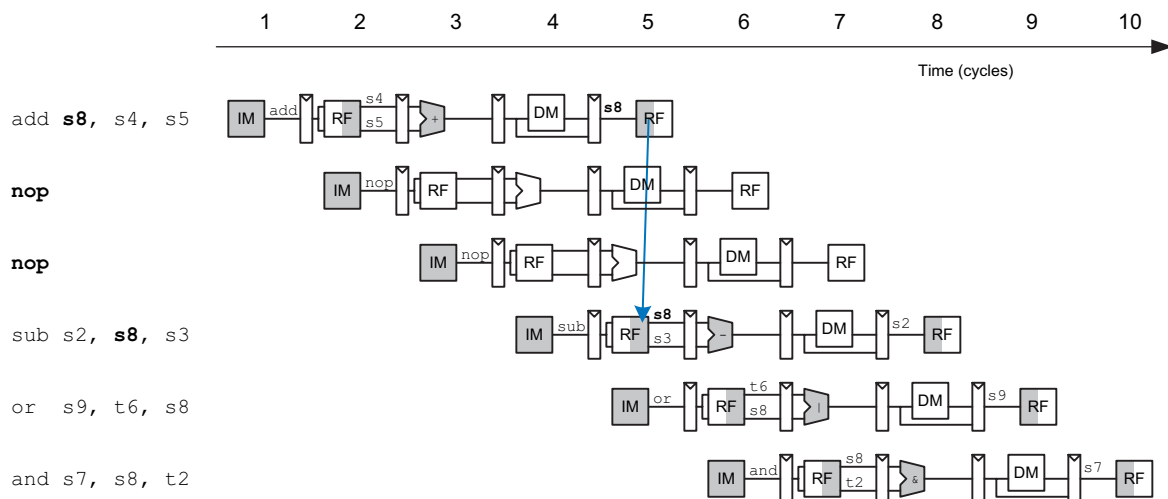


图7.53 使用nops解决数据冒险

在寄存器文件中，如图7.53所示。这样的*software interlock*会使编程复杂化并降低性能，因此并不理想。

仔细查看图7.52，可以发现加法指令的结果在第三个周期由ALU计算得出，而and指令直到第四个周期才需要使用该结果。原则上，我们应当能够将一条指令的结果直接转发给下一条指令，以解决RAW冒险，而无需等待结果写入寄存器文件，也无需减慢流水线速度。在本节后续讨论的其他情况下，我们可能需要暂停流水线，以便在后续指令使用结果之前留出时间生成结果。无论如何，必须采取措施解决冒险问题，以确保程序在流水线环境下能够正确执行。

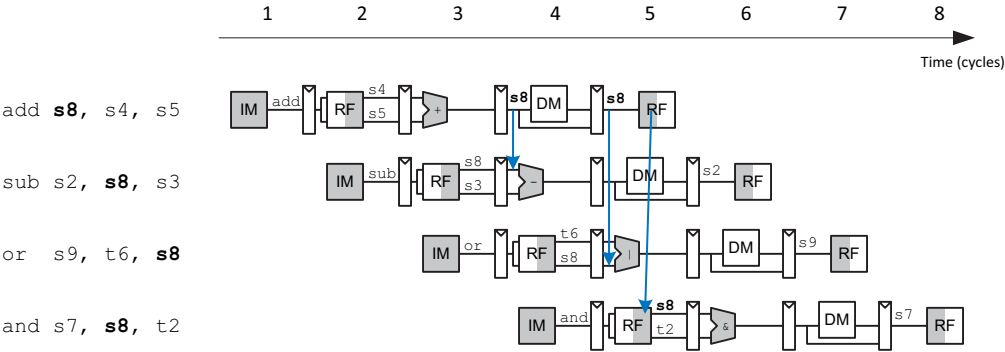


图7.54 说明转发机制的抽象流水线图

冒险分为数据冒险和控制冒险。当一条指令试图读取尚未由前一条指令写回的寄存器时，会发生 *data hazard*。当取指时尚未决定下一条要取哪条指令时，会发生 *control hazard*。在本节剩余部分，我们将为流水线处理器增加一个冒险检测单元，该单元能够检测冒险并妥善处理，从而使处理器正确执行程序。

使用转发解决数据冒险

某些数据冒险可以通过 *forwarding*（也称为 *bypassing*）解决，即从存储器阶段或写回阶段将结果转发给执行阶段中的依赖指令。这需要在ALU前添加多路选择器，以从寄存器文件、存储器阶段或写回阶段中选择其操作数。图7.54展示了这一原理。该程序使用 `add` 指令计算 `s8`，然后在后续三条指令中使用 `s8`。在第4个周期，`s8` 从 `add` 指令的存储器阶段转发到依赖的 `sub` 指令的执行阶段。在第5个周期，`s8` 从 `add` 指令的写回阶段转发到依赖的 `or` 指令的执行阶段。同样，`and` 指令不需要转发，因为 `s8` 在第5个周期的前半段写入寄存器文件，并在后半段被读取。

当执行阶段的指令的源寄存器与存储器或写回阶段指令的目的寄存器匹配时，需要进行转发。图7.55修改了流水线处理器以支持转发。它增加了一个 *Hazard Unit* 和两个 *forwarding multiplexers*。冒险检测单元从执行阶段的指令中接收两个源寄存器 `Rs1E` 和 `Rs2E`，以及从存储器阶段和写回阶段的指令中接收目的寄存器。

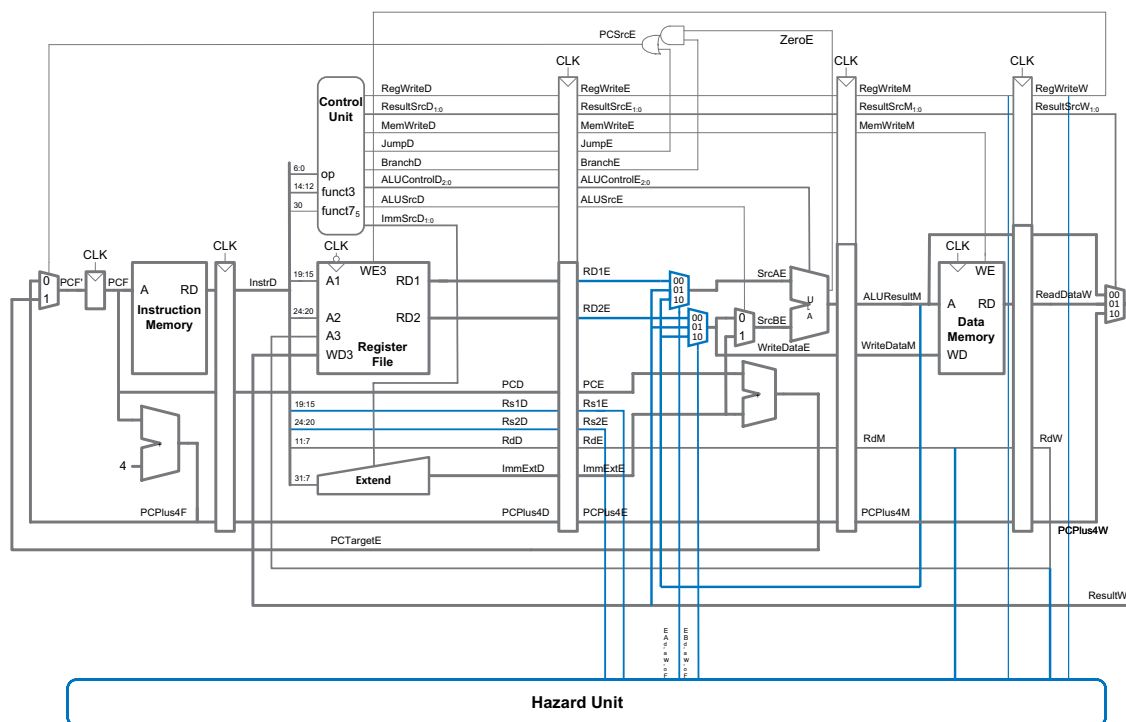


图7.55 采用转发机制解决部分数据冒险的流水线处理器

阶段， $RdM$ 和 $RdW$ 。它还从内存和写回阶段（ $RegWriteM$ 和 $RegWriteW$ ）接收 $RegWrite$ 信号，以了解目标寄存器是否实际会被写入（例如， $sw$ 和 $beq$ 指令不会将结果写入寄存器文件，因此不会转发其结果）。

冒险单元计算转发多路复用器的控制信号，以从寄存器文件或存储器/写回阶段（ $ALUResultM$ 或 $ResultW$ ）的结果中选择操作数。如果某个阶段将写入目标寄存器 $and$ ，且该目标寄存器与源寄存器匹配，则冒险单元应从该阶段转发。然而， $x0$ 被硬连线为0，因此永远不应转发。如果存储器阶段和写回阶段都包含匹配的目标寄存器，则存储器阶段应具有优先权，因为它包含最近执行的指令。总之，用于 $SrcAE$ （ $ForwardAE$ ）的转发逻辑功能在下一页给出。用于 $SrcBE$ （ $ForwardBE$ ）的转发逻辑与前者相同，只是它检查 $Rs2E$ 而非 $Rs1E$ 。



如果  $((Rs1E == RdM) \& RegWriteM) \& (Rs1E != 0)$  则 // 从内存阶段转发  $ForwardAE = 10$   
 否则如果  $((Rs1E == RdW) \& RegWriteW) \& (Rs1E != 0)$  则 // 从写回阶段转发  $ForwardAE = 01$   
 否则  $ForwardAE = 00$

// 无转发（使用射频输出）

使用停顿解决数据冒险

转发足以解决当结果在指令的执行阶段计算完成时的RAW数据冒险，因为其结果可以转发到下一条指令的执行阶段。不幸的是，lw指令直到访存阶段结束时才完成数据读取，因此其结果无法转发到下一条指令的执行阶段。我们称lw指令存在*two-cycle latency*，因为依赖指令必须等待两个周期后才能使用其结果。图7.56展示了这一问题。lw指令在周期4结束时从内存接收数据，但and指令需要在周期4开始时就将该数据（s7中的值）作为源操作数。这种情况下无法通过转发来解决该冒险。

一种解决方案是*stall*流水线，暂停操作直到数据可用。图7.57展示了在译码阶段暂停依赖指令（and）。and在周期3进入译码阶段，并在周期4期间暂停在那里。后续指令（or）也必须在这两个周期内保持在取指阶段，因为译码阶段已满。

在周期5中，结果可以从lw的写回阶段转发到and的执行阶段。同样在周期5中，or指令的源操作数s7直接从寄存器文件中读取，无需转发。

请注意，执行阶段在第4个周期未被使用。同样，存储器在第5个周期未被使用，写回在第6个周期未被使用。这种未使用的阶段在流水线中传播被称为*bubble*，其行为类似于空操作指令。气泡是通过在译码阶段停顿期间将执行阶段控制信号清零而引入的，因此气泡不执行任何操作，也不改变任何架构状态。

总之，停顿一个阶段是通过禁用其流水线寄存器（即该阶段左侧的寄存器）来实现的，这样该阶段的输入就不会改变。当一个阶段被停顿时，所有之前的阶段也必须被停顿，以防止后续指令丢失。紧接在停顿阶段之后的流水线寄存器必须被清空（刷新），以防止虚假信息向前传播。停顿会降低性能，因此应仅在必要时使用。

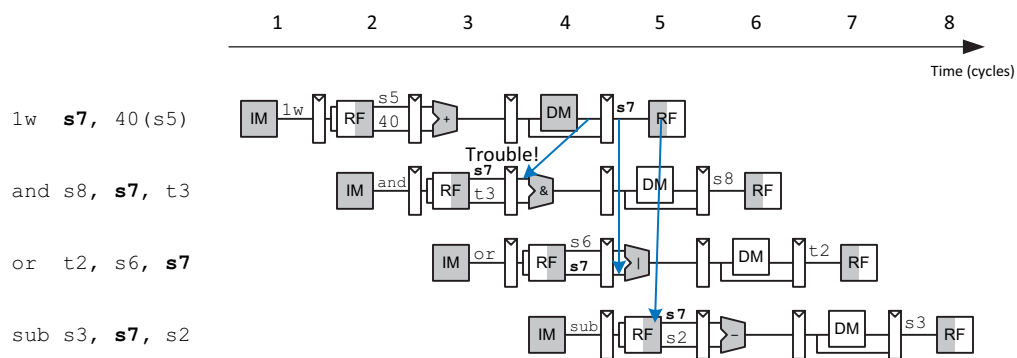


Figure 7.56 Abstract pipeline diagram illustrating trouble forwarding from lw

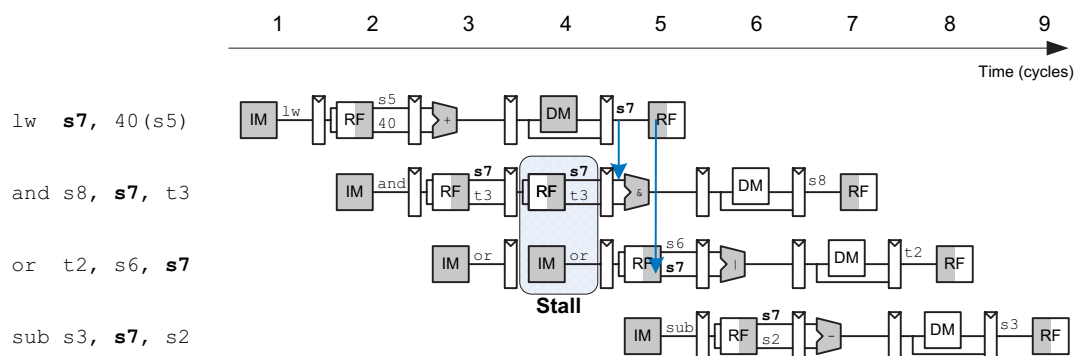


图7.57 说明停顿解决冒险的抽象流水线图

图7.58修改了流水线处理器，为lw数据依赖添加了停顿。为了使冒险单元停顿流水线，必须满足以下条件：

1. 加载字处于执行阶段（由  $ResultSrcE_0 = 1$  表示）且
2. 加载的目标寄存器（ $RdE$ ）与解码阶段指令的源操作数 $Rs1D$ 或 $Rs2D$ 匹配

通过向取指和译码流水线寄存器添加使能输入（ $EN$ ）以及向执行流水线寄存器添加同步复位/清零（ $CLR$ ）输入来支持停顿。当加载字（lw）停顿发生时， $StallD$ 和 $StallF$ 被置位，强制译码和取指阶段的流水线寄存器保持其现有值。 $FlushE$ 也被置位以清除执行阶段流水线寄存器的内容，从而引入一个气泡。冒险单元 $lwStall$ （加载字停顿）信号指示流水线何时

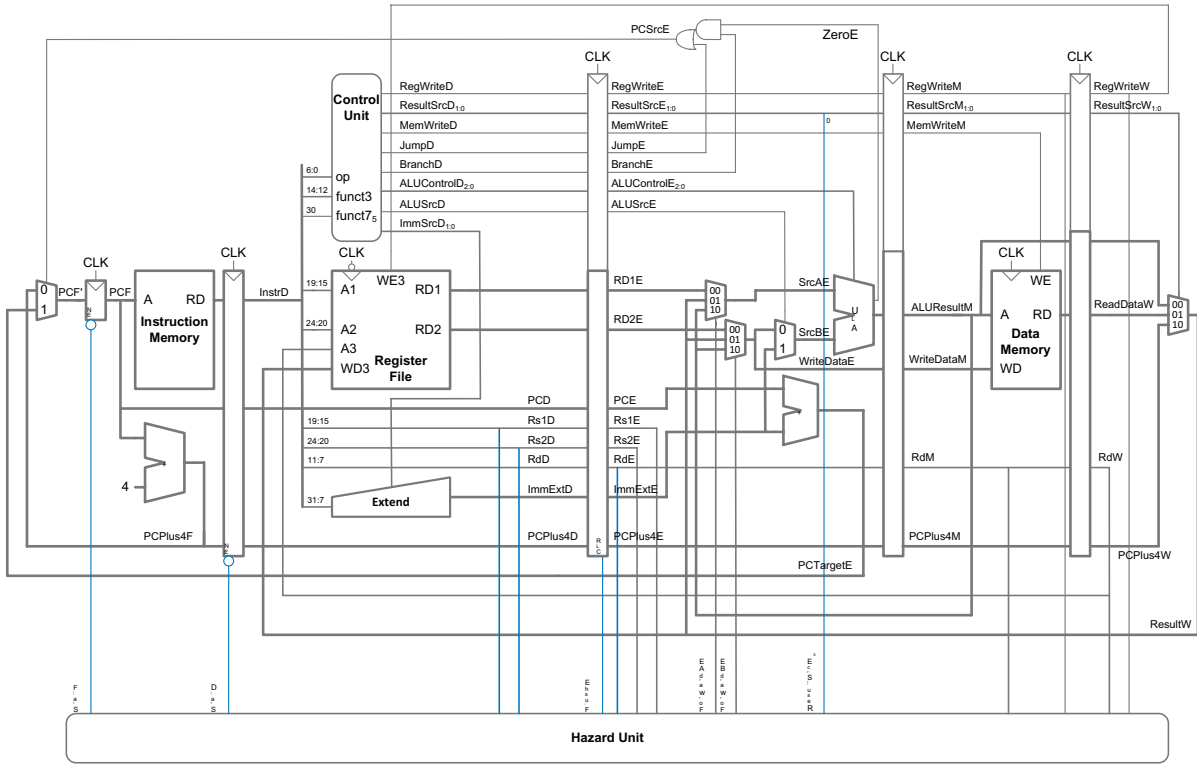


图7.58 使用停顿解决lw数据冒险的流水线处理器

此处描述的lwStall逻辑可能导致处理器在加载目标为x0或存在虚假依赖时（即译码阶段的指令为J型或I型指令，其立即数字段与RdE随机产生虚假匹配）发生不必要的停顿。不过，这些情况较为罕见（且若x0为加载目标，则属于不良编码实践）。

并且它们只会导致很小的性能损失。

由于加载字依赖，应暂停执行。每当lwStall为真时，所有暂停和刷新信号均被断言。因此，计算暂停和刷新的逻辑为：

$$lwStall = ResultSrcE_0 \ \& \ ((Rs1D == RdE) \vee (Rs2D == RdE))$$
$$StallF = StallD = FlushE = lwStall$$

解决控制冒险  
beq指令带来了控制冒险：流水线处理器在取指下一条指令时，由于分支决策尚未做出，因此无法确定接下来该取哪条指令。

处理这种控制冒险的一种机制是暂停流水线，直到分支决策做出（即计算出PCSrcE）。由于决策在执行阶段做出，每个分支处流水线将不得不暂停两个周期。如果分支频繁出现（这通常是常见情况），这将严重降低系统性能。

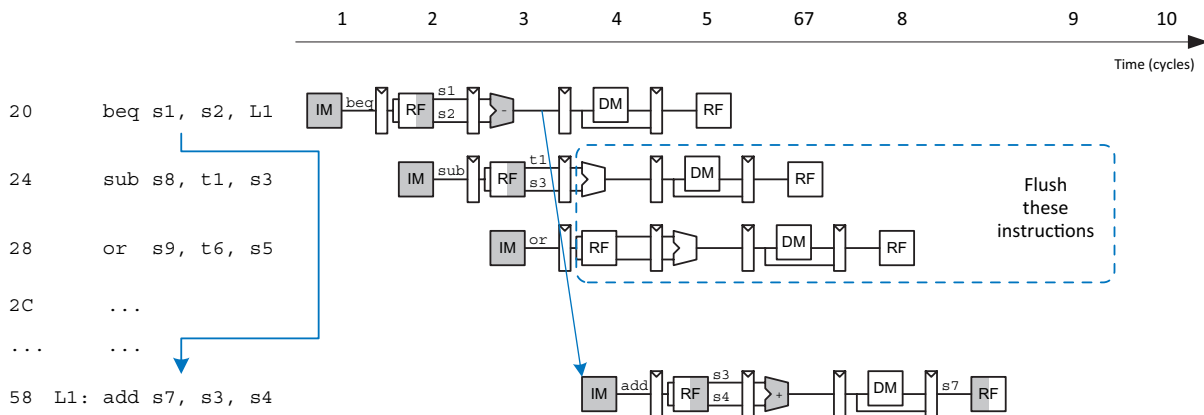


图7.59 抽象流水线图，展示了分支发生时清空流水线的过程

另一种避免流水线停顿的方法是预测分支是否会被执行，并基于该预测开始执行指令。一旦分支决策可用，如果预测错误，处理器可以丢弃这些指令。在目前介绍的流水线（图7.58）中，处理器预测分支不会被执行，并继续按顺序执行程序，直到PCSrcE被断言以从PCTargetE中选择下一个PC。如果本应执行分支，则必须通过清除这些指令的流水线寄存器来flushed（丢弃）分支后的两条指令。这些浪费的指令周期被称为 *branch misprediction penalty*。

图7.59展示了这样一种方案，其中从地址0x20到地址0x58的分支被采用。PC直到周期3才被写入，此时地址0x24和0x28处的sub和or指令已经被取出。这些指令必须被清除，而add指令在周期4从地址0x58取出。

最后，我们必须计算出停顿和冲刷信号，以处理分支和PC写入。当分支被采用时，后续两条指令必须从译码和执行阶段的流水线寄存器中冲刷掉。因此，我们为译码流水线寄存器添加一个同步清零输入（CLR），并为冒险单元添加FlushD输出。（当CLR = 1时，寄存器内容被清零，即变为0。）当分支被采用时（由PCSrcE为1指示），必须断言FlushD和FlushE以冲刷译码和执行流水线寄存器。图7.60展示了用于处理控制冒险的增强型流水线处理器。现在冲刷计算如下：

$$\begin{aligned} FlushD &= PCSrcE = lwStall \mid PCSrcE \\ FlushE & \end{aligned}$$

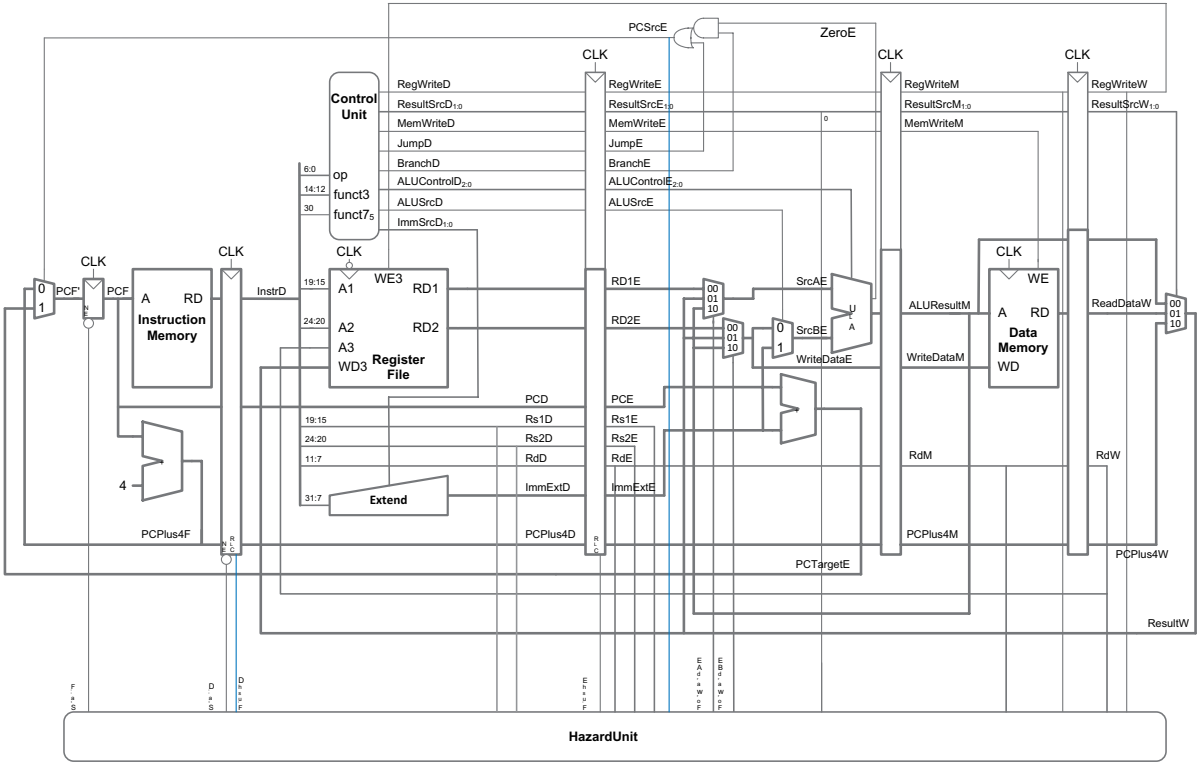


图7.60 用于处理分支控制冒险的扩展冒险单元

危害总结

总之，RAW数据冒险发生在一条指令依赖于另一条指令尚未写入寄存器文件的结果时。如果结果能及时计算出来，数据冒险可以通过转发解决；否则就需要暂停流水线直到结果可用。控制冒险发生在需要获取下一条指令时，尚未确定该获取哪条指令的情况下。控制冒险通过暂停流水线直到决策做出，或通过预测应获取哪条指令并在预测错误时空流水线来解决。尽早做出决策可以最大限度地减少因预测错误而清空的指令数量。您可能已经注意到，设计流水线处理器的挑战之一在于理解指令之间所有可能的交互，并发现所有可能存在的冒险。图 7.61展示了处理所有冒险的完整流水线处理器。冒险逻辑总结在下一页。

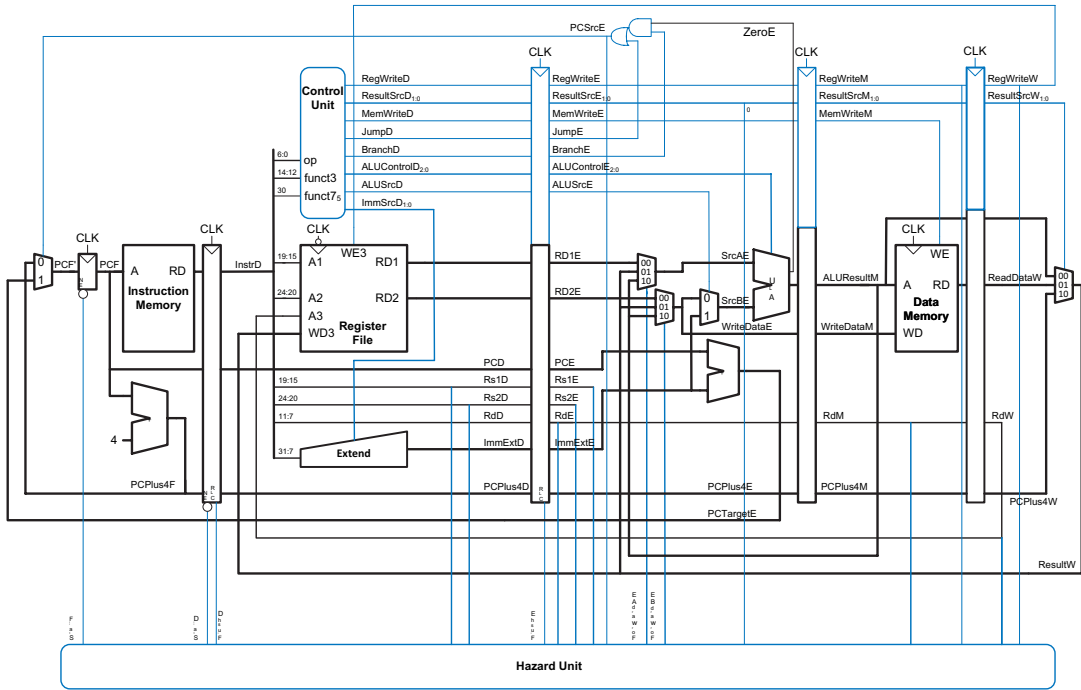


图7.61 具有完整冒险处理功能的流水线处理器

尽可能向前解决数据冒险<sup>3</sup>: 如果  $((Rs1E == RdM) \& RegWriteM) \& (Rs1E != 0)$  则  $ForwardAE = 10$  否则  
 如果  $((Rs1E == RdW) \& RegWriteW) \& (Rs1E != 0)$  则  $ForwardAE = 01$  否则  $ForwardAE = 00$

当发生负载冲突时停顿:

$$lwStall = ResultSrcE_0 \& ((Rs1D == RdE) \vee (Rs2D == RdE))$$

$$StallF = lwStall = lwStall$$

$$StallD$$

当分支被采用或加载引入气泡时刷新

le:

$$FlushD = PCSrcE = lwStall \vee PCSrcE$$

$$FlushE$$

<sup>3</sup> Recall that the forwarding logic for  $SrcBE$  ( $ForwardBE$ ) is identical except that it checks  $Rs2E$  instead of  $Rs1E$ .

### 7.5.4 性能分析

流水线处理器理想情况下CPI为1，因为每个周期都会取一条新指令（即 *issued*）。然而，一次停顿或冲刷会浪费1到2个周期，因此CPI略高，且取决于所执行的具体程序。

#### 示例7.9 流水线处理器CPI

示例7.4中考虑的SPECINT2000基准测试包含约25%的加载指令、10%的存储指令、11%的分支指令、2%的跳转指令以及52%的R型或I型ALU指令。假设40%的加载指令后紧跟着一条使用其结果的指令，需要插入一个停顿周期；同时50%的分支指令被预测为跳转（即预测错误），需要冲刷两条指令。忽略其他冒险。计算该流水线处理器的平均CPI。

**解** 平均CPI是每条指令的CPI乘以该指令使用频率的加权和。加载指令在无依赖时占用一个时钟周期，在处理器必须因依赖而停顿时占用两个周期，因此其CPI为 $(0.6)(1) + (0.4)(2) = 1.4$ 。分支指令在正确预测时占用一个时钟周期，预测错误时占用三个周期，因此其CPI为 $(0.5)(1) + (0.5)(3) = 2$ 。跳转指令占用三个时钟周期（CPI = 3）。所有其他指令的CPI为1。因此，对于该基准测试，平均CPI =  $(0.25)(1.4) + (0.1)(1) + (0.11)(2) + (0.02)(3) + (0.52)(1) = 1.25$ 。

我们可以通过考虑图7.61所示的五个流水线阶段中的关键路径来确定周期时间。请注意，寄存器文件在一个周期内被使用两次：它在写回阶段的前半部分被写入，并在译码阶段的后半部分被读取；因此这些阶段只能使用半个周期时间来处理其关键路径。另一种表述方式是：这些阶段中每个阶段的关键路径的两倍必须适应一个周期。图7.62展示了执行阶段的关键路径。当一条分支指令处于执行阶段且需要从写回阶段进行转发时，该路径出现：路径从写回流水线寄存器开始，经过结果、ForwardBE和SrcB多路选择器，通过ALU和与或逻辑到达PC多路选择器，最终到达PC寄存器。

执行阶段的临界路径分析假设，用于计算ForwardAE和ForwardBE的冒险单元延迟小于或等于结果多路选择器的延迟。如果冒险单元延迟更长，则必须将其纳入临界路径，以替代结果多路选择器的延迟。

$$T_{c\_pipelined} = \max \left[ \begin{array}{l} t_{pcq} + t_{mem} + t_{setup} \\ 2(t_{RFread} + t_{setup}) \\ t_{pcq} + 4t_{mux} + t_{ALU} + t_{AND-OR} + t_{setup} \\ t_{pcq} + t_{mem} + t_{setup} \\ 2(t_{pcq} + t_{mux} + t_{RFsetup}) \end{array} \right. \begin{array}{l} \text{Fetch} \\ \text{Decode} \\ \text{Execute} \\ \text{Memory} \\ \text{Writeback} \end{array} \quad (7.5)$$

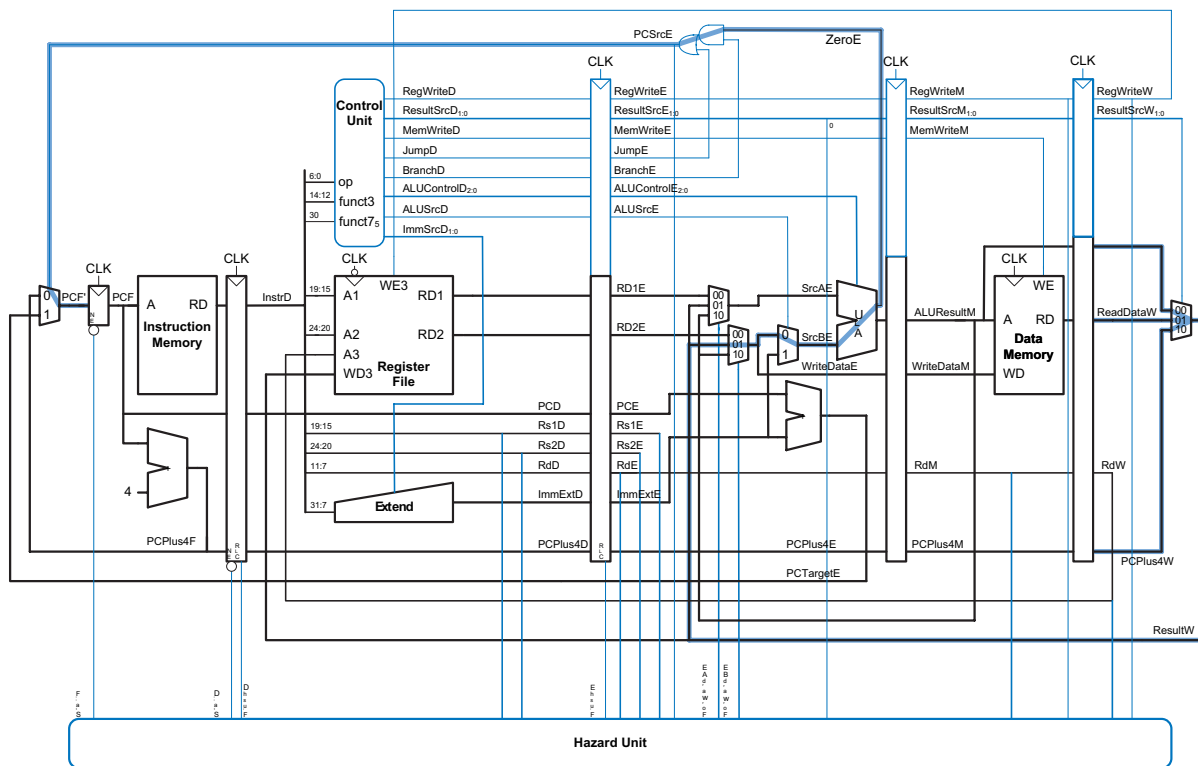


图7.62 流水线处理器关键路径

### 示例7.10 流水线处理器性能比较

Ben Bitdiddle需要将流水线处理器的性能与示例7.4和7.8中考虑的单周期及多周期处理器进行比较。逻辑延迟已在表7.7（第415页）中给出。请帮助Ben比较每个处理器执行SPECINT2000基准测试中1000亿条指令的执行时间。

解 根据公式7.5，流水线处理器的周期时间为  $T_{c\_pipelined} = \max[40 + 200 + 50, 2(100 + 50), 40 + 4(30) + 120 + 20 + 50, 40 + 200 + 50, 2(40 + 30 + 60)] = 350$  ps。执行阶段耗时最长。根据公式7.1，总执行时间为  $T_{pipelined} = (100 \times 10^9 \text{ 条指令}) (1.25 \text{ 周期/指令}) (350 \times 10^{-12} \text{ 秒/周期}) = 44$  秒。相比之下，单周期处理器需要75秒，多周期处理器需要155秒。

流水线处理器的速度明显快于其他处理器。然而，与单周期处理器相比，它的优势远未达到人们期望从五级流水线中获得的五倍加速比。

我们的流水线处理器存在不平衡问题，执行阶段的分支解析耗时远超过其他阶段。通过将结果多路选择器移回内存阶段，可将周期时间缩短至320 ps，从而更好地平衡流水线。



流水线冒险会引入较小的CPI惩罚。更重要的是，寄存器的时序开销（时钟到Q延迟和建立时间）适用于每个流水线阶段，而不仅仅是一次性作用于整个数据通路。时序开销限制了人们期望从流水线中获得的收益。流水线阶段中延迟的不平衡也会降低流水线的优势。流水线处理器在硬件需求上与单周期处理器相似，但它增加了许多32位流水线寄存器，以及多路选择器、较小的流水线寄存器和用于解决冒险的控制逻辑。

7.6 HDL表示\*

本节给出了支持本章所讨论指令的单周期RISC-V处理器的HDL代码。该代码展示了中等复杂度系统的良好编码实践。多周期处理器和流水线处理器的HDL代码留作练习7.25至7.27以及7.42至7.44。

在本节中，指令存储器和数据存储器与数据路径分离，并通过地址和数据总线连接。实际上，大多数处理器从独立的缓存中获取指令和数据。然而，为了处理数据可能与指令混合的较小内存映射，更完整的处理器还必须能够从指令存储器中读取数据（除指令外）。第8章将重新讨论内存系统，包括缓存与主内存的交互。

图7.63展示了单周期RISC-V处理器与外部存储器接口的框图。该处理器由

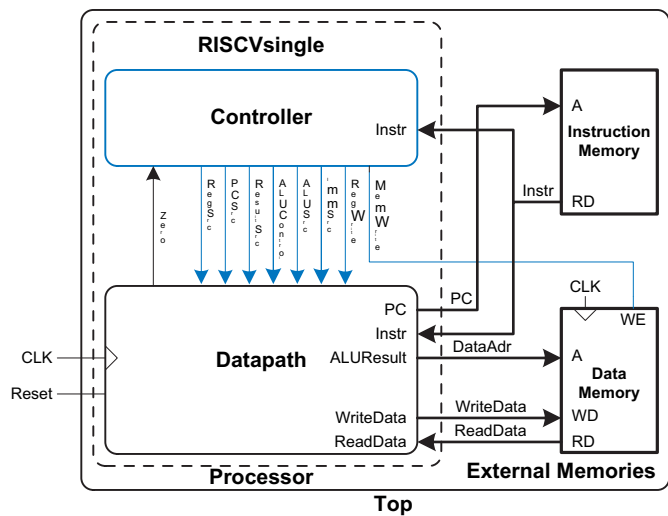


图7.63 单周期处理器与外部存储器接口

图7.15中的数据路径和图7.16中的控制器。而控制器又由主解码器和ALU解码器组成。

HDL 代码分为几个部分。第 7.6.1 节提供了单周期处理器数据通路和控制器的 HDL。第 7.6.2 节介绍了通用构建模块，例如寄存器和多路选择器，这些模块可用于任何微架构。第 7.6.3 节介绍了测试程序、测试平台和外部存储器。HDL 和测试程序以电子形式提供于本书网站（见前言）。

### 7.6.1 单周期处理器

单周期处理器模块的主要模块在以下HDL示例中给出。

#### HDL示例7.1 单周期处理器

##### SystemVerilog

```
module riscvsingle(input logic clk, reset,
                  output logic [31:0] PC,
                  input logic [31:0] Instr,
                  output logic MemWrite,
                  output logic [31:0] ALUResult, WriteData,
                  input logic [31:0] ReadData);

    logic ALUSrc, RegWrite, Jump, Zero;
    logic [1:0] ResultSrc, ImmSrc;
    logic [2:0] ALUControl;

    controller c(Instr[6:0], Instr[14:12], Instr[30], Zero,
                ResultSrc, MemWrite, PCSrc,
                ALUSrc, RegWrite, Jump,
                ImmSrc, ALUControl);
    datapath dp(clk, reset, ResultSrc, PCSrc,
                ALUSrc, RegWrite,
                ImmSrc, ALUControl,
                Zero, PC, Instr,
                ALUResult, WriteData, ReadData);
endmodule
```

##### VHDL

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;

entity riscvsingle is
    port(clk, reset: in STD_LOGIC;
         PC: out STD_LOGIC_VECTOR(31 downto 0);
         Instr: in STD_LOGIC_VECTOR(31 downto 0);
         MemWrite: out STD_LOGIC;
         ALUResult, WriteData: out STD_LOGIC_VECTOR(31 downto 0);
         ReadData: in STD_LOGIC_VECTOR(31 downto 0));
end;

architecture struct of riscvsingle is
    component controller
        port(op: in STD_LOGIC_VECTOR(6 downto 0);
             funct3: in STD_LOGIC_VECTOR(2 downto 0);
             funct7b5, Zero: in STD_LOGIC;
             ResultSrc: out STD_LOGIC_VECTOR(1 downto 0);
             MemWrite: out STD_LOGIC;
             PCSrc, ALUSrc: out STD_LOGIC;
             RegWrite, Jump: out STD_LOGIC;
             ImmSrc: out STD_LOGIC_VECTOR(1 downto 0);
             ALUControl: out STD_LOGIC_VECTOR(2 downto 0));
    end component;
    component datapath
        port(clk, reset: in STD_LOGIC;
             ResultSrc: in STD_LOGIC_VECTOR(1 downto 0);
             PCSrc, ALUSrc: in STD_LOGIC;
             RegWrite: in STD_LOGIC;
             ImmSrc: in STD_LOGIC_VECTOR(1 downto 0);
             ALUControl: in STD_LOGIC_VECTOR(2 downto 0);
             Zero: out STD_LOGIC;
             PC: out STD_LOGIC_VECTOR(31 downto 0);
             Instr: in STD_LOGIC_VECTOR(31 downto 0);
             ALUResult, WriteData: out STD_LOGIC_VECTOR(31 downto 0);
             ReadData: in STD_LOGIC_VECTOR(31 downto 0));
    end component;
```

|  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|--|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | <pre>signal ALUSrc, RegWrite, Jump, Zero, PCSrc: STD_LOGIC; signal ResultSrc, ImmSrc: STD_LOGIC_VECTOR(1 downto 0); signal ALUControl: STD_LOGIC_VECTOR(2 downto 0); begin     c: controller port map(Instr(6 downto 0), Instr(14 downto 12),                           Instr(30), Zero, ResultSrc, MemWrite,                           PCSrc, ALUSrc, RegWrite, Jump,                           ImmSrc, ALUControl);     dp: datapath port map(clk, reset, ResultSrc, PCSrc, ALUSrc,                           RegWrite, ImmSrc, ALUControl, Zero,                           PC, Instr, ALUResult, WriteData,                           ReadData);  end;</pre> |
|--|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

HDL示例7.2 控制器

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>SystemVerilog</b> <pre>module controller(input  logic [6:0] op,                   input  logic [2:0] funct3,                   input  logic      funct7b5,                   input  logic      Zero,                   output logic [1:0] ResultSrc,                   output logic      MemWrite,                   output logic      PCSrc, ALUSrc,                   output logic      RegWrite, Jump,                   output logic [1:0] ImmSrc,                   output logic [2:0] ALUControl);     logic [1:0] ALUOp;     logic      Branch;      maindec md(op, ResultSrc, MemWrite, Branch,                ALUSrc, RegWrite, Jump, ImmSrc, ALUOp);     aludec ad(op[5], funct3, funct7b5, ALUOp, ALUControl);      assign PCSrc = Branch &amp; Zero   Jump; endmodule</pre> | <b>VHDL</b> <pre>library IEEE; use IEEE.STD_LOGIC_1164.all;  entity controller is     port(op:          in      STD_LOGIC_VECTOR(6 downto 0);           funct3:     in      STD_LOGIC_VECTOR(2 downto 0);           funct7b5, Zero: in  STD_LOGIC;           ResultSrc:  out     STD_LOGIC_VECTOR(1 downto 0);           MemWrite:   out     STD_LOGIC;           PCSrc, ALUSrc: out   STD_LOGIC;           RegWrite:   out     STD_LOGIC;           Jump:       buffer  STD_LOGIC;           ImmSrc:     out     STD_LOGIC_VECTOR(1 downto 0);           ALUControl: out     STD_LOGIC_VECTOR(2 downto 0)); end;  architecture struct of controller is     component maindec         port(op:          in  STD_LOGIC_VECTOR(6 downto 0);               ResultSrc:  out STD_LOGIC_VECTOR(1 downto 0);               MemWrite:   out STD_LOGIC;               Branch, ALUSrc: out STD_LOGIC;               RegWrite, Jump: out STD_LOGIC;               ImmSrc:     out STD_LOGIC_VECTOR(1 downto 0);               ALUOp:      out STD_LOGIC_VECTOR(1 downto 0));     end component;     component aludec         port(opb5:      in  STD_LOGIC;               funct3:   in  STD_LOGIC_VECTOR(2 downto 0);               funct7b5: in  STD_LOGIC;               ALUOp:    in  STD_LOGIC_VECTOR(1 downto 0);               ALUControl: out STD_LOGIC_VECTOR(2 downto 0));     end component;      signal ALUOp: STD_LOGIC_VECTOR(1 downto 0);     signal Branch: STD_LOGIC; begin     md: maindec port map(op, ResultSrc, MemWrite, Branch,                         ALUSrc, RegWrite, Jump, ImmSrc, ALUOp);     ad: aludec port map(op(5), funct3, funct7b5, ALUOp, ALUControl);     PCSrc &lt;= (Branch and Zero) or Jump; end;</pre> |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## HDL示例7.3 主译码器

**SystemVerilog**

```

module maindec(input  logic [6:0] op,
               output logic [1:0] ResultSrc,
               output logic      MemWrite,
               output logic      Branch, ALUSrc,
               output logic      RegWrite, Jump,
               output logic [1:0] ImmSrc,
               output logic [1:0] ALUOp);
    logic [10:0] controls;

    assign {RegWrite, ImmSrc, ALUSrc, MemWrite,
           ResultSrc, Branch, ALUOp, Jump} = controls;

    always_comb
    case(op)
        // RegWrite_ImmSrc_ALUSrc_MemWrite_ResultSrc_Branch_ALUOp_Jump
        7'b0000011: controls = 11'b1_00_1_0_01_0_00_0; // lw
        7'b0100011: controls = 11'b0_01_1_1_00_0_00_0; // sw
        7'b0110011: controls = 11'b1_xx_0_0_00_0_10_0; // R-type
        7'b1100011: controls = 11'b0_10_0_0_00_1_01_0; // beq
        7'b0010011: controls = 11'b1_00_1_0_00_0_10_0; // I-type ALU
        7'b1101111: controls = 11'b1_11_0_0_10_0_00_1; // jal
        default:    controls = 11'bx_xx_x_x_xx_x_xx_x; // ???
    endcase
endmodule

```

**VHDL**

```

library IEEE;
use IEEE.STD_LOGIC_1164.all;

entity maindec is
    port(op:          in  STD_LOGIC_VECTOR(6 downto 0);
          ResultSrc:  out STD_LOGIC_VECTOR(1 downto 0);
          MemWrite:   out STD_LOGIC;
          Branch, ALUSrc: out STD_LOGIC;
          RegWrite, Jump: out STD_LOGIC;
          ImmSrc:     out STD_LOGIC_VECTOR(1 downto 0);
          ALUOp:      out STD_LOGIC_VECTOR(1 downto 0));
end;

architecture behave of maindec is
    signal controls: STD_LOGIC_VECTOR(10 downto 0);
begin
    process(op) begin
        case op is
            when "0000011" => controls <= "10010010000"; -- lw
            when "0100011" => controls <= "00111000000"; -- sw
            when "0110011" => controls <= "1--00000100"; -- R-type
            when "1100011" => controls <= "01000001010"; -- beq
            when "0010011" => controls <= "10010000100"; -- I-type ALU
            when "1101111" => controls <= "11100100001"; -- jal
            when others     => controls <= "-----"; -- not valid
        end case;
    end process;

    (RegWrite, ImmSrc(1), ImmSrc(0), ALUSrc, MemWrite,
     ResultSrc(1), ResultSrc(0), Branch, ALUOp(1), ALUOp(0),
     Jump) <= controls;
end;

```

## HDL示例7.4 算术逻辑单元解码器

**SystemVerilog**

```

module aludec(input  logic      opb5,
               input  logic [2:0] funct3,
               input  logic      funct7b5,
               input  logic [1:0] ALUOp,
               output logic [2:0] ALUControl);

    logic RtypeSub;
    assign RtypeSub = funct7b5 & opb5; // TRUE for R-type subtract

    always_comb
    case(ALUOp)
        2'b00: ALUControl = 3'b000; // addition
        2'b01: ALUControl = 3'b001; // subtraction
        default: case(funct3) // R-type or I-type ALU
            3'b000: if (RtypeSub)
                ALUControl = 3'b001; // sub
            else

```

**VHDL**

```

library IEEE;
use IEEE.STD_LOGIC_1164.all;

entity aludec is
    port(opb5:          in  STD_LOGIC;
          funct3:       in  STD_LOGIC_VECTOR(2 downto 0);
          funct7b5:     in  STD_LOGIC;
          ALUOp:        in  STD_LOGIC_VECTOR(1 downto 0);
          ALUControl:   out STD_LOGIC_VECTOR(2 downto 0));
end;

architecture behave of aludec is
    signal RtypeSub: STD_LOGIC;
begin
    RtypeSub <= funct7b5 and opb5; -- TRUE for R-type subtract
    process(opb5, funct3, funct7b5, ALUOp, RtypeSub) begin
        case ALUOp is

```

```

        ALUControl = 3'b000; // add, addi
3'b010:    ALUControl = 3'b101; // slt, slti
3'b110:    ALUControl = 3'b011; // or, ori
3'b111:    ALUControl = 3'b010; // and, andi
default:   ALUControl = 3'bxxx; // ???
    endcase
endmodule

```

```

when "00" =>    ALUControl <= "000"; -- addition
when "01" =>    ALUControl <= "001"; -- subtraction
when others => case funct3 is
    when "000" = if RtypeSub = '1' then
        ALUControl <= "001"; -- sub
    else
        ALUControl <= "000"; -- add, addi
    end if;
    when "010" => ALUControl <= "101"; -- slt, slti
    when "110" => ALUControl <= "011"; -- or, ori
    when "111" => ALUControl <= "010"; -- and, andi
    when others => ALUControl <= "---"; -- unknown
end case;
end case;
end process;
end;

```

## HDL示例7.5 数据通路

### SystemVerilog

```

module datapath(input logic clk, reset,
               input logic [1:0] ResultSrc,
               input logic PCSrc, ALUSrc,
               input logic RegWrite,
               input logic [1:0] ImmSrc,
               input logic [2:0] ALUControl,
               output logic Zero,
               output logic [31:0] PC,
               input logic [31:0] Instr,
               output logic [31:0] ALUResult, WriteData,
               input logic [31:0] ReadData);

    logic [31:0] PCNext, PCPlus4, PCTarget;
    logic [31:0] ImmExt;
    logic [31:0] SrcA, SrcB;
    logic [31:0] Result;

    // next PC logic
    flopr #(32) pcreg(clk, reset, PCNext, PC);
    adder pcadd4(PC, 32'd4, PCPlus4);
    adder pcaddbranch(PC, ImmExt, PCTarget);
    mux2 #(32) pcmux(PCPlus4, PCTarget, PCSrc, PCNext);

    // register file logic
    regfile rf(clk, RegWrite, Instr[19:15], Instr[24:20],
              Instr[11:7], Result, SrcA, WriteData);
    extend ext(Instr[31:7], ImmSrc, ImmExt);

    // ALU logic
    mux2 #(32) srcbmux(WriteData, ImmExt, ALUSrc, SrcB);
    alu alu(SrcA, SrcB, ALUControl, ALUResult, Zero);
    mux3 #(32) resultmux(ALUResult, ReadData, PCPlus4,
                       ResultSrc, Result);
endmodule

```

### VHDL

```

library IEEE;
use IEEE.STD_LOGIC_1164.all;
use IEEE.STD_LOGIC_ARITH.all;

entity datapath is
    port(clk, reset: in STD_LOGIC;
          ResultSrc: in STD_LOGIC_VECTOR(1 downto 0);
          PCSrc, ALUSrc: in STD_LOGIC;
          RegWrite: in STD_LOGIC;
          ImmSrc: in STD_LOGIC_VECTOR(1 downto 0);
          ALUControl: in STD_LOGIC_VECTOR(2 downto 0);
          Zero: out STD_LOGIC;
          PC: buffer STD_LOGIC_VECTOR(31 downto 0);
          Instr: in STD_LOGIC_VECTOR(31 downto 0);
          ALUResult, WriteData: buffer STD_LOGIC_VECTOR(31 downto 0);
          ReadData: in STD_LOGIC_VECTOR(31 downto 0));
end entity;

architecture struct of datapath is
    component flopr generic(width: integer);
        port(clk, reset: in STD_LOGIC;
              d: in STD_LOGIC_VECTOR(width-1 downto 0);
              q: out STD_LOGIC_VECTOR(width-1 downto 0));
    end component;
    component adder
        port(a, b: in STD_LOGIC_VECTOR(31 downto 0);
              y: out STD_LOGIC_VECTOR(31 downto 0));
    end component;
    component mux2 generic(width: integer);
        port(d0, d1: in STD_LOGIC_VECTOR(width-1 downto 0);
              s: in STD_LOGIC;
              y: out STD_LOGIC_VECTOR(width-1 downto 0));
    end component;
    component mux3 generic(width: integer);
        port(d0, d1, d2: in STD_LOGIC_VECTOR(width-1 downto 0);
              s: in STD_LOGIC_VECTOR(1 downto 0);
              y: out STD_LOGIC_VECTOR(width-1 downto 0));
    end component;
    component regfile
        port(clk: in STD_LOGIC;
              we3: in STD_LOGIC;
              a1, a2, a3: in STD_LOGIC_VECTOR(4 downto 0);

```

|  |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|--|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|  | <pre>        wd3:      in  STD_LOGIC_VECTOR(31 downto 0);         rd1, rd2: out STD_LOGIC_VECTOR(31 downto 0)); end component; component extend     port(instr: in  STD_LOGIC_VECTOR(31 downto 7);           immsrc: in  STD_LOGIC_VECTOR(1  downto 0);           immext: out STD_LOGIC_VECTOR(31 downto 0)); end component; component alu     port(a, b:      in      STD_LOGIC_VECTOR(31 downto 0);           ALUControl: in      STD_LOGIC_VECTOR(2  downto 0);           ALUResult: buffer STD_LOGIC_VECTOR(31 downto 0);           Zero:      out     STD_LOGIC); end component;  signal PCNext, PCPlus4, PCTarget: STD_LOGIC_VECTOR(31 downto 0); signal ImmExt:                    STD_LOGIC_VECTOR(31 downto 0); signal SrcA, SrcB:                STD_LOGIC_VECTOR(31 downto 0); signal Result:                    STD_LOGIC_VECTOR(31 downto 0); begin     -- next PC logic     pcreg: flopr generic map(32) port map(clk, reset, PCNext, PC);     pcadd4: adder port map(PC, X"00000004", PCPlus4);     pcaddbranch: adder port map(PC, ImmExt, PCTarget);     pcmux: mux2 generic map(32) port map(PCPlus4, PCTarget, PCSrc,                                           PCNext);      -- register file logic     rf: regfile port map(clk, RegWrite, Instr(19 downto 15),                         Instr(24 downto 20), Instr(11 downto 7),                         Result, SrcA, WriteData);     ext: extend port map(Instr(31 downto 7), ImmSrc, ImmExt);     -- ALU logic     srcbmux: mux2 generic map(32) port map(WriteData, ImmExt,                                           ALUSrc, SrcB);     mainalu: alu port map(SrcA, SrcB, ALUControl, ALUResult, Zero);     resultmux: mux3 generic map(32) port map(ALUResult, ReadData,                                           PCPlus4, ResultSrc,                                           Result); end;</pre> |
|--|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

7.6.2 通用构建模块

本节包含在任何数字系统中都可能有用的一般构建模块，包括加法器、触发器和2:1多路选择器。寄存器文件出现在HDL示例5.8中。ALU的HDL留待练习5.11至5.14。

HDL示例7.6 加法器

|                                                                                                                                    |                                                                                                                                                                                                                                                                                               |
|------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>SystemVerilog</b> <pre>module adder(input  [31:0] a, b,                output [31:0] y);      assign y = a + b; endmodule</pre> | <b>VHDL</b> <pre>library IEEE; use IEEE.STD_LOGIC_1164.all; use IEEE.NUMERIC_STD.UNSIGNED.all;  entity adder is     port(a, b: in  STD_LOGIC_VECTOR(31 downto 0);           y:  out STD_LOGIC_VECTOR(31 downto 0)); end;  architecture behave of adder is begin     y &lt;= a + b; end;</pre> |
|------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

HDL示例7.7 扩展单元

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>SystemVerilog</b></p> <pre>module extend(input  logic [31:7] instr,               input  logic [1:0] immsrc,               output logic [31:0] immext);  always_comb   case(immsrc)     // I-type     2'b00: immext = {{20{instr[31]}}, instr[31:20]};     // S-type (stores)     2'b01: immext = {{20{instr[31]}}, instr[31:25],                     instr[11:7]};     // B-type (branches)     2'b10: immext = {{20{instr[31]}}, instr[7],                     instr[30:25], instr[11:8], 1'b0};     // J-type (jal)     2'b11: immext = {{12{instr[31]}}, instr[19:12],                     instr[20], instr[30:21], 1'b0};     default: immext = 32'bx; // undefined   endcase endmodule</pre> | <p><b>VHDL</b></p> <pre>library IEEE; use IEEE.STD_LOGIC_1164.all;  entity extend is   port(instr: in  STD_LOGIC_VECTOR(31 downto 7);         immsrc: in  STD_LOGIC_VECTOR(1 downto 0);         immext: out STD_LOGIC_VECTOR(31 downto 0)); end;  architecture behave of extend is begin   process(instr, immsrc) begin     case immsrc is       -- I-type       when "00" =&gt;         immext &lt;= (31 downto 12 =&gt; instr(31)) &amp; instr(31 downto 20);       -- S-types (stores)       when "01" =&gt;         immext &lt;= (31 downto 12 =&gt; instr(31)) &amp;                   instr(31 downto 25) &amp; instr(11 downto 7);       -- B-type (branches)       when "10" =&gt;         immext &lt;= (31 downto 12 =&gt; instr(31)) &amp; instr(7) &amp; instr(30           downto 25) &amp; instr(11 downto 8) &amp; '0';       -- J-type (jal)       when "11" =&gt;         immext &lt;= (31 downto 20 =&gt; instr(31)) &amp;                   instr(19 downto 12) &amp; instr(20) &amp;                   instr(30 downto 21) &amp; '0';       when others =&gt;         immext &lt;= (31 downto 0 =&gt; '-');     end case;   end process; end;</pre> |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

HDL示例7.8 可复位触发器

|                                                                                                                                                                                                                                                                              |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                               |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>SystemVerilog</b></p> <pre>module flopr #(parameter WIDTH = 8)   (input  logic      clk, reset,    input  logic [WIDTH-1:0] d,    output logic [WIDTH-1:0] q);  always_ff @(posedge clk, posedge reset)   if (reset) q &lt;= 0;   else      q &lt;= d; endmodule</pre> | <p><b>VHDL</b></p> <pre>library IEEE; use IEEE.STD_LOGIC_1164.all; use IEEE.STD_LOGIC_ARITH.all;  entity flopr is   generic(width: integer);   port(clk, reset: in  STD_LOGIC;         d:         in  STD_LOGIC_VECTOR(width-1 downto 0);         q:         out STD_LOGIC_VECTOR(width-1 downto 0)); end;  architecture asynchronous of flopr is begin   process(clk, reset) begin     if reset = '1' then      q &lt;= (others =&gt; '0');     elsif rising_edge(clk) then q &lt;= d;     end if;   end process; end;</pre> |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

## HDL示例7.9 带使能的可复位触发器

**SystemVerilog**

```

module flopenr #(parameter WIDTH = 8)
    (input logic clk, reset, en,
     input logic [WIDTH-1:0] d,
     output logic [WIDTH-1:0] q);

always_ff @(posedge clk, posedge reset)
    if (reset) q <= 0;
    else if (en) q <= d;
endmodule

```

**VHDL**

```

library IEEE;
use IEEE.STD_LOGIC_1164.all;
use IEEE.STD_LOGIC_ARITH.all;

entity flopenr is
    generic(width: integer);
    port(clk, reset, en: in STD_LOGIC;
         d: in STD_LOGIC_VECTOR(width-1 downto 0);
         q: out STD_LOGIC_VECTOR(width-1 downto 0));
end;

architecture asynchronous of flopenr is
begin
    process(clk, reset, en) begin
        if reset = '1' then q <= (others => '0');
        elsif rising_edge(clk) and en = '1' then q <= d;
        end if;
    end process;
end;

```

## HDL示例7.10 2:1多路选择器

**SystemVerilog**

```

module mux2 #(parameter WIDTH = 8)
    (input logic [WIDTH-1:0] d0, d1,
     input logic s,
     output logic [WIDTH-1:0] y);

    assign y = s ? d1 : d0;
endmodule

```

**VHDL**

```

library IEEE;
use IEEE.STD_LOGIC_1164.all;

entity mux2 is
    generic(width: integer := 8);
    port(d0, d1: in STD_LOGIC_VECTOR(width-1 downto 0);
         s: in STD_LOGIC;
         y: out STD_LOGIC_VECTOR(width-1 downto 0));
end;

architecture behave of mux2 is
begin
    y <= d1 when s = '1' else d0;
end;

```

## HDL示例7.11 3:1多路选择器

**SystemVerilog**

```

module mux3 #(parameter WIDTH = 8)
    (input logic [WIDTH-1:0] d0, d1, d2,
     input logic [1:0] s,
     output logic [WIDTH-1:0] y);

    assign y = s[1] ? d2 : (s[0] ? d1 : d0);
endmodule

```

**VHDL**

```

library IEEE;
use IEEE.STD_LOGIC_1164.all;

entity mux3 is
    generic(width: integer := 8);
    port(d0, d1, d2: in STD_LOGIC_VECTOR(width-1 downto 0);
         s: in STD_LOGIC_VECTOR(1 downto 0);
         y: out STD_LOGIC_VECTOR(width-1 downto 0));
end;

architecture behave of mux3 is
begin
    process(d0, d1, d2, s) begin
        if (s = "00") then y <= d0;
        elsif (s = "01") then y <= d1;
        elsif (s = "10") then y <= d2;
        end if;
    end process;
end;

```



```
# riscvtest.s
# Sarah.Harris@unlv.edu
# David_Harris@hmc.edu
# 27 Oct 2020
#
# Test the RISC-V processor:
#   add, sub, and, or, slt, addi, lw, sw, beq, jal
# If successful, it should write the value 25 to address 100
#
#           RISC-V Assembly      Description      Address  Machine Code
main:      addi x2, x0, 5          # x2 = 5              0          00500113
          addi x3, x0, 12         # x3 = 12              4          00C00193
          addi x7, x3, -9         # x7 = (12 - 9) = 3    8          FF718393
          or   x4, x7, x2         # x4 = (3 OR 5) = 7    C          0023E233
          and  x5, x3, x4         # x5 = (12 AND 7) = 4  10         0041F2B3
          add  x5, x5, x4         # x5 = 4 + 7 = 11     14         004282B3
          beq  x5, x7, end        # shouldn't be taken   18         02728863
          slt  x4, x3, x4         # x4 = (12 < 7) = 0    1C         0041A233
          beq  x4, x0, around     # should be taken      20         00020463
          addi x5, x0, 0          # shouldn't execute    24         00000293
around:    slt  x4, x7, x2        # x4 = (3 < 5) = 1     28         0023A233
          add  x7, x4, x5         # x7 = (1 + 11) = 12   2C         005203B3
          sub  x7, x7, x2        # x7 = (12 - 5) = 7    30         402383B3
          sw   x7, 84(x3)         # [96] = 7            34         0471AA23
          lw   x2, 96(x0)         # x2 = [96] = 7        38         06002103
          add  x9, x2, x5         # x9 = (7 + 11) = 18   3C         005104B3
          jal  x3, end            # jump to end, x3 = 0x44 40         008001EF
          addi x2, x0, 1          # shouldn't execute    44         00100113
end:       add  x2, x2, x9        # x2 = (7 + 18) = 25   48         00910133
          sw   x2, 0x20(x3)       # [100] = 25          4C         0221A023
done:      beq  x2, x2, done      # infinite loop        50         00210063
```

图 7.64 riscvtest.s

7.6.3 测试平台

测试平台将程序加载到存储器中。图7.64中的程序通过执行一项计算来测试所有指令，只有当所有指令都正确运行时，该计算才会产生正确结果。具体来说，如果程序正确运行，它将把值25写入地址100；但如果硬件存在缺陷，则不太可能做到这一点。这是*ad hoc*测试的一个示例。

机器码存储在一个名为riscvtest.txt的文本文件中（图 7.65），该文件在仿真期间由测试平台加载。文件包含用十六进制编写的指令机器码，每行一条指令。

以下示例给出了测试平台、顶层RISC-V模块（实例化RISC-V处理器和存储器）以及外部存储器HDL代码。测试平台实例化被测试的顶层模块，并在仿真开始时生成周期性时钟和复位信号。它检查存储器写入操作，如果正确值（25）被写入地址100，则报告成功。此示例中的每个存储器各包含64个32位字。

```
00500113 0
0C00193 F
F718393 00
23E233 004
1F2B3 004
282B3 027
28863 0041
A233 0002
0463 00000
293 0023A
233 005203
B3 402383
B3 0471AA
23 0600210
3 005104B
3 008001E
F 00100113
00910133 0
221A023 0
0210063
```

图7.65 riscvtest.txt

## HDL示例7.12 测试平台

**SystemVerilog**

```

module testbench();

    logic        clk;
    logic        reset;
    logic [31:0] WriteData, DataAdr;
    logic        MemWrite;

    // instantiate device to be tested
    top dut(clk, reset, WriteData, DataAdr, MemWrite);

    // initialize test
    initial
    begin
        reset <= 1; # 22; reset <= 0;
    end

    // generate clock to sequence tests
    always
    begin
        clk <= 1; # 5; clk <= 0; # 5;
    end

    // check results
    always @(negedge clk)
    begin
        if(MemWrite) begin
            if(DataAdr == 100 & WriteData == 25) begin
                $display("Simulation succeeded");
                $stop;
            end else if (DataAdr != 96) begin
                $display("Simulation failed");
                $stop;
            end
        end
    end
endmodule

```

**VHDL**

```

library IEEE;
use IEEE.STD_LOGIC_1164.all;
use IEEE.NUMERIC_STD_UNSIGNED.all;

entity testbench is
end;

architecture test of testbench is
    component top
        port(clk, reset:          in  STD_LOGIC;
              WriteData, DataAdr: out STD_LOGIC_VECTOR(31 downto 0);
              MemWrite:           out STD_LOGIC);
    end component;

    signal WriteData, DataAdr:  STD_LOGIC_VECTOR(31 downto 0);
    signal clk, reset, MemWrite: STD_LOGIC;
begin
    -- instantiate device to be tested
    dut: top port map(clk, reset, WriteData, DataAdr, MemWrite);

    -- Generate clock with 10 ns period
    process begin
        clk <= '1';
        wait for 5 ns;
        clk <= '0';
        wait for 5 ns;
    end process;

    -- Generate reset for first two clock cycles
    process begin
        reset <= '1';
        wait for 22 ns;
        reset <= '0';
        wait;
    end process;

    -- check that 25 gets written to address 100 at end of program
    process(clk) begin
        if(clk'event and clk = '0' and MemWrite = '1') then
            if(to_integer(DataAdr) = 100 and
               to_integer(writedata) = 25) then
                report "NO ERRORS: Simulation succeeded" severity
                    failure;
            elsif (DataAdr /= 96) then
                report "Simulation failed" severity failure;
            end if;
        end if;
    end process;
end;

```

HDL示例7.13 顶层模块

|                                                                                                                                                                                                                                                                                                                                                                                                                                                                                             |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <div><b>SystemVerilog</b></div> <div><pre>module top(input logic      clk, reset,            output logic [31:0] WriteData, DataAdr,            output logic      MemWrite);      logic [31:0] PC, Instr, ReadData;      // instantiate processor and memories     riscvsingle rvsingle(clk, reset, PC, Instr, MemWrite,                         DataAdr, WriteData, ReadData);     imem imem(PC, Instr);     dmem dmem(clk, MemWrite, DataAdr, WriteData, ReadData); endmodule</pre></div> | <div><b>VHDL</b></div> <div><pre>library IEEE; use IEEE.STD_LOGIC_1164.all; use IEEE.NUMERIC_STD_UNSIGNED.all;  entity top is     port(clk, reset:      in      STD_LOGIC;           WriteData, DataAdr: buffer STD_LOGIC_VECTOR(31 downto 0);           MemWrite:      buffer STD_LOGIC); end;  architecture test of top is     component riscvsingle         port(clk, reset:      in      STD_LOGIC;               PC:             out STD_LOGIC_VECTOR(31 downto 0);               Instr:          in      STD_LOGIC_VECTOR(31 downto 0);               MemWrite:       out STD_LOGIC;               ALUResult, WriteData: out STD_LOGIC_VECTOR(31 downto 0);               ReadData:       in      STD_LOGIC_VECTOR(31 downto 0));     end component;     component imem         port(a: in STD_LOGIC_VECTOR(31 downto 0);               rd: out STD_LOGIC_VECTOR(31 downto 0));     end component;     component dmem         port(clk, we: in STD_LOGIC;               a, wd: in STD_LOGIC_VECTOR(31 downto 0);               rd: out STD_LOGIC_VECTOR(31 downto 0));     end component;      signal PC, Instr, ReadData: STD_LOGIC_VECTOR(31 downto 0); begin     -- instantiate processor and memories     rvsingle: riscvsingle port map(clk, reset, PC, Instr,                                    MemWrite, DataAdr,                                    WriteData, ReadData);     imem1: imem port map(PC, Instr);     dmem1: dmem port map(clk, MemWrite, DataAdr, WriteData,                          ReadData); end;</pre></div> |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

HDL示例7.14 指令存储器

|                                                                                                                                                                                                                                                                           |                                                                                                                                                                                                                                                                                                                                                                                                                     |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <div><b>SystemVerilog</b></div> <div><pre>module imem(input logic [31:0] a,             output logic [31:0] rd);      logic [31:0] RAM[63:0];      initial         \$readmemh("riscvtest.txt", RAM);      assign rd = RAM[a[31:2]]; // word aligned endmodule</pre></div> | <div><b>VHDL</b></div> <div><pre>library IEEE; use IEEE.STD_LOGIC_1164.all; use STD.TEXTIO.all; use IEEE.NUMERIC_STD_UNSIGNED.all; use ieee.std_logic_textio.all;  entity imem is     port(a: in STD_LOGIC_VECTOR(31 downto 0);           rd: out STD_LOGIC_VECTOR(31 downto 0)); end;  architecture behave of imem is     type ramtype is array(63 downto 0) of         STD_LOGIC_VECTOR(31 downto 0);</pre></div> |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

```

-- initialize memory from file
impure function init_ram_hex return ramtype is
file text_file : text open read_mode is "riscvtest.txt";
variable text_line : line;
variable ram_content : ramtype;
variable i : integer := 0;
begin
for i in 0 to 63 loop -- set all contents low
ram_content(i) := (others => '0');
end loop;
while not endfile(text_file) loop -- set contents from file
readline(text_file, text_line);
hread(text_line, ram_content(i));
i := i + 1;
end loop;

return ram_content;
end function;

signal mem : ramtype := init_ram_hex;
begin
-- read memory
process(a) begin
rd <= mem(to_integer(a(31 downto 2)));
end process;
end;

```

### HDL示例7.15 数据存储器

#### SystemVerilog

```

module dmem(input logic clk, we,
input logic [31:0] a, wd,
output logic [31:0] rd);

logic [31:0] RAM[63:0];

assign rd = RAM[a[31:2]]; // word aligned

always_ff @(posedge clk)
if (we) RAM[a[31:2]] <= wd;
endmodule

```

#### VHDL

```

library IEEE;
use IEEE.STD_LOGIC_1164.all;
use STD.TEXTIO.all;
use IEEE.NUMERIC_STD_UNSIGNED.all;

entity dmem is
port(clk, we: in STD_LOGIC;
a, wd: in STD_LOGIC_VECTOR(31 downto 0);
rd: out STD_LOGIC_VECTOR(31 downto 0));
end;

architecture behave of dmem is
begin
process is
type ramtype is array (63 downto 0) of
STD_LOGIC_VECTOR(31 downto 0);
variable mem: ramtype;
begin
-- read or write memory
loop
if rising_edge(clk) then
if (we = '1') then mem(to_integer(a(7 downto 2))) := wd;
end if;
end if;
rd <= mem(to_integer(a(7 downto 2)));
wait on clk, a;
end loop;
end process;
end;

```

## 7.7 高级微架构\*

高性能微处理器采用多种技术来加速程序运行。回顾一下，运行程序所需的时间与时钟周期以及每条指令的时钟周期数（CPI）成正比。因此，为了提高性能，我们希望加快时钟速度和/或降低CPI。本节概述了一些现有的加速技术。实现细节变得相当复杂，因此我们侧重于概念。如果你想全面了解细节，Hennessy和Patterson的*Computer Architecture*教材是权威参考。

集成电路制造的进步持续缩小了晶体管尺寸。更小的晶体管速度更快，且通常功耗更低。因此，即使微架构不变，时钟频率也能提升，因为所有逻辑门都更快了。此外，更小的晶体管使得芯片上能集成更多晶体管。微架构师利用这些额外晶体管来构建更复杂的处理器，或在单芯片上集成更多处理器。遗憾的是，功耗会随晶体管数量及其运行速度的增加而上升（参见第1.8节）。功耗已成为关键问题。微处理器设计者面临艰巨挑战，需在速度、功耗与成本之间权衡，以制造出人类历史上最复杂系统中那些包含数十亿晶体管的芯片。

### 7.7.1 深度管道

在20世纪90年代末至21世纪初，微处理器的市场推广主要基于时钟频率（ $f = 1/T_c$ ）。这促使微处理器采用极深的流水线（奔腾4达到20-31级）以最大化时钟频率，即便对整体性能的提升存疑。功耗与时钟频率成正比，且随流水线寄存器数量增加而上升，因此如今功耗至关重要，流水线深度已缩短。

除了制造工艺的进步，加速时钟最简单的方法是将流水线划分为更多阶段。每个阶段包含的逻辑更少，因此可以运行得更快。本章讨论的是经典的五级流水线，但目前通常使用8到20级流水线。例如，由西部数据开发的开源商业RISC-V处理器SweRV EH1核心就采用了九级流水线。

流水线级数的最大值受限于流水线冒险、时序开销和成本。更长的流水线会引入更多依赖关系。部分依赖可通过转发解决，但其他依赖则需要停顿，从而增加CPI。各级之间的流水线寄存器因其建立时间和时钟到Q延迟（以及时钟偏斜）产生时序开销。由于这种时序开销，增加流水线级数会带来收益递减。最后，增加级数会因额外的流水线寄存器和处理冒险所需的硬件而提高成本。

示例 7.11 深度流水线

考虑通过将单周期处理器拆分为 $N$ 个阶段来构建流水线处理器。单周期处理器通过组合逻辑的传播延迟为750 ps。寄存器的时序开销为90 ps。假设组合延迟可以任意划分为任意数量的阶段，并且流水线冒险逻辑不会增加延迟。示例7.9中的五级流水线CPI为1.25。假设每增加一个阶段，由于分支预测错误和其他流水线冒险，CPI增加0.1。为了使处理器尽可能快地执行程序，应使用多少个流水线阶段？

解： $N$ 级流水线的周期时间为 $T_c = [(750/N) + 90]$  ps。CPI为 $1.25 + 0.1(N - 5)$ ，其中 $N \geq 5$ 。每条指令的时间（即指令时间）是周期时间 $T_c$ 与CPI的乘积。图7.66绘制了周期时间和指令时间随级数的变化曲线。指令时间在 $N = 8$ 级时达到最小值281 ps。该最小值仅略优于五级流水线实现的每条指令295 ps，且曲线在7到10级之间几乎平坦。

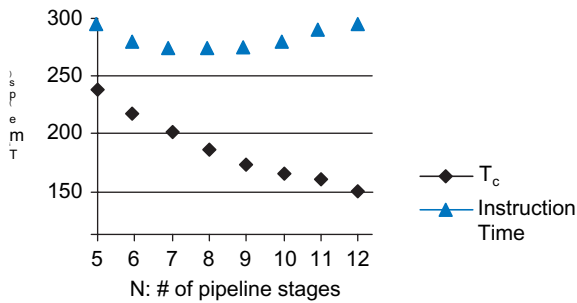


图7.66 周期时间和指令时间与流水线级数的关系

7.7.2 微操作

回顾我们的设计原则：“规整性支持简洁性”和“让常见情况更快”。纯精简指令集计算机（RISC）架构（如RISC-V）仅包含简单指令，通常这些指令可以在一个简单、快速的数据通路上单周期执行，该通路配备三端口寄存器堆、单个ALU以及单个数据存储器访问（类似于本章中我们设计的通路）。复杂指令集计算机（CISC）架构通常包含需要更多寄存器、更多加法操作或每条指令需要多次存储器访问的指令。例如，x86指令 `ADD [ESP],[EDX+80+EDI*2]` 涉及读取三个寄存器（ESP、EDX和EDI），将基址（EDX）、偏移量（80）和比例因子

索引(EDI\*2)，读取两个内存位置，将其值相加，并将结果写回内存。一个能够同时执行所有这些功能的微处理器，在执行更常见、更简单的指令时会不必要地变慢。

微架构师需要决定是直接提供硬件来实现复杂操作，还是将其分解为微操作序列。他们还会对本节后面描述的其他选项做出类似决策。这些选择会导致性能-功耗-成本设计空间中的不同权衡点。

CISC处理器的计算机架构师通过定义一组简单的*micro-operations*（也称为*micro-ops*或 $\mu$ 操作）来加速常见情况，这些操作可以在简单数据路径上执行。每条CISC指令被解码为一个或多个微操作。例如，如果我们定义类似于RISC-V指令的 $\mu$ 操作，并使用临时寄存器t1和t2来保存中间结果，那么上述x86指令可以变成六个 $\mu$ 操作：

```
slli t2, EDI, 1 # t2 {v*} EDI*2
add t1, EDX, t2 # t1 {v*} EDX {v*} EDI*2
lw t1, 80(t1) # t1 {v*} MEM[EDX {v*} EDI*2 {v*} 80]
lw t2, 0(ESP) # t2 {v*} MEM[ESP]
add t1, t2, t1 # t1 {v*} MEM[ESP] {v*} MEM[EDX {v*} EDI*2 {v*} 80]
sw t1, 0(ESP) # MEM[ESP] {v*} MEM[ESP] {v*} MEM[EDX {v*} EDI*2 {v*} 80]
```

### 7.7.3 分支预测

理想的流水线处理器CPI应为1。分支误预测惩罚是导致CPI增加的主要原因。随着流水线深度增加，分支在流水线中的解析位置会延后。因此，分支误预测惩罚会变大，因为误预测分支之后发出的所有指令都必须被清除。为解决此问题，大多数流水线处理器使用*branch predictor*来猜测分支是否应该被选取。回顾第7.5.3节中的流水线，它只是简单地预测分支永远不会被选取。

某些分支出现在循环开始时，用于检查条件并在条件不再满足时跳转到循环之外（例如在*for*和*while*循环中）。循环往往执行多次，因此这些前向分支通常不会被执行。另一些分支出现在程序到达循环末尾时，跳转回循环起点以重复执行（例如在*do/while*循环中）。同样，由于循环往往执行多次，这些后向分支通常会被执行。最简单的分支预测方法会检查分支方向，并预测后向分支会被执行而前向分支不会。这被称为*static branch prediction*，因为它不依赖于程序的历史记录。

然而，分支（尤其是前向分支）在缺乏对具体程序的更多了解时难以预测。因此，大多数处理器采用*dynamic branch predictors*，通过程序执行历史来推测分支是否应该被选取。动态分支预测器维护一个包含最近数百条

(或处理器已执行的千)条分支指令。该表称为*branch target buffer*，包含分支的目标地址以及分支是否被采用的历史记录。

为了观察动态分支预测器的工作方式，请考虑代码示例6.20中的以下循环。该循环重复10次，且循环出口分支（`bge s0, t0, done`）仅在最后一次迭代时被执行。

```

addi s1, zero, 0 # s1 = sum = 0
addi s0, zero, 0 # s0 = i = 0
addi t0, zero, 10 # t0 = 10
for: bge s0, t0, done # i >= 10? add s1, s1, s0 # sum
      = sum + i addi s0, s0, 1 # i = i + 1 j for # 重
复循环 done:

```

一个*one-bit dynamic branch predictor*会记住分支上次是否被采用，并预测下次会执行相同操作。当循环重复时，它记住`beq`上次未被采用，并预测下次也不应被采用。这一预测在循环的最后一个分支之前都是正确的，而最后一个分支确实被采用了。不幸的是，如果循环再次运行，分支预测器会记住最后一个分支被采用。因此，当循环再次首次运行时，它会错误地预测该分支应被采用。总之，1位分支预测器会错误预测循环的第一个和最后一个分支。

一种*two-bit dynamic branch predictor*通过四种状态解决了这个问题：强预测为“已跳转”、弱预测为“已跳转”、弱预测为“未跳转”和强预测为“未跳转”，如图7.67所示。当循环重复执行时，它会进入强预测为“未跳转”状态，并预测下一次分支不应跳转。这一预测在循环的最后一个分支之前都是正确的，而最后一个分支会实际跳转，使预测器进入弱预测为“未跳转”状态。当循环再次首次运行时，分支预测器正确预测分支不应跳转，并重新进入强预测为“未跳转”状态。总之，一个两位分支预测器仅在循环的最后一个分支上预测错误。它被称为两位分支预测器，是因为需要两个比特来编码这四种状态。

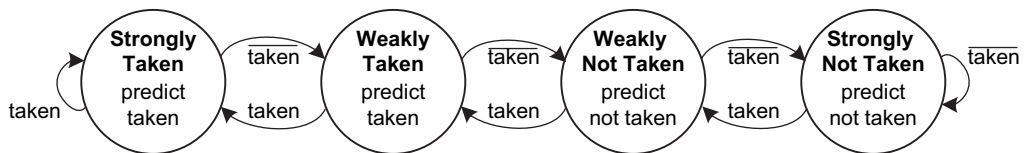


图7.67 两位分支预测器状态转换图



西部数据的RISC-V SweRV EH1是一款双路超标量核心。

*scalar*处理器一次处理一条数据。*vector*处理器通过单条指令处理多条数据。*superscalar*处理器一次发出多条指令，每条指令操作一条数据。

标量处理器被归类为单指令单数据（SISD）机器。向量处理器和图形处理器（GPU：图形处理单元）属于单指令多数据（SIMD）

机器。多处理器（如多核处理器）被归类为多指令多数据（MIMD）机器。通常，MIMD机器运行单个程序，该程序使用全部或部分核心。这种编程风格称为单程序多数据（SPMD），但多处理器也可以采用其他方式进行编程。

分支预测器在流水线的取指阶段运行，以便确定下一个周期要执行的指令。当它预测分支应该被采取时，处理器从存储在分支目标缓冲区中的分支目标地址获取下一条指令。

可以想象，分支预测器可用于跟踪程序的更多历史信息，以提高预测的准确性。优秀的分支预测器在典型程序上能达到超过90%的准确率。

### 7.7.4 超标量处理器

一个*superscalar processor*包含数据路径硬件的多个副本，以同时执行多条指令。图7.68展示了一个双路超标量处理器的框图，该处理器每个周期获取并执行两条指令。数据路径每次从指令存储器中获取两条指令。它拥有一个六端口的寄存器文件，每个周期可读取四个源操作数并写回两个结果。此外，它还包含两个ALU和一个双端口数据存储器，以便同时执行这两条指令。

图7.69是一个流水线图，展示了双向超标量处理器在每个周期执行两条指令的情况。对于该程序，处理器的CPI为0.5。设计人员通常将CPI的倒数称为*instructions per cycle*，即IPC。该处理器在此程序上的IPC为2。

同时执行多条指令因依赖关系而变得困难。例如，图7.70展示了一个运行存在数据依赖程序的流水线图。代码中的依赖关系以蓝色标出。*add*指令依赖于*s8*，而*s8*是

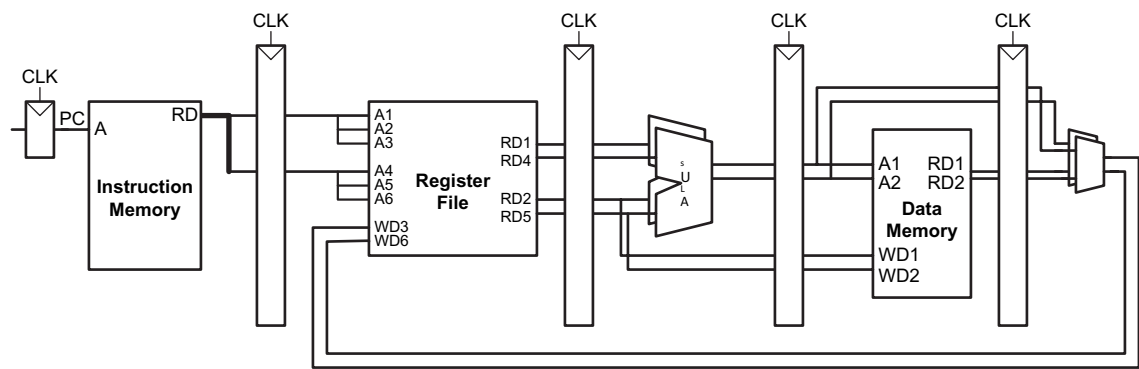


图7.68 超标量数据通路

由 `lw` 指令产生，因此不能与 `lw` 同时发出。`add` 指令再停顿一个周期，以便 `lw` 能在第 5 周期将 `s8` 转发给 `add`。其他依赖关系（`sub` 与 `and` 基于 `s8` 的依赖，以及 `or` 与 `sw` 基于 `s11` 的依赖）通过将某一周期产生的结果转发至下一周期使用来处理。该程序需要五个周期发出六条指令，IPC 为  $6/5 = 1.2$ 。

回想一下，并行性分为时间并行和空间并行两种形式。流水线是时间并行的一个例子，而使用多个执行单元则是空间并行的一个例子。超标量处理器利用这两种并行形式，以榨取出远超我们单周期和多周期处理器的性能。

商用处理器可能是三路、四路甚至六路超标量。它们必须处理分支等控制冒险以及数据冒险。不幸的是，实际程序存在大量依赖关系，因此宽超标量处理器很少能完全利用所有执行单元。此外，大量的执行单元和复杂的转发网络消耗了大量的电路和功耗。

我们的RISC-V流水线处理器是一种标量处理器。向量处理器在20世纪80年代和90年代在超级计算机中很流行，因为它们能高效处理科学计算中常见的长向量数据，并且现在在GPU中得到了广泛应用。处理向量（或广义上的SIMD）是*data-level parallelism*的一个例子，其中多个数据可以并行操作。现代高性能微处理器是超标量的，因为同时发出多条独立指令比处理向量更灵活。然而，现代处理器也包含称为SIMD单元的硬件，用于处理多媒体和图形应用中常见的短向量数据。RISC-V包含向量（V）扩展以支持向量操作。

### 7.7.5 乱序处理器

为了应对依赖问题，乱序执行处理器会前瞻多条指令，以尽可能快地发射独立指令。只要依赖关系得到遵守，指令可以按照与程序员编写的顺序不同的顺序发射，从而确保程序产生预期的结果。

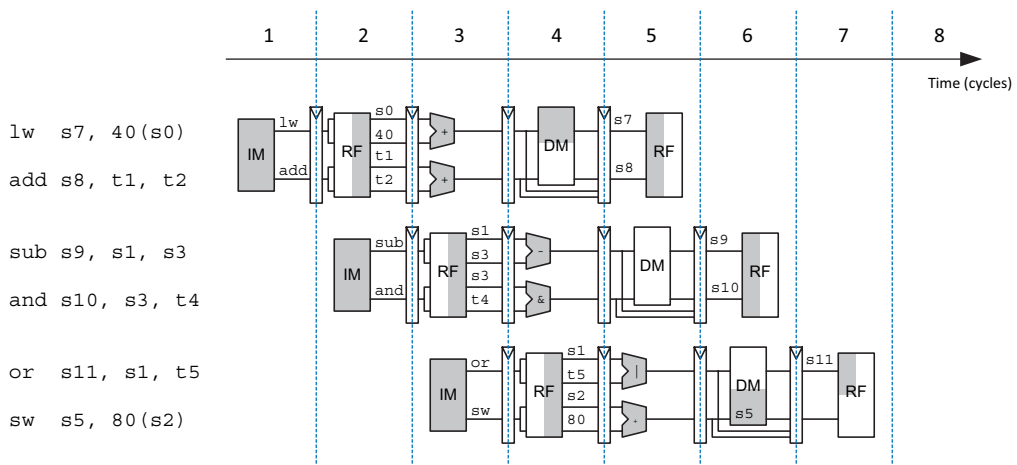


图7.69 超标量流水线运行中的抽象视图

考虑在图7.70所示的同一程序上运行一个双向超标量乱序处理器。该处理器每个周期可以从程序中的任意位置最多发出两条指令，只要满足依赖关系即可。图7.71展示了数据依赖关系以及

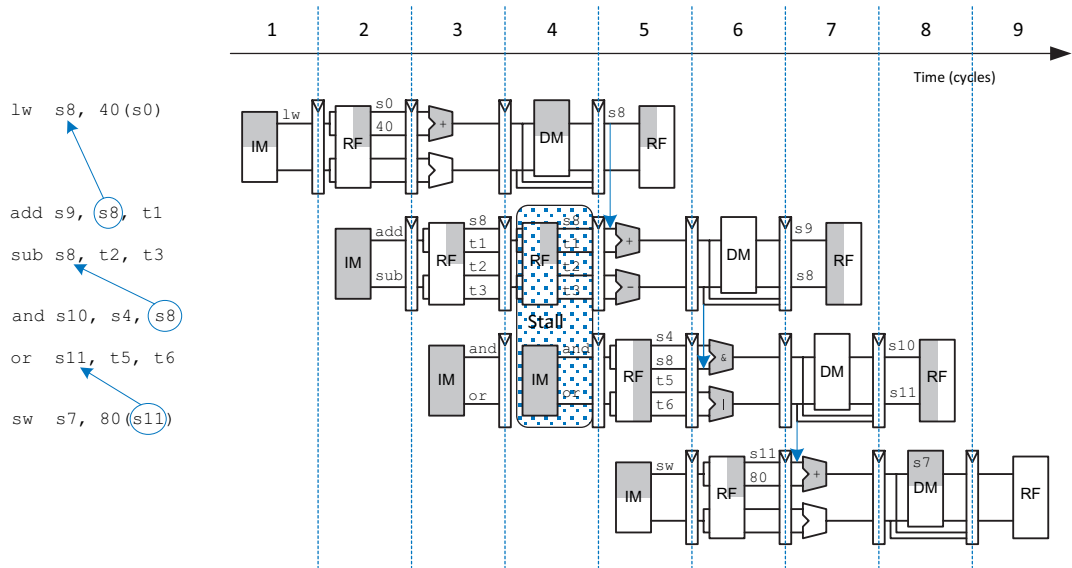


图7.70 具有数据依赖性的程序

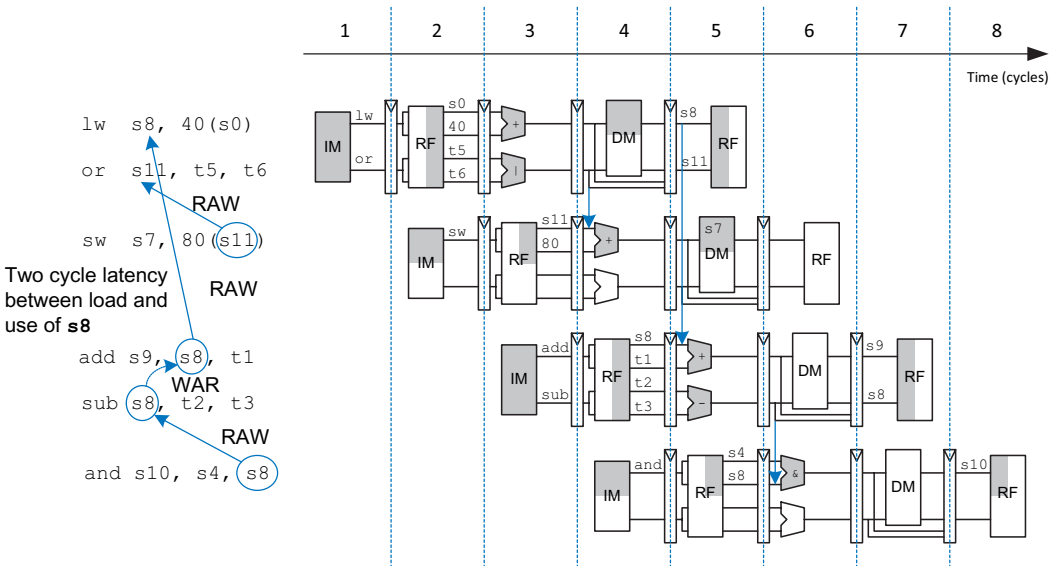


图7.71 具有依赖关系的程序的乱序执行

处理器的操作。依赖关系分类为RAW和WAR将在稍后讨论。指令发射的约束条件为：

#### 周期1

lw指令发出。► add、sub和and指令通过s8依赖于lw，因此它们还不能发出。然而，or指令是独立的，所以它也发出。

#### 周期2

请记住，在发出lw指令与依赖指令之间存在两个周期的延迟，因此由于对s8的依赖，add指令尚无法发出。sub指令会写入s8，因此它不能在add之前发出，以免add接收到错误的s8值。and指令依赖于sub指令。► 只有sw指令发出。

#### 周期3

在第3个周期，s8可用（或者更准确地说，当加法需要它时它就会可用），因此加法发出指令。减法同时发出指令，因为它要等到加法消耗（即读取）s8之后才会写入s8。

#### 周期4

and指令问题。s8从sub转发到and。

乱序处理器在四个周期内发出六条指令，IPC为 $6/4 = 1.5$ 。add对lw通过s8的依赖是一种*read after write*（RAW）冒险。add必须等到lw写入s8之后才能读取该寄存器。这是我们在流水线处理器中习惯处理的一类依赖关系。即使有无限多的执行单元可用，它本质上也会限制程序的运行速度。类似地，sw对or通过s11的依赖，以及and对sub通过s8的依赖，都属于RAW依赖。

通过s8方式，sub与add之间的依赖关系被称为*write after read*（WAR）冒险或*antidependence*。sub不能在add读取s8之前写入s8，以便add根据程序的原始顺序接收到正确的值。WAR冒险在简单流水线中不会发生，但在乱序处理器中，如果依赖指令（此处为sub）被过早移动，则可能出现。

战争冒险并非程序运行的必要条件，它仅仅是程序员选择将两个无关指令使用同一寄存器而产生的人为产物。如果sub指令写入的是s3而非s8，那么依赖关系就会消失，sub指令便可在add指令之前发出。RISC-V架构仅有31个寄存器，因此有时程序员被迫重复使用寄存器，仅仅因为所有其他寄存器都已被占用，从而引入了冒险。

程序中未显示的第三种冒险称为*write after write* (WAW) 冒险或 *output dependence*。当一条指令试图在后续指令已经写入某个寄存器之后再次写入该寄存器时, 就会发生WAW冒险。这种冒险会导致寄存器被写入错误的值。例如, 在以下代码中, *lw*和*add*都写入*s7*。根据程序顺序, *s7*的最终值应来自*add*。如果乱序处理器尝试先执行*add*再执行*lw*, 就会发生WAW冒险。

```
lw s7, 0(t3) add s7, s  
1, t2
```

WAW 冒险也不是本质性的; 同样, 它们是由程序员对两条无关指令使用相同目标寄存器所导致的人为产物。如果先发出 *add* 指令, 那么程序可以通过丢弃 *lw* 的结果而不是将其写入 *s7* 来消除 WAW 冒险。这被称为 *squashing* 丢弃 *lw* 的结果。<sup>4</sup>

乱序处理器使用一张表格来跟踪等待发射的指令。该表格有时被称为*scoreboard*, 其中包含有关依赖关系的信息。表格的大小决定了可以纳入发射考虑的指令数量。在每个周期中, 处理器会检查该表格, 并尽可能多地发射指令, 但受限于依赖关系以及可用的执行单元(例如, ALU、内存端口)数量。

*instruction-level parallelism* (ILP) 是指特定程序与微架构下可同时执行的指令数量。理论研究表明, 对于具备完美分支预测器和海量执行单元的无序微架构而言, ILP可能相当大。然而, 实际处理器即使采用六路超标量数据通路并支持无序执行, 其ILP也很少能超过二或三。

### 7.7.6 寄存器重命名

乱序处理器使用一种称为*register renaming*的技术来消除WAR和WAW冒险。寄存器重命名为处理器添加了一些非架构性的重命名寄存器。例如, 处理器可能添加20个重命名寄存器, 称为*r0*到*r19*。程序员无法

---

<sup>4</sup> You might wonder why the *lw* needs to be issued at all. The reason is that out-of-order processors must guarantee that all of the same exceptions occur that would have occurred if the program had been executed in its original order. The *lw* potentially may produce a load address misaligned exception or load access fault, so it must be issued to check for the exception, even though the result can be discarded.

直接使用这些寄存器，因为它们不属于架构的一部分。然而，处理器可以自由地使用它们来消除冒险。

例如，在上一节中，基于重用s8，sub s8, t2, t3 和 add s9, s8, t1 指令之间发生了WAR冒险。乱序处理器可以将sub指令中的s8重命名为r0。这样，sub可以更早执行，因为r0与add指令没有依赖关系。处理器维护一张寄存器重命名表，以便在后续依赖指令中一致地重命名寄存器。在此例中，and指令中的s8也必须重命名为r0，因为它引用了sub的结果。图7.72展示了与图7.71相同的程序在具有寄存器重命名的乱序处理器上执行的情况。s8在sub和and中被重命名为r0，以消除WAR冒险。指令发射的约束条件为：

#### 周期1

lw指令发出。► add指令通过s8依赖于lw，因此尚不能发出。然而，sub指令现在独立了，因为其目标已被重命名为r0，所以sub也发出。

#### 周期2

请记住，在发出lw指令和依赖指令之间必须存在两个周期的延迟，因此由于s8的依赖关系，add指令尚无法发出。

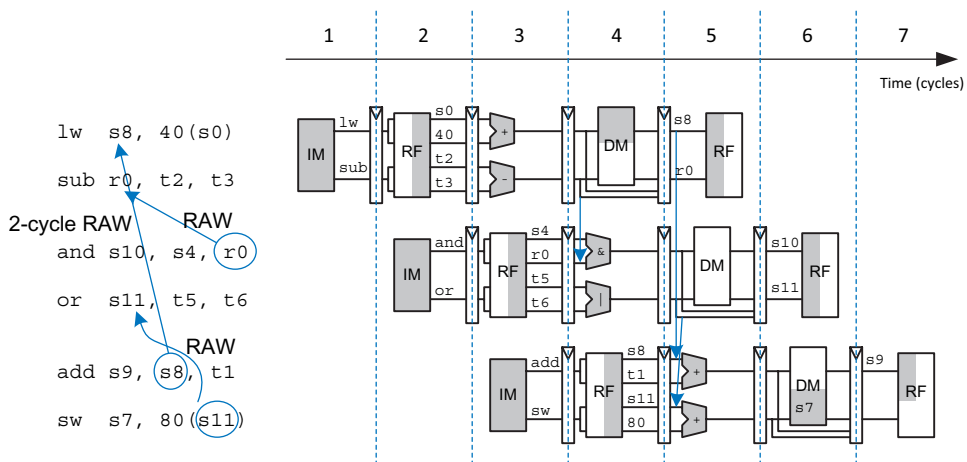


图7.72 使用寄存器重命名的程序乱序执行

and 指令依赖于 sub，因此它可以发出。r0 从 sub 转发到 and。► or 指令是独立的，因此它也可以发出。

周期 3

在第3个周期，s8可用，因此add发出。► s11也可用，因此sw发出。

采用寄存器重命名的乱序处理器在三个周期内发射了六条指令，IPC 为2。

### 7.7.7 多线程

由于实际程序的指令级并行性（ILP）通常较低，为超标量或乱序处理器增加更多执行单元会带来收益递减。另一个在第8章讨论的问题是，内存速度远慢于处理器。大多数加载和存储操作会访问一个更小、更快的内存，称为*cache*。然而，当指令或数据在缓存中不可用时，处理器可能会在从主存检索信息时停顿100个或更多周期。多线程技术有助于让拥有多个执行单元的处理器保持忙碌，即使程序的ILP较低或程序因等待内存而停顿。

为了解释多线程，我们需要定义几个新术语。在计算机上运行的程序称为*process*。计算机可以同时运行多个进程；例如，你可以在个人电脑上一边播放音乐，一边浏览网页并运行病毒检查程序。每个进程由一个或多个同时运行的*threads*组成。例如，文字处理器可能有一个线程处理用户输入，第二个线程在用户工作时对文档进行拼写检查，第三个线程打印文档。这样，用户无需等待文档打印完成才能继续输入。一个进程能够被拆分为多个可同时运行的线程的程度，定义了其 *thread-level parallelism*（TLP）的级别。

在传统处理器中，线程仅提供同时运行的假象。实际上，线程在操作系统（OS）的控制下轮流在处理器上执行。当一个线程的执行轮次结束时，操作系统会保存其架构状态，加载下一个线程的架构状态，并开始执行该线程。这一过程称为*context switching*。只要处理器在所有线程间切换得足够快，用户就会感知到所有线程在同时运行。RISC-V将其32个寄存器之一——线程指针寄存器tp（即x4）——专门用于指向（保存地址）线程的本地内存。

一个 *hardware multithreaded* 处理器包含其架构状态的多个副本，以便同时可以有多个线程处于活动状态。例如，如果我们将处理器扩展为拥有四个程序计数器和128个寄存器，那么一次就可以有四个线程可用。如果一个线程在等待主存数据时发生停顿，处理器可以立即切换到另一个线程而无需任何延迟，因为程序计数器和寄存器已经就绪。此外，如果某个线程缺乏足够的并行性来让超标量设计中的所有执行单元保持忙碌，那么另一个线程可以向空闲单元发出指令。线程之间的切换可以是细粒度的，也可以是粗粒度的。*Fine-grained* 多线程在每个指令上切换线程，并且必须由硬件多线程支持。*Coarse-grained* 多线程仅在发生代价高昂的停顿（例如由于缓存未命中导致的长内存访问）时才会切换出线程。

多线程并不会提升单个线程的性能，因为它并未增加指令级并行度（ILP）。然而，它确实能提高处理器的整体吞吐量，因为多个线程可以利用原本在单线程执行时会闲置的处理器资源。此外，多线程的实现成本相对较低，因为它只需复制程序计数器（PC）和寄存器文件，而无需复制执行单元和存储器。

### 7.7.8 多处理器

*With contributions from Matthew Watkins*

现代处理器拥有数量庞大的晶体管。利用这些晶体管来增加流水线深度，或为超标量处理器添加更多执行单元，不仅性能提升甚微，还会浪费功耗。大约在2005年，计算机架构师们做出了重大转变，开始在一个芯片上构建多个处理器副本；这些副本被称为 *cores*。

一个 *multiprocessor* 系统由多个处理器以及处理器之间的通信方法组成。三类常见的多处理器包括 *symmetric*（或 *homogeneous*）多处理器、*heterogeneous* 多处理器和 *clusters*。

#### 对称多处理器

对称多处理器包含两个或更多共享单一主存的相同处理器。这些多个处理器可以是独立的芯片，也可以是同一芯片上的多个核心。

多处理器可用于同时运行更多线程，或使特定线程运行得更快。同时运行更多线程很容易，只需将线程分配到各个处理器即可。然而，典型的个人电脑用户通常只需要运行少量



在任何给定时间，线程的数量是固定的。要更快地运行某个特定线程则更具挑战性。程序员必须将现有线程拆分为多个线程，以便在每个处理器上执行。当处理器之间需要相互通信时，这一过程变得尤为复杂。计算机设计者和程序员面临的主要挑战之一，是如何有效利用大量的处理器核心。

对称多处理器具有多项优势。其设计相对简单，因为处理器只需设计一次，随后可通过多次复制来提升性能。在对称多处理器上编写和执行代码也较为直接，因为任何程序都可在系统中的任意处理器上运行，并达到大致相同的性能。

#### 异构多处理器

遗憾的是，持续增加更多对称核心并不能保证性能的持续提升。大多数消费类应用在任意时刻仅使用少量线程，而消费者通常只有少数几个应用同时进行计算。尽管这足以让双核和四核系统保持忙碌，但除非程序开始引入显著更高的并行度，否则在此基础上继续增加核心将带来递减的收益。此外，由于通用处理器旨在提供良好的平均性能，它们在执行特定操作时通常并非最高能效的选择。这种能效低下的问题在手机等高度受限功耗的系统中尤为突出。

异构多处理器旨在通过在同一系统中集成不同类型的核心和/或专用硬件来解决这些问题。每个应用程序会使用为其提供最佳性能或最佳能效比的资源。由于如今晶体管相当充裕，并非每个应用程序都会用到每一块硬件这一事实已不那么令人担忧。

异构系统可以采取多种形式。异构系统可以集成具有相同架构但不同微架构的核心，每种核心在功耗、性能和面积方面各有不同的权衡。RISC-V架构在设计之初就旨在支持从低成本嵌入式处理器到高性能多处理器的多种处理器实现。另一种异构策略是加速器，即系统中包含针对特定类型任务在性能或能效方面优化的专用硬件。例如，当前的移动片上系统（SoC）可能包含用于图形处理、视频、无线通信、实时任务和加密的专用加速器。这些加速器的效率（在性能、成本方面）可提升10到100倍。

对于相同任务，其面积效率高于通用处理器。数字信号处理器是另一类加速器，这些处理器拥有针对数学密集型任务优化的专用指令集。

异构系统并非没有缺点。它们在设计不同异构元素以及决定何时及如何利用各种资源所需的额外编程工作方面增加了复杂性。对称系统和异构系统在现代系统中各有其用武之地。对称多处理器适用于拥有大量线程级并行性的场景，例如大型数据中心。而异构系统则更适合工作负载变化较大或具有特殊用途的系统，比如移动设备。

### 集群

与其他多处理器不同，*clustered*多处理器系统中的每个处理器都拥有自己的本地内存系统，而非共享内存。集群的一种类型是由网络连接的一组个人计算机，它们运行软件共同解决大型问题。计算机之间通过消息传递而非共享内存进行通信。一种日益重要的大规模计算机集群是 *data center*，也称为 *warehouse-scale computers* (WSC)，其中成排的计算机和磁盘通过网络连接，并共享电源和冷却系统。此类系统通常包含5万至10万台计算机，即 *servers*，成本达1.5亿美元。<sup>5</sup>包括谷歌、亚马逊和Facebook在内的主要互联网公司推动了数据中心的快速发展，以支持全球数百万用户。这类集群的一大优势在于，可根据需要因故障或升级而更换单台计算机。

近年来，各公司拥有的传统服务器正被 *cloud computing* 取代，小型企业从谷歌、亚马逊等公司租用WSC的一部分。同样，应用程序不再完全运行于智能手机或平板电脑等手持设备（通常称为 *personal mobile device* (PMD)）上，部分应用可能运行在云端以加速计算并提高数据存储效率。这被称为 *software as a service* (SaaS)。SaaS的一个常见示例是网络搜索，其数据库存储在WSC上。租用云或网络服务的公司既要求隐私（防止云端其他软件运行带来的风险），也要求性能。这两者均通过虚拟机实现，虚拟机模拟整个计算机（包括其操作系统），运行在可能自身正在运行

---

<sup>5</sup> D. Patterson and J. Hennessy, *Computer Organization and Design, The Hardware-Software Interface: RISC-V Edition*, Morgan Kaufmann, © 2018.

不同的操作系统。多个虚拟机可以同时运行在一台物理机器上，内存和I/O等资源按时间进行分区或共享。这使得亚马逊云服务（AWS）等提供商能够高效利用物理资源，在虚拟机之间提供保护，并将虚拟机从无法工作或性能低下的计算机上迁移出去。*hypervisor*，也称为*virtual machine monitor*（VMM），是运行虚拟机并将虚拟资源映射到物理资源的软件。虚拟机监控程序执行通常由操作系统执行的功能，例如管理I/O、CPU资源和内存。虚拟机监控程序运行在*host*（底层物理硬件平台）与其模拟的操作系统之间。允许虚拟机监控程序直接在硬件上运行（而非通过软件）的指令集架构称为*virtualizable*。这使得虚拟机更高效、性能更高。可虚拟化架构的示例包括x86（自2005年起）、RISC-V和IBM 370。ARMv7和MIPS架构不可虚拟化，但ARM在2013年引入ARMv8时确实增加了虚拟化扩展。

云计算也是物联网（IoT）应用的关键组成部分，其中扬声器、手机和传感器等设备通过蓝牙或Wi-Fi等网络进行连接。物联网应用的示例包括通过蓝牙将耳机连接到智能手机，或通过Wi-Fi连接Alexa或Siri。低成本设备（耳机、用于Google Assistant的Google Home或用于Alexa的Echo Dot）通过网络连接到更高性能的服务器，这些服务器可以流式传输音乐，或者在Siri和Alexa的情况下，执行语音识别、查询数据库并进行计算。

## 7.8 现实视角：RISC-V 微架构的演进\*

本节追溯了自2010年RISC-V诞生以来其微架构的发展历程。由于基础指令集直到2017年才被完整描述，许多RISC-V芯片仍在开发中，截至2021年仅少数已上市。但随着支持工具和开发周期的成熟，这一局面预计将迅速改变。

大多数现有的处理器实现都采用低端或嵌入式处理器，但高性能芯片即将问世。RISC-V国际组织（[riscv.org](http://riscv.org)）提供了不断增长的处理器和SoC平台列表。RISC-V商用核心出现在SiFive的HiFive开发板、西部数据硬盘以及NVIDIA GPU等产品中。

截至2021年，两款知名的商用RISC-V处理器分别是SiFive的Freedom E310核心和西部数据的开源SweRV核心（该核心有三个版本）。Freedom E310是一款低成本

尽管RISC-V是一种开源架构，而非微架构，但许多开源硬件实现正在涌现，包括西部数据的SweRV内核、SweRVolf SoC以及PULP（并行超低功耗）平台。不断增长的RISC-V硬件实现及支持工具被称为RISC-V生态系统。关于如何访问和使用其中一些开源工具与硬件的信息，请参阅前言。

SiFive的HiFive和Sparkfun的RED-V开发板中使用的嵌入式处理器。它运行RV32IMAC（包含乘除法[M]、原子内存访问[A]和压缩指令[C]扩展的RV32I），并拥有8 KB程序存储器、8 KB用于启动代码的掩膜ROM、16 KB数据SRAM以及一个16 KB两路组相联指令缓存。它还包含JTAG、SPI、I2C和UART接口，以及一个QSPI闪存接口。该处理器运行频率为320 MHz，是一个单发射、顺序执行的核心，具有本章描述的相同阶段的5级流水线。图7.73展示了HiFive 1 Rev B板上FE310-G002处理器的框图。

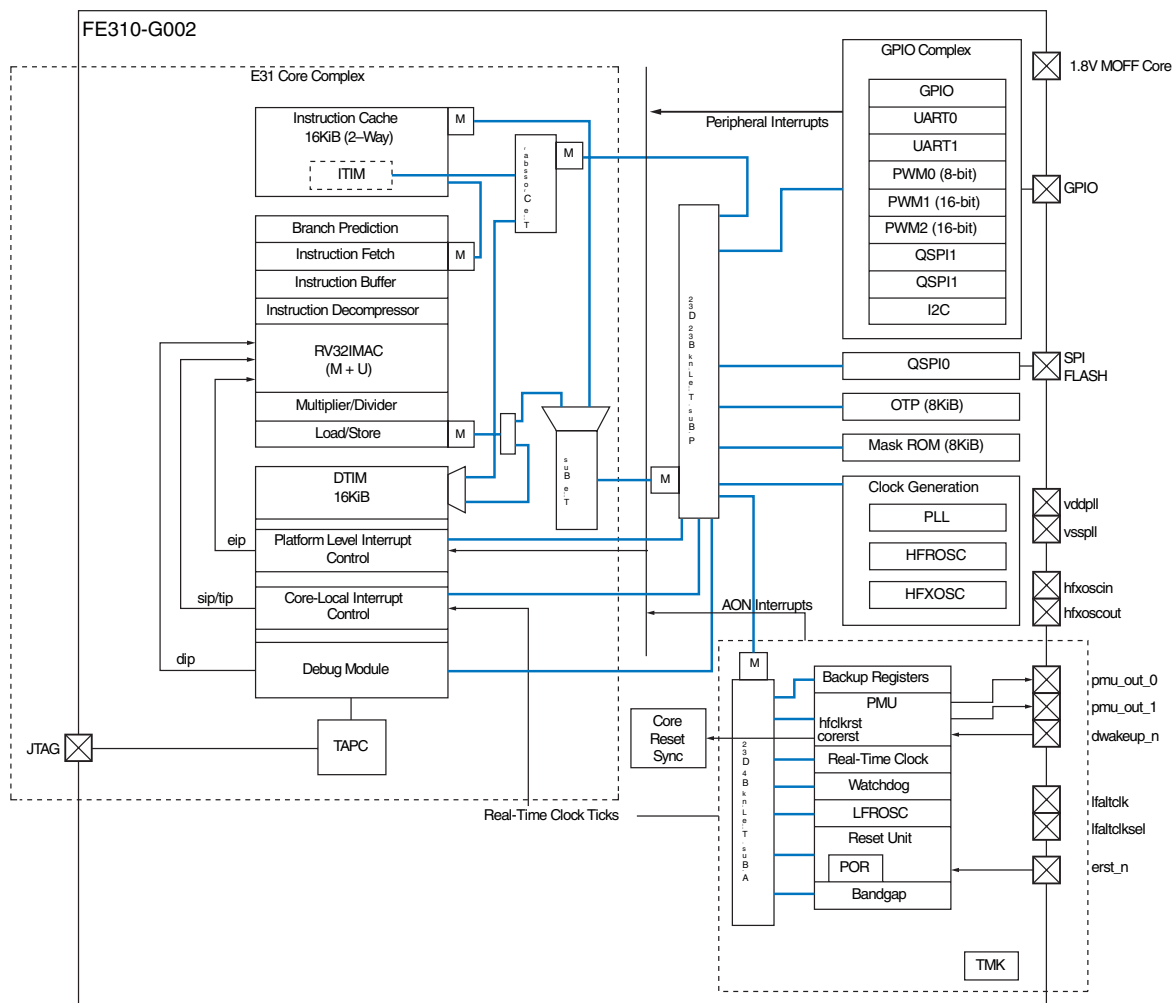


图7.73 Freedom E310-G002 框图（由SiFive Inc.提供，*SiFive FE310-G002 Preliminary Datasheet v1p0*，©2019）

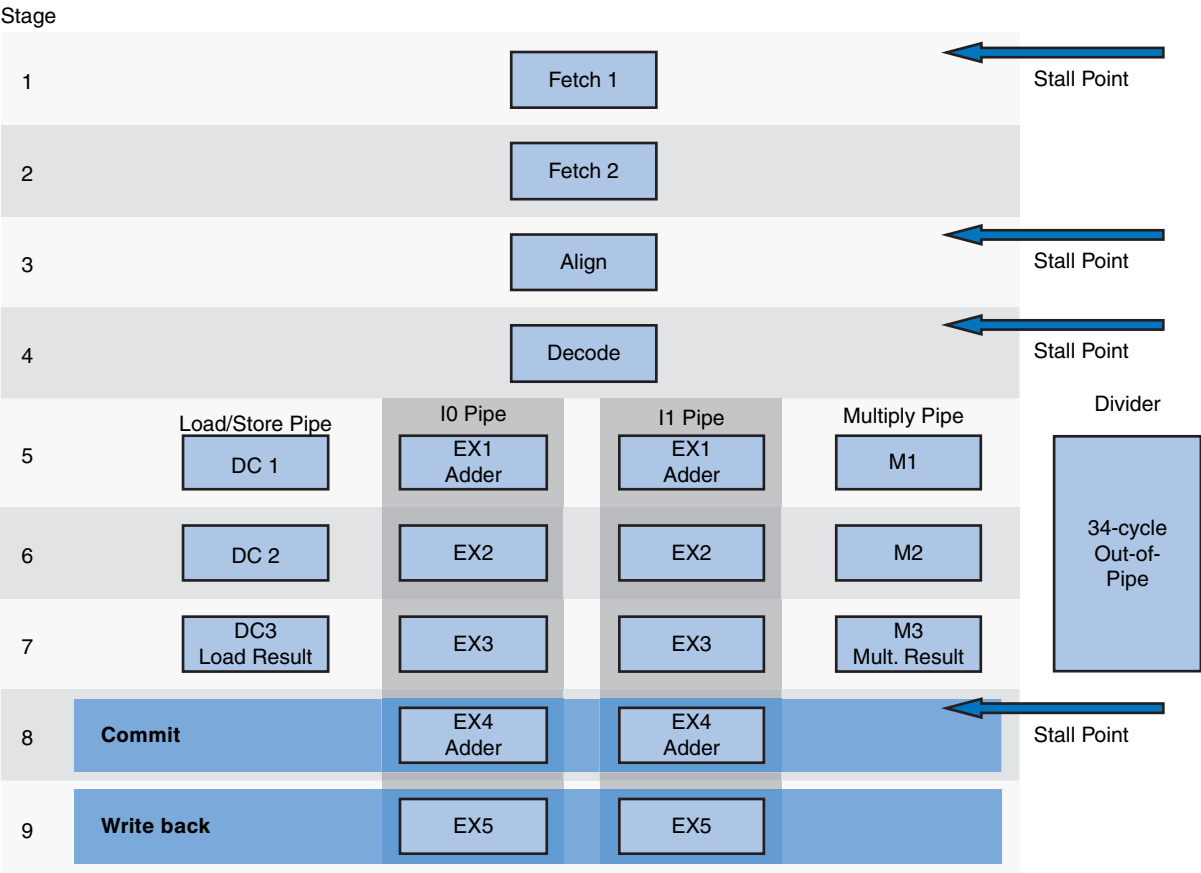


图7.74 SweRV EH1 9级流水线  
(由西部数据公司提供, RISC-V SweRV™ EH1 Programmer's Reference Manual, ©2020)

西部数据SweRV内核提供三种开源版本：EH1、EH2和EL2。EH1是一款32位双发射超标量内核，采用九级流水线并支持部分乱序执行。这些内核实现RV32IMC指令集，包含32位基础指令集以及压缩（C）和乘除（M）扩展。采用28纳米芯片制造工艺时，目标频率可达1 GHz，其硬件描述语言也可综合至FPGA。EH2内核在EH1基础上增加了双线程支持。EL2内核则是面向嵌入式系统的低性能处理器。图7.74展示了EH1的九个流水线阶段，从两个取指阶段、一个对齐阶段和一个译码阶段开始。译码阶段最多可解码两条指令。此后，流水线

The source code (SystemVerilog) for Western Digital's open-source cores is available for download at <https://github.com/chipsalliance/Cores-SweRV>

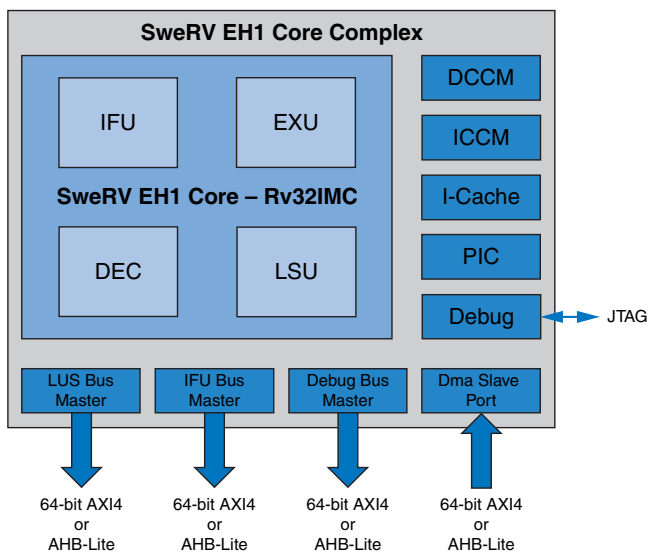


图7.75 SweRV EH1内核复合体（由西部数据公司提供，RISC-V SweRV™ EH1 Programmer's Reference Manual, ©2020）

将流水线分为五条并行路径：一条加载/存储路径、两条整数指令路径（如加法、减法、异或）、一条乘法路径以及一条除法路径——由于34个周期的延迟，该路径位于流水线之外。最后两个流水线阶段是提交阶段和写回阶段。提交阶段因乱序执行和存储缓冲区而必需。最终阶段，即写回阶段，在需要时将结果写回寄存器文件。

图7.75展示了SweRV EH1核心复合体的框图。它包括处理器（图中标注为SweRV EH1核心）、指令缓存（I-Cache）、数据与指令存储器（DCCM和ICCM——数据与指令紧耦合存储器）、可编程中断控制器（PIC）、JTAG调试器接口，以及可配置为AXI4或AHB-Lite总线的存储器/调试接口。处理器由指令获取单元（IFU）、解码单元（DEC）、执行单元（EXU）和加载/存储单元（LSU）组成。IFU包含流水线获取阶段；DEC包含对齐和解码阶段；EXU包含除加载/存储流水线之外的所有其他阶段，而加载/存储流水线位于加载/存储单元（LSU）中。该系统包含一个4路组关联指令缓存，可配置为16至256 KiB。DCCM和ICCM被称为 *closely coupled* 存储器，因为它们是低延迟的片上存储器，可配置为4至512 KiB。



Robert Golla 是西部数据（Western Digital）的高级研究员，曾负责西部数据 EH1、EL2 和 EH2 RISC-V 开源嵌入式处理器内核的架构设计。他还主导了 Oracle T4、M7、M8 和 M9 乱序执行内核、Sun N2 多线程内核以及摩托罗拉嵌入式 e500 内核的架构设计，并为 Cyrix 下一代 x86 M3、恩智浦低功耗 603 和 603e、IBM POWER1 和 POWER2 微处理器做出了贡献。他拥有 50 多项与微处理器设计相关的专利。

Imagination Technologies 推出的 RISC-V FPGA (RVfpga) 课程展示了如何将 SweRV EH1 内核部署到 FPGA 上，以及如何在 RISC-V 内核上编译和运行程序。该免费课程还提供实验和练习，演示如何扩展内核以添加外设、如何理解和修改流水线、超标量执行、分支预测器和存储层次结构，以及如何使用定时器、中断和性能计数器等功能。这些实验和材料可从 Imagination Technologies 大学计划免费获取，网址为 <https://university.imgtec.com/rvfpga/>。

## 7.9 总结

本章描述了构建处理器的三种方式，每种方式在性能、面积和成本上各有不同的权衡。我们认为这一主题近乎神奇：像微处理器这样看似复杂的设备，如何能简单到仅用半页原理图就能呈现？此外，其内部运作对未入门者而言虽显神秘，但实际上却相当直接明了。

微架构几乎涵盖了本文到目前为止所讨论的所有主题。将微架构的各个部分拼凑起来，展示了前几章介绍的原则，包括组合电路和时序电路的设计（涵盖在第1至第3章）、许多构建模块的应用（在第5章中描述），以及RISC-V架构的实现（在第6章中介绍）。利用第4章的技术，微架构可以用几页HDL代码来描述。

微架构的构建也高度依赖于我们管理复杂性的技术。微架构抽象构成了逻辑抽象与架构抽象之间的桥梁，成为这本关于数字设计与计算机架构的书籍的核心。我们还利用框图抽象和硬件描述语言来简洁地描述组件的布局。微架构利用规律性和模块化，复用常见构建模块库，如算术逻辑单元、存储器、多路选择器和寄存器。层次结构以多种方式被应用。微架构被划分为数据通路和控制单元，而这些单元本身又被划分为更小的单元。每个单元由逻辑块构建，逻辑块可由门电路构建，门电路又可由晶体管构建——所有这些都运用了前五章中开发的技术。

本章比较了RISC-V处理器的单周期、多周期和流水线微架构。这三种微架构都实现了RISC-V指令集的相同子集，并具有相同的架构状态。单周期处理器最为直接，其CPI为1。

多周期处理器使用可变数量的较短步骤来执行指令。因此，它可以复用ALU，而不需要多个加法器，并包含统一的内存。然而，它确实需要多个非架构寄存器来在步骤之间存储结果。多周期设计原则上可能更快，因为并非所有指令都必须等长。但在实践中，它通常更慢，因为它受限于最慢的步骤以及每个步骤中的顺序开销。

流水线处理器将单周期处理器划分为五个相对较快的流水线阶段。它在各阶段之间添加流水线寄存器，以分隔同时执行的五条指令。

执行中。其名义CPI为1，但冒险会导致停顿或冲刷，使CPI略有增加。冒险解决也需要额外的硬件和设计复杂度。理想情况下，时钟周期可比单周期处理器缩短五倍。实际上，由于受最慢阶段和每个阶段的时序开销限制，时钟周期并未如此之短。尽管如此，流水线化仍带来了显著的性能提升。如今所有现代高性能微处理器均采用流水线技术。

尽管本章中的微架构仅实现了RISC-V架构的一个子集，但我们已看到，支持更多指令需要对数据通路和控制器进行直接的增强。

本章的一个主要局限是，我们假设了一个理想的内存系统，它足够快且足够大，能够存储整个程序和数据。实际上，大容量高速内存的成本高得令人望而却步。下一章将展示如何通过使用一个保存最常用信息的小容量高速内存，以及一个或多个更大但更慢的内存来保存其余信息，从而获得大容量高速内存的大部分优势。



## 练习

练习7.1 假设单周期RISC-V处理器中的以下某个控制信号存在恒为0故障，即该信号始终为0，无论其预期值如何。哪些指令会出现故障？为什么？请使用图7.15和图7.16所示的扩展版单周期处理器。

(a) *RegWrite* (b)  
) *ALUOp<sub>1</sub>* (c)  
*ALUOp<sub>0</sub>* (d)  
*MemWrite* (e)  
*ImmSrc<sub>1</sub>* (f)  
*ImmSrc<sub>0</sub>* (g)  
*ResultSrc<sub>1</sub>* (h)  
*ResultSrc<sub>0</sub>* (i)  
*PCSrc* (j)  
*ALUSrc*

练习7.2 重复练习7.1，假设信号存在固定为1的故障。

练习7.3 修改单周期RISC-V处理器以实现以下指令之一。指令定义见附录B。在图7.15的副本上标注数据通路的变化。命名所有新的控制信号。在表7.3和表7.6的副本上标注ALU译码器和主译码器的变化。同时，根据需要标注ALU、ALU译码器和主译码器图（见图7.16）的任何变化。描述所需的任何其他变化。

(a) 异或  
(b) 逻辑  
左移 (c)  
逻辑右  
移 (d) 不  
等时跳  
转

练习7.4 对以下RISC-V指令重复练习7.3。

(a) *lui* (b)  
) *sra* (c) *l*  
*bu*

(d) blt (e) b  
 ltu (f) bge (  
 g) bgeu (h)  
 jalr (i) auip  
 c (j) sb (k)  
 slli (l) srai

练习7.5 扩展RISC-V指令集以包含lwpostinc，该指令执行后索引操作。lwpostinc rd, imm(rs)等价于以下两条指令：

```
lw rd, 0(rs) addi rs, rs,
imm
```

对使用后索引的lwpostinc重复练习7.3。

练习7.6 扩展RISC-V指令集以包含lwpreinc，该指令执行预索引。lwpreinc rd, imm(rs)等价于以下两条指令：

```
lw rd, imm(rs) addi rs, rs, imm
```

对lwpreinc重复练习7.3。

练习7.7 你的朋友是一位顶尖的电路设计师。她提出重新设计单周期RISC-V处理器中的一个单元，使其延迟减半。使用第415页表7.7中的延迟数据，她应该改进哪个单元才能获得最大的整体处理器加速比？改进后的机器周期时间是多少？请解释原因。

练习7.8 考虑表7.7（第415页）中给出的延迟。Ben Bitdiddle构建了一个前缀加法器，将ALU延迟减少了20皮秒。如果其他元件延迟保持不变，请找出单周期RISC-V处理器的新周期时间，并确定执行一个包含1000亿条指令的基准测试需要多长时间。

练习7.9 修改第7.6节中给出的单周期RISC-V处理器的HDL代码，以处理练习7.3中的一条新指令。增强

使用第7.6.3节中给出的测试平台和测试程序（`riscvtest.s`和`riscvtest.txt`）来测试新指令。添加注释以指示任何更改。

练习7.10 对练习7.4中的新指令重复练习7.9。

练习7.11 假设多周期RISC-V处理器中的某个控制信号存在固定为0故障，即该信号始终为0，无论其预期值如何。哪些指令会出现故障？为什么？请参考图7.27和图7.45所示的多周期数据通路和控制逻辑。

(a) *ResultSrc<sub>1</sub>* (  
 b) *ResultSrc<sub>0</sub>* (c  
 ) *ALUSrcB<sub>1</sub>* (d)  
*ALUSrcB<sub>0</sub>* (e)  
*ALUSrcA<sub>1</sub>* (f)  
*ALUSrcA<sub>0</sub>* (g)  
*ImmSrc<sub>1</sub>* (h)  
*ImmSrc<sub>0</sub>* (i)  
*RegWrite* (j)  
*PCUpdate* (k)  
*Branch* (l)  
*AdrSrc* (m)  
*MemWrite* (n)  
*IRWrite*

练习7.12 重复练习7.11，假设每个信号存在固定为1的故障。

练习7.13 修改多周期RISC-V处理器以实现以下指令之一。指令定义见附录B。命名所有新的控制信号。在图7.27的副本上标注数据通路的更改，在图7.45的副本上标注控制器有限状态机的更改。描述所需的任何其他更改。

(a) 异或  
 (b) 逻辑  
 左移 (c)  
 逻辑右  
 移 (d) 不  
 等时跳  
 转

练习7.14 对以下RISC-V指令重复练习7.13。

(a) lui (b) s  
ra (c) lbu (  
d) blt (e) b  
ltu (f) bge  
(g) bgeu (h  
) jalr (i) au  
ipc (j) sb (  
k) slli (l) sr  
ai

练习7.15 针对多周期RISC-V处理器重复练习7.5。展示多周期数据通路和控制有限状态机的变化。是否可以在不修改寄存器文件的情况下添加该指令？如果可以，请说明如何实现。

练习7.16 针对多周期RISC-V处理器重复练习7.6。展示多周期数据通路和控制有限状态机的变化。是否可以在不修改寄存器文件的情况下添加该指令？如果可以，请说明如何实现。

练习7.17 对多周期RISC-V处理器重复练习7.7。

练习7.18 对多周期RISC-V处理器重复练习7.8。使用来自示例7.7的指令混合。

练习7.19 你的朋友，一位顶尖的电路设计师，提议重新设计多周期RISC-V处理器中的一个单元，使其速度大幅提升。根据第415页表7.7中的延迟数据，她应该改进哪个单元才能获得最大的整体处理器加速？该单元应达到多快？（使其比必要速度更快会浪费你朋友的努力。）改进后处理器的周期时间是多少？请解释并展示你的计算过程。

练习7.20 Goliath公司声称拥有一项关于三端口寄存器文件的专利。为了避免与Goliath对簿公堂，Ben Bitdiddle设计了一种新的寄存器文件，该文件只有一个读/写端口（类似于组合指令与数据存储器）。请重新设计RISC-V多周期数据通路和控制器，以使用他的新型寄存器文件。

练习7.21 假设多周期RISC-V处理器具有表7.7（第415页）中给出的组件延迟。Al yssa P. Hacker设计了一种新的寄存器文件，其功耗降低40%，但延迟增加一倍。她是否应该在她的多周期处理器设计中改用这种更慢但功耗更低的寄存器文件？请解释原因。

练习7.22 重新设计的多周期RISC-V处理器（来自练习7.20）的CPI是多少？使用示例7.7中的指令混合比例。

练习7.23 在多周期RISC-V处理器上运行以下程序需要多少个时钟周期？该程序的CPI是多少？

```
addi s0, zero, 5    # 结果 = 5
L1: bge zero, s0, Done # 如果结果 <= 0
    , 退出循环      addi s0, s0, -1    # 结果 = 结果 - 1    j    L1 Done:
```

练习7.24 对以下程序重复练习7.23。

```
addi s0, zero, 0    # i = 0
addi s1, zero, 0    # 和 = 0
addi t3, zero, 10   # t3 = 10
循环: beq s0, t3, L2 # 如果 i == 10, 跳转到 L2    add
s1, s1, s0          # 和 = 和 + i    addi s0, s0, 1    # i = i + 1    j
循环 L2:
```

练习7.25 为多周期RISC-V处理器编写HDL代码，并将模块命名为riscvmulti。该处理器应支持本章所述指令：lw、sw、add、sub、and、or、slt、addi、andi、ori、slli、beq和jal。处理器应与以下顶层模块兼容。mem模块用于同时存储指令和数据。请记住，可以使用第7.6节单周期处理器HDL中的构建模块。使用第7.6.3节的测试平台和测试程序（riscvtest.s和riscvtest.txt）测试您的处理器。添加注释以标明任何更改。

```
模块 top(输入 逻辑    clk, reset,          输出逻辑 [31:0] WriteData, DataAdr,
输出逻辑    MemWrite); 逻辑 [31:0] ReadData; // 实例化处理器和存储器
riscvmulti rvmulti(clk, reset, MemWrite, DataAdr,          WriteData, ReadD
ata);  mem mem(clk, MemWrite, DataAdr, WriteData, ReadData); endmodule
```

练习7.26 将练习7.25中多周期RISC-V处理器的HDL代码扩展，以处理练习7.14中的一条新指令。增强第7.6.3节中的测试平台和测试程序（riscvtest.s和riscvtest.txt），以测试新指令。添加注释以标明所有更改。

练习7.27 对来自以下的新指令之一重复练习7.26  
练习7.13。

练习7.28 流水线RISC-V处理器正在运行以下代码片段。在第五个周期中，哪些寄存器被写入，哪些被读取？请记住，流水线RISC-V处理器有一个冒险单元。你可以假设内存系统在一个周期内返回结果。

```
addi s1, zero, 11 # s1 = 11 lw s2, 25(s0) # s2 = mem
ory[s0+25] add s3, s3, s4 # s3 = s3 + s4 or s4, s1, s
2 # s4 = s1 | s2 lw s5, 16(s2) # s5 = memory[s2+1
6]
```

练习7.29 对以下RISC-V代码重复练习7.28

nippet.

```
异或 s1, s2, s3 # s1 = s2 ^ s3 加立即数 s3, -4 #
s0 = s3 - 4 加载字 s3, 16(s7) # s3 = 内存[s7+16]
存储字 s4, 20(s1) # 内存[s1+20] = s4 或 s0, s1
# t2 = s0 | s1
```

练习7.30 对以下RISC-V代码片段重复练习7.28。

```
addi s1, zero, 11 # s1 = 11 lw s2, 25(s1) # s2 = mem
ory[36] lw s5, 16(s2) # s5 = memory[s2+16] add s
3, s2, s5 # s3 = s2 + s5 or s4, s3, t4 # s4 = s3 | t4
and s2, s3, s4 # s2 = s3 & s4
```

练习7.31 对以下RISC-V代码片段重复练习7.28。

```
addi s1, zero, 52 # s1 = 52 addi s0, s1, -4 # s0 = s
1 - 4 = 48 lw s3, 16(s0) # s3 = memory[64] sw
s3, 20(s0) # memory[68] = s3 xor s2, s0, s3 # s2
= s0 ^ s3 or s2, s2, s3 # s2 = s2 | s3
```

练习7.32 使用类似于图7.57的图表，展示在流水线RISC-V处理器上执行练习7.30中的指令所需的转发和停顿。

练习7.33 对练习7.31中的指令重复练习7.32。

练习7.34 流水线RISC-V处理器执行练习7.30程序中的所有指令需要多少个时钟周期？该处理器在此程序上的CPI是多少？

练习7.35 对练习7.31中程序的指令重复练习7.34。

练习7.36 解释如何扩展流水线RISC-V处理器以处理加载高位立即数指令lui。命名任何新的控制信号。在图7.61的副本上标记数据通路的更改，并在表7.3和7.6上标记ALU解码器和主解码器的更改。描述所需的任何其他更改。

练习7.37 对xor指令重复练习7.36。

练习7.38 如果分支在译码阶段而非执行阶段发生，流水线处理器的性能可能会更好。请说明如何修改图7.61中的流水线处理器，将分支逻辑移至译码阶段。停顿、冲刷和转发信号将如何变化？重新完成示例7.9和7.10，以计算新的CPI、周期时间以及执行程序的总时间。

练习7.39 你的朋友，一位顶尖的电路设计师，提出要重新设计流水线RISC-V处理器中的一个单元，使其速度大幅提升。根据第415页表7.7中的延迟数据，她应该改进哪个单元才能获得最大的整体处理器加速？该单元需要达到多快？（使其比必要速度更快会浪费你朋友的努力。）改进后处理器的周期时间是多少？解释你的答案并展示计算过程。

练习7.40 考虑表7.7（第415页）中的延迟。现在假设ALU的速度提高20%。流水线RISC-V处理器的周期时间会改变吗？如果ALU的速度降低20%呢？解释你的答案并展示你的计算过程。

习题7.41 假设RISC-V流水线处理器分为10级，每级400 ps（包含时序开销）。采用例7.7的指令混合比例。同时假设50%的加载指令后紧跟着一条使用其结果的指令，需要插入6个停顿周期，且30%的分支预测错误。分支指令的目标地址直到第二级结束时才计算得出。针对该10级流水线处理器，计算处理SPECINT2000基准测试中1000亿条指令的平均CPI和执行时间。

练习7.42 为流水线RISC-V处理器编写HDL代码，并将模块命名为riscv。该处理器应与下面的顶层模块兼容，并支持本章所述的指令：lw、sw、add、sub、and、or、slt、addi、andi、ori、slti、beq和jal。请注意，你可以使用第7.6节单周期处理器HDL中的构建模块。修改第7.6.3节的测试平台，使用该节中的测试程序（riscvtest.s和riscvtest.txt）来测试你的处理器。

```
模块 top(输入 逻辑 clk, reset,      输出 逻辑 [31:0] WriteDataM, DataAdrM,
输出 逻辑 MemWriteM);  逻辑 [31:0] PCF, InstrF, ReadDataM;  // 实例化处理器和
存储器   riscv riscv(clk, reset, PCF, InstrF, MemWriteM, DataAdrM,      WriteDa
taM, ReadDataM);  imem imem(PCF, InstrF);  dmem dmem(clk, MemWriteM, DataA
drM, WriteDataM, ReadDataM); endmodule
```

练习7.43 将习题7.42中流水线RISC-V处理器的HDL扩展，以处理习题7.37中的xor指令。修改第7.6.3节中的测试平台和测试程序（riscvtest.s和riscvtest.txt），以测试增强后的处理器。

练习7.44 将习题7.42中流水线RISC-V处理器的HDL扩展，以处理习题7.36中的lui指令。修改第7.6.3节中的测试平台和测试程序（riscvtest.s和riscvtest.txt），以测试增强后的处理器。

练习7.45 为流水线RISC-V处理器设计图7.61所示的风险单元。使用HDL实现你的设计。画出综合工具可能从你的HDL生成的硬件草图。

练习7.46 展示如何修改RISC-V多周期处理器，使其在遇到未定义指令时触发异常。异常的描述见第6.6.2节。未定义指令的原因码为2（参见第357页表6.6）。根据需要修改数据通路（图7.27）和控制单元，包括主有限状态机（图7.45）。可以假设mtvec已写入异常处理程序地址。

练习7.47 对于错位加载异常（其原因为代码4，见表6.6），重复练习7.46。

练习7.48 展示如何修改RISC-V多周期处理器以实现特权指令csrrw（CSR读/写）。图7.76显示了



| Assembly                            | Field Values        |        |        |        |        | Machine Code        |        |        |        |          |              |
|-------------------------------------|---------------------|--------|--------|--------|--------|---------------------|--------|--------|--------|----------|--------------|
|                                     | imm <sub>11:0</sub> | rs1    | funct3 | rd     | op     | imm <sub>11:0</sub> | rs1    | funct3 | rd     | op       |              |
| <code>cssrw x9, mscratch, x8</code> | 0x340               | 8      | 1      | 9      | 115    | 0011 0100 0000      | 01000  | 001    | 01001  | 111 0011 | (0x340414F3) |
|                                     | 12 bits             | 5 bits | 3 bits | 5 bits | 7 bits | 12 bits             | 5 bits | 3 bits | 5 bits | 7 bits   |              |

图7.76 特权指令cssrw（CSR读/写）

csrrw x9, mscratch, x8 的汇编与机器码同时将 mscratch 复制到 x9，并将 x8 复制到 mscratch。CSR 编号编码在 I 型指令的 12 位立即数字段中。mscratch 的 CSR 编号为 0x340。任何 12 位的 CSR 编号都应能够被读取/写入。有关特权指令格式的更多信息，请参见附录 B 中的表 B.8。根据需要修改数据通路（图 7.27）和控制单元（包括主有限状态机，图 7.45），以支持 csrrw 指令。

练习7.49 将练习7.48中的特权指令csrrs（CSR读取并设置）重复一遍。图7.77展示了 csrrs x7, mcause, x3的汇编代码和机器码，该指令同时将mcause复制到x7，并将(mcause | x3)写入mcause。mcause的CSR编号为0x342。任何12位的CSR编号都应能被读取/设置。

| Assembly                          | Field Values        |        |        |        |        | Machine Code        |        |        |        |          |              |
|-----------------------------------|---------------------|--------|--------|--------|--------|---------------------|--------|--------|--------|----------|--------------|
|                                   | imm <sub>11:0</sub> | rs1    | funct3 | rd     | op     | imm <sub>11:0</sub> | rs1    | funct3 | rd     | op       |              |
| <code>csrrs x7, mcause, x3</code> | 0x342               | 3      | 2      | 7      | 115    | 0011 0100 0010      | 00011  | 010    | 00111  | 111 0011 | (0x3421A3F3) |
|                                   | 12 bits             | 5 bits | 3 bits | 5 bits | 7 bits | 12 bits             | 5 bits | 3 bits | 5 bits | 7 bits   |              |

图7.77 特权指令csrrs（CSR读取/设置）

## 面试问题

---

以下练习题展示了数字设计岗位面试中曾出现的问题。

问题7.1 解释流水线微处理器的优势。

问题7.2 如果增加流水线级数能让处理器运行更快，为什么处理器不采用100级流水线？

问题7.3 描述微处理器中的冒险是什么，并解释解决冒险的方法。每种方法的优缺点是什么？

问题7.4 描述超标量处理器的概念及其优缺点。

